

```

    setDragging(true);
    return true;
  }

  if (layout === undefined && ref) {
    // This page's text is still coming (the page was just switched to).
    // Hold the press and start it the moment the text is in, rather than
    // scrolling the page away from under the pen.
    pendingRef.current = { pointerId: e.pointerId, x0: e.clientX, y0: e.clientY, x: e.clientX, y: e.clientY
    };
    return true;
  }
  if (!layout || !box) {
    setSel(null);
    return false;
  }
  const w = p.getWorldPoint(e.clientX, e.clientY);
  const onPage = w.x >= box.x && w.x <= box.x + box.w && w.y >= box.y && w.y <= box.y + box.h;
  if (!onPage) {
    setSel(null);
    return false;
  }
  const at = offsetAtClient(e.clientX, e.clientY);
  if (at === null) return false;

  // Double tap: the word. Triple tap: the whole line.
  const count = again ? t.count + 1 : 1;
  tapsRef.current = { at: now, x: e.clientX, y: e.clientY, type: e.pointerType, count };
  if (count >= 2) {
    let range: [number, number];
    if (count === 2) {
      range = wordRangeAt(layout.text, at);
    } else {
      const run = layout.runs.find((r) => at >= r.start && at <= r.end) ?? layout.runs[0];
      const line = layout.runs.filter((r) => r.line === run.line);
      range = [line[0].start, line[line.length - 1].end];
    }
    dragRef.current = null;
    setDragging(false);
    setSel(range[1] > range[0] ? { start: range[0], end: range[1] } : null);
    return true;
  }

  dragRef.current = { pointerId: e.pointerId, anchor: at, startX: e.clientX, startY: e.clientY, moved: false
    , prev: sel };
  setDragging(true);
  return true;
};

const pointerMove = (e: React.PointerEvent<Element>): boolean => {
  const pend = pendingRef.current;
  if (pend && pend.pointerId === e.pointerId) {
    pend.x = e.clientX;
    pend.y = e.clientY;
    return true;
  }
  const d = dragRef.current;
  if (!d || d.pointerId !== e.pointerId) return false;
  if (!d.moved && Math.hypot(e.clientX - d.startX, e.clientY - d.startY) < 4) return true;
  d.moved = true;
  const at = offsetAtClient(e.clientX, e.clientY);
  if (at === null) return true;
  setSel(at === d.anchor ? null : { start: Math.min(at, d.anchor), end: Math.max(at, d.anchor) });
  return true;
};

const pointerUp = (e: React.PointerEvent<Element>): boolean => {
  if (pendingRef.current?.pointerId === e.pointerId) {
    pendingRef.current = null;
    return true;
  }
  const d = dragRef.current;
  if (!d || d.pointerId !== e.pointerId) return false;
  dragRef.current = null;
  setDragging(false);
  // A plain tap on text clears the selection, as in every reader.
  if (!d.moved && tapsRef.current.count < 2) setSel(null);
  return true;
};

```

```

};

/**
 * A second finger arrived (pinch) or the system took the pointer: whatever
 * the first finger dragged out on its way was not meant as a selection.
 */
const cancel = (pointerId?: number) => {
  // Only the pointer doing the drag can cancel it: a palm the system
  // cancels while the pen selects must leave the pen's selection alone.
  if (pointerId !== undefined && pendingRef.current?.pointerId === pointerId) pendingRef.current = null;
  if (pointerId === undefined) pendingRef.current = null;
  const d = dragRef.current;
  if (!d) return;
  if (pointerId !== undefined && d.pointerId !== pointerId) return;
  dragRef.current = null;
  setDragging(false);
  setSel(d.prev);
};

const text = layout && sel ? selectedText(layout, sel.start, sel.end) : "";

const doCopy = useCallback(async () => {
  if (!text) return;
  const copiedSel = sel;
  if (await copyToClipboard(text)) {
    setCopied(true);
    setTimeout(() => {
      setCopied(false);
      // Only the selection that was copied goes away - not one you started
      // in the moment since.
      setSel((cur) => (cur && copiedSel && cur.start === copiedSel.start && cur.end === copiedSel.end ? null
        : cur));
    }, 900);
  }
}, [text, sel]);

// Ctrl/Cmd+C copies the selection while there is one.
useEffect(() => {
  if (!active || !text) return;
  const onKey = (e: KeyboardEvent) => {
    if ((e.ctrlKey || e.metaKey) && (e.code === "KeyC" || e.key.toLowerCase() === "c")) {
      const tgt = e.target as HTMLInputElement | null;
      if (tgt && (tgt.isContentEditable || /^(INPUT|TEXTAREA)$/.test(tgt.tagName))) return;
      e.preventDefault();
      void doCopy();
    }
  };
  window.addEventListener("keydown", onKey);
  return () => window.removeEventListener("keydown", onKey);
}, [active, text, doCopy]);

const mark = (kind: MarkKind, ci = colorIndex) => {
  if (!layout || !sel || !box) return;
  const aspect = layout.page.h > 0 ? layout.page.w / layout.page.h : 1;
  const segs = lineSegments(rangeRects(layout, sel.start, sel.end), aspect).map((r: LineRect) => toWorld(r));
  const strokes = markStrokes(kind, segs, MARK_COLORS[ci], () => nanoid(8));
  if (strokes.length) p.addElements(strokes, true);
  setSel(null);
};

const pickColor = (ci: number) => {
  setColorIndex(ci);
  try {
    localStorage.setItem(COLOR_KEY, String(ci));
  } catch {}
  /* remembered for this session only */
  mark("highlight", ci);
};

const findOnPage = () => {
  const q = text.replace(/\s+/g, " ").trim().slice(0, 80);
  if (!q || !board) return;
  // The find bar opens on this query, goes to the first match from this
  // page on and lights up every match there (via wb:pdf-find).
  window.dispatchEvent(new CustomEvent("wb:open-search", { detail: { query: q } }));
  setSel(null);
};

```

```

};

// ---- world-space layer -----

const zoom = camera.zoom || 1;
const flashRects = useMemo(
  () =>
    layout && flash
      ? flash.ranges.flatMap(([s, e]) => rangeRects(layout, s, e).map(toWorld))
      : [],
  [layout, flash, toWorld]
);

const worldLayer =
  selWorldRects.length || flashRects.length ? (
    <g pointerEvents="none" data-pdf-text-layer="">
      {flashRects.map((r, i) => (
        <rect
          key={`f${flash?.key}-${i}`}
          x={r.x - 1.5 / zoom}
          y={r.y - 1 / zoom}
          width={r.w + 3 / zoom}
          height={r.h + 2 / zoom}
          rx={2 / zoom}
          fill="#f59e0b"
          stroke="#d97706"
          strokeWidth={1.5 / zoom}
          opacity={0}
        >
          <animate
            attributeName="opacity"
            values="0;0.6;0.45;0.6;0.45;0"
            keyTimes="0;0.08;0.3;0.5;0.8;1"
            dur={`${FLASH_MS}ms`}
            fill="freeze"
          />
        </rect>
      )
    )}
    {selWorldRects.map((r, i) => (
      <rect key={i} x={r.x} y={r.y} width={r.w} height={r.h} rx={1.5 / zoom} fill="#3b82f6" fillOpacity={0
        .3} />
    )
  )}
    {carets &&
      ([ "start", "end" ] as const).map((which) => {
        const c = carets[which];
        const k = KNOB / zoom;
        return (
          <g key={which}>
            <line x1={c.x} y1={c.y} x2={c.x} y2={c.y + c.h} stroke="#2563eb" strokeWidth={2 / zoom} />
            <circle
              cx={c.x}
              cy={c.y + c.h + k}
              r={k}
              fill="#2563eb"
              stroke="ffffff"
              strokeWidth={1.5 / zoom}
            />
          </g>
        );
      })
    }
  </g>
) : null;

// ---- screen overlay: the action bar and the hint -----

let menu: React.ReactNode = null;
if (active && sel && !dragging && selWorldRects.length && text) {
  const xs = selWorldRects.flatMap((r) => [r.x, r.x + r.w]);
  const top = Math.min(...selWorldRects.map((r) => r.y));
  const bottom = Math.max(...selWorldRects.map((r) => r.y + r.h));
  const a = toScreen((Math.min(...xs) + Math.max(...xs)) / 2, top);
  const b = toScreen(0, bottom);
  const BAR_H = 48;
  // Centred over the selection, but never past either edge: the bar is
  // ~370px wide, and right-aligned Hebrew near the edge hid Copy or Find.
  const half = Math.min(barW, viewport.width - 16) / 2;
  const barX = Math.max(8 + half, Math.min(viewport.width - 8 - half, a.x));
  const above = a.y - BAR_H - 14 > 8;

```

```

const y = above ? a.y - BAR_H - 14 : Math.min(viewport.height - BAR_H - 8, b.y + KNOB * 2 + 16);
menu = (
  <div
    className="pointer-events-none absolute inset-0 z-30 overflow-hidden"
    style={{ touchAction: "none" }}
  >
    <div
      role="toolbar"
      aria-label="Selected text"
      ref={(el) => {
        if (el && Math.abs(el.offsetWidth - barW) > 1) setBarW(el.offsetWidth);
      }}
      className="pointer-events-auto absolute flex items-center gap-0.5 rounded-2xl border border-white/10
        bg-zinc-900/95 p-1 text-zinc-100 shadow-[0_12px_32px_-8px_rgba(0,0,0,0.55)] backdrop-blur-md an
        imate-in fade-in zoom-in-95 duration-150"
      style={{
        left: barX,
        top: Math.max(8, y),
        transform: "translateX(-50%)",
        maxWidth: viewport.width - 16,
      }}
      onPointerDown={(e) => e.stopPropagation()}
    >
      <button
        type="button"
        onClick={doCopy}
        className="flex h-10 items-center gap-1.5 rounded-xl px-3 text-sm font-medium transition-colors ho
          ver:bg-white/10 active:bg-white/15"
        aria-label="Copy text"
      >
        {copied ? <Check size={16} className="text-emerald-400" /> : <Copy size={16} />}
        <span>{copied ? "Copied" : "Copy"}</span>
      </button>
      <span className="mx-1 h-6 w-px bg-white/15" />
      <div className="flex items-center gap-1 px-1" aria-label="Highlight">
        {MARK_COLORS.map((c, i) => (
          <button
            key={c.name}
            type="button"
            onClick={() => pickColor(i)}
            aria-label={`Highlight ${c.name.toLowerCase()}`}
            title={`Highlight ${c.name.toLowerCase()}`}
            className={cn(
              "flex h-10 w-8 items-center justify-center rounded-xl transition-transform hover:bg-white/10
                active:scale-90"
            )}
          >
            <span
              className={cn(
                "block h-5 w-5 rounded-full ring-offset-2 ring-offset-zinc-900",
                i === colorIndex && "ring-2 ring-white/80"
              )}
              style={{ backgroundColor: c.fill }}
            />
          </button>
        )})}
      </div>
      <span className="mx-1 h-6 w-px bg-white/15" />
      <button
        type="button"
        onClick={() => mark("underline")}
        className="flex h-10 w-10 items-center justify-center rounded-xl transition-colors hover:bg-white/
          10 active:bg-white/15"
        aria-label="Underline"
        title="Underline"
      >
        <Underline size={17} style={{ color: MARK_COLORS[colorIndex].fill }} />
      </button>
      <button
        type="button"
        onClick={() => mark("strike")}
        className="flex h-10 w-10 items-center justify-center rounded-xl transition-colors hover:bg-white/
          10 active:bg-white/15"
        aria-label="Strikethrough"
        title="Strikethrough"
      >
        <Strikethrough size={17} style={{ color: MARK_COLORS[colorIndex].fill }} />
      </button>

```

```

    <span className="mx-1 h-6 w-px bg-white/15" />
    <button
      type="button"
      onClick={findOnPage}
      className="flex h-10 w-10 items-center justify-center rounded-xl transition-colors hover:bg-white/
        10 active:bg-white/15"
      aria-label="Find in note"
      title="Find in note"
    >
      <Search size={17} />
    </button>
  </div>
</div>
);
}

let hint: string | null = null;
if (active) {
  if (!board || !isNb || !activePage) hint = "Select text works on PDF, Word and PowerPoint notebooks.";
  else if (!ref) hint = "This page has no document under it - there is no text to select.";
  else if (layout === undefined) hint = "Reading the page text...";
  else if (layout === null)
    hint = legacyOffice
      ? "This page was converted before ProNotes kept document text. Open the Word or PowerPoint file again
        to select its text."
      : "No selectable text on this page - it is a picture (a scan or a photo).";
}
const hintNode = hint ? (
  // Below the floating top bar, never over it.
  <div className="pointer-events-none absolute inset-x-0 top-20 z-20 flex justify-center px-4">
    <div className="flex max-w-md items-center gap-2 rounded-2xl border border-border/60 bg-background/90 px
      -3.5 py-2 text-xs leading-snug text-muted-foreground shadow-sm backdrop-blur animate-in fade-in dura
        tion-300">
      <TextSelectIcon size={14} className="shrink-0 opacity-70" />
      <span>{hint}</span>
    </div>
  </div>
) : null;

const overlay = (
  <>
    {hintNode}
    {menu}
  </>
);

return { pointerDown, pointerMove, pointerUp, cancel, worldLayer, overlay };
}

```

```

=====
// === END FILE: src/components/whiteboard/PdfTextSelect.tsx ===
=====

```

```

=====
// === START FILE: src/components/whiteboard/PenDiagnostics.tsx ===
// File 70 of 194 | Lines: 283 | Size: 10530 bytes
=====

```

```

"use client";

/**
 * Pen diagnostics overlay.
 *
 * Why this exists
 * -----
 * Hold-to-erase was written against the Pointer Events spec, where a stylus
 * side button is `buttons & 2` and the eraser end is `buttons & 32`. That is
 * correct on paper, and it is what Windows delivers – but it did NOT work on a
 * real Samsung tablet, and no amount of reading settles what one particular
 * device and WebView version actually send.
 *
 * Chromium on Android has several ways it MIGHT report the S Pen button:
 * - `buttons & 2` on pointerdown/pointermove (what the spec implies)
 * - `buttons & 32` (the eraser end)
 * - a `contextmenu` event, with nothing in `buttons` at all
 * - only while the nib is in contact, never on hover
 * - nothing whatsoever (the BLE remote button never reaches web content)

```

```

*
* So this panel prints what actually arrives. Hold the button, read the
* numbers, and it stops being a guess.
*/

import React from "react";
import { X, Copy, Check } from "lucide-react";
import { useWhiteboard } from "@store/whiteboard";

interface Row {
  n: number;
  type: string;
  pointerType: string;
  button: number;
  buttons: number;
  pressure: number;
  w: number;
  h: number;
  tiltX: number;
  tiltY: number;
}

/** Names the bits, so the readout needs no decoding by hand. */
function describeButtons(b: number): string {
  if (b === 0) return "none";
  const names: string[] = [];
  if (b & 1) names.push("1 nib");
  if (b & 2) names.push("2 BARREL");
  if (b & 4) names.push("4 middle");
  if (b & 8) names.push("8 X1");
  if (b & 16) names.push("16 X2");
  if (b & 32) names.push("32 ERASER");
  const known = 1 | 2 | 4 | 8 | 16 | 32;
  if (b & ~known) names.push((b & ~known) + " other");
  return names.join(" + ");
}

export function PenDiagnostics() {
  const on = useWhiteboard((s) => s.touchSettings.penDiagnostics === true);
  const setTouchSettings = useWhiteboard((s) => s.setTouchSettings);

  const [rows, setRows] = React.useState<Row[]>([]);
  const [live, setLive] = React.useState<{ buttons: number; pointerType: string } | null>(null);
  const [copied, setCopied] = React.useState(false);
  const seqRef = React.useRef(0);
  // pointermove fires hundreds of times a second. Only a CHANGE in the button
  // state is interesting; logging every move would bury it instantly.
  const lastMoveKeyRef = React.useRef("");

  React.useEffect(() => {
    if (!on) return;

    const push = (e: PointerEvent, type: string) => {
      seqRef.current += 1;
      const row: Row = {
        n: seqRef.current,
        type,
        pointerType: e.pointerType,
        button: e.button,
        buttons: e.buttons,
        pressure: Math.round((e.pressure ?? 0) * 100) / 100,
        w: Math.round(e.width ?? 0),
        h: Math.round(e.height ?? 0),
        tiltX: e.tiltX ?? 0,
        tiltY: e.tiltY ?? 0,
      };
      setRows((r) => [row, ...r].slice(0, 14));
    };

    const onDown = (e: PointerEvent) => push(e, "down");
    const onUp = (e: PointerEvent) => push(e, "up");
    const onMove = (e: PointerEvent) => {
      setLive({ buttons: e.buttons, pointerType: e.pointerType });
      const key = e.pointerType + ":" + e.buttons + ":" + e.button;
      if (key === lastMoveKeyRef.current) return;
      lastMoveKeyRef.current = key;
      push(e, "move*");
    };
  });
}

```

```

const onCtx = (e: Event) => {
  seqRef.current += 1;
  const pe = e as PointerEvent;
  setRows((r) =>
    [
      {
        n: seqRef.current,
        type: "CONTEXTMENU",
        pointerType: pe.pointerType ?? "?",
        button: pe.button ?? -1,
        buttons: pe.buttons ?? -1,
        pressure: 0,
        w: 0,
        h: 0,
        tiltX: 0,
        tiltY: 0,
      },
      ...r,
    ].slice(0, 14)
  );
};

// The native bridge (MainActivity.java). On Android this is the ONLY route
// that sees the S Pen side button, so it gets its own clearly marked row.
const onNative = (e: Event) => {
  const d = (e as CustomEvent).detail ?? {};
  seqRef.current += 1;
  setRows((r) =>
    [
      {
        n: seqRef.current,
        // `because` names the MotionEvent that decided it, so a wrong
        // decision says which rule got it wrong instead of just "up".
        type: (d.pressed ? "NATIVE down" : "NATIVE up") + (d.because ? " · " + d.because : ""),
        pointerType: "native",
        button: d.toolType ?? -1,
        buttons: d.buttonState ?? -1,
        pressure: 0,
        w: 0,
        h: 0,
        tiltX: 0,
        tiltY: 0,
      },
      ...r,
    ].slice(0, 14)
  );
};

// Capture phase, on window: the canvas calls preventDefault and stops
// propagation in places, and a diagnostic the app can silence would report
// "nothing arrived" when something did.
const opt = { capture: true, passive: true } as AddEventListenerOptions;
window.addEventListener("pointerdown", onDown, opt);
window.addEventListener("pointerup", onUp, opt);
window.addEventListener("pointermove", onMove, opt);
window.addEventListener("contextmenu", onCtx, { capture: true });
window.addEventListener("pronotes:stylusbutton", onNative);
return () => {
  window.removeEventListener("pronotes:stylusbutton", onNative);
  window.removeEventListener("pointerdown", onDown, opt);
  window.removeEventListener("pointerup", onUp, opt);
  window.removeEventListener("pointermove", onMove, opt);
  window.removeEventListener("contextmenu", onCtx, { capture: true });
};
}, [on]);

if (!on) return null;

const asText = () =>
[
  "ua: " + (typeof navigator !== "undefined" ? navigator.userAgent : "?"),
  "maxTouchPoints: " + (typeof navigator !== "undefined" ? navigator.maxTouchPoints : "?"),
  "n type ptrType button buttons pressure w h tiltX tiltY",
  ...rows.map((r) =>
    [r.n, r.type, r.pointerType, r.button, r.buttons, r.pressure, r.w, r.h, r.tiltX, r.tiltY].join(" ")
  ),
].join("\n");

```

```

return (
  <div
    data-wb-modal
    className="fixed bottom-2 left-1/2 z-[200] w-[min(680px,96vw)] -translate-x-1/2 rounded-xl border border
      -border bg-background/95 p-2 shadow-2xl backdrop-blur"
  >
    <div className="mb-1 flex items-center gap-2">
      <span className="text-[11px] font-semibold">Pen diagnostics</span>
      <span className="text-[10px] text-muted-foreground">
        hover the pen, then hold its side button
      </span>
    </div>
    <div className="ml-auto flex items-center gap-1">
      <button
        onClick={async () => {
          try {
            await navigator.clipboard.writeText(asText());
            setCopied(true);
            setTimeout(() => setCopied(false), 1500);
          } catch {
            // Clipboard is often refused inside an Android WebView. The
            // panel stays readable on screen precisely so a screenshot
            // works when this fails.
            setCopied(false);
          }
        }}
        className="rounded-md px-1.5 py-1 text-muted-foreground hover:bg-muted"
        title="Copy the log"
      >
        {copied ? <Check size={13} /> : <Copy size={13} />}
      </button>
      <button
        onClick={() => setTouchSettings({ penDiagnostics: false })}
        className="rounded-md px-1.5 py-1 text-muted-foreground hover:bg-muted"
        title="Close diagnostics"
      >
        <X size={13} />
      </button>
    </div>
  </div>

  {/* The one number that answers the question, large enough to read at
    arm's length while the other hand holds the pen. */}
  <div className="mb-1 rounded-lg bg-muted/60 px-2 py-1.5">
    <div className="text-[10px] uppercase tracking-wider text-muted-foreground">
      live buttons {live ? "(" + live.pointerType + ")" : ""}
    </div>
    <div className="font-mono text-[15px] font-bold">
      {live ? live.buttons + " – " + describeButtons(live.buttons) : "move the pen over the screen"}
    </div>
  </div>

  <div className="max-h-[38vh] overflow-auto rounded-lg bg-muted/40">
    <table className="w-full font-mono text-[10px]">
      <thead className="sticky top-0 bg-muted text-muted-foreground">
        <tr>
          {[ "#", "event", "type", "btn", "buttons", "press", "w", "h", "tiltX", "tiltY"].map((h) => (
            <th key={h} className="px-1 py-0.5 text-left font-semibold">
              {h}
            </th>
          ))}
        </tr>
      </thead>
      <tbody>
        {rows.length === 0 && (
          <tr>
            <td colspan={10} className="px-1 py-2 text-muted-foreground">
              nothing yet...
            </td>
          </tr>
        )}
        {rows.map((r) => (
          <tr
            key={r.n}
            className={
              r.type.startsWith("NATIVE")
                ? "bg-violet-500/30 font-bold"
                : r.type === "CONTEXTMENU"
                  ? "bg-amber-500/25 font-bold"
            }
          >

```

```

        : r.buttons & (2 | 32)
        ? "bg-emerald-500/25 font-bold"
        : ""
    }
}
>
<td className="px-1">{r.n}</td>
<td className="px-1">{r.type}</td>
<td className="px-1">{r.pointerType}</td>
<td className="px-1">{r.button}</td>
<td className="px-1">{r.buttons}</td>
<td className="px-1">{r.pressure}</td>
<td className="px-1">{r.w}</td>
<td className="px-1">{r.h}</td>
<td className="px-1">{r.tiltX}</td>
<td className="px-1">{r.tiltY}</td>
</tr>
)))
</tbody>
</table>
</div>

<p className="mt-1 text-[9px] leading-snug text-muted-foreground">
  Violet NATIVE row = Android itself reported the side button and the native
  bridge delivered it – this is the route that works on Samsung. Green row =
  the barrel (2) or eraser (32) bit reached web content on its own. Amber
  CONTEXTMENU = the button arrives as a right-click. Nothing at all while you
  hold it means the press never left the pen.
</p>
</div>
);
}

=====
// === END FILE: src/components/whiteboard/PenDiagnostics.tsx ===
=====

=====
// === START FILE: src/components/whiteboard/PropertiesPanel.tsx ===
// File 71 of 194 | Lines: 635 | Size: 27793 bytes
=====

"use client";

/**
 * Contextual Properties HUD & Inspector (v3 – Obsidian Canvas Grade)
 * =====
 * Floating contextual inspector that appears when an element is selected.
 */

import React, { useState } from "react";
import {
  Bold,
  Italic,
  Type,
  Palette,
  Minus,
  X,
  BringToFront,
  SendToBack,
  Copy,
  Trash2,
  Lock,
  Unlock,
  ChevronDown,
  ChevronUp,
  ArrowRight,
  GitBranch,
  CheckSquare,
} from "lucide-react";
import { useWhiteboard, useActiveBoard } from "@store/whiteboard";
import { getBoardElements } from "@lib/whiteboard/notebook";
import {
  PEN_COLORS,
  STROKE_WIDTHS,
  STICKY_COLORS,
  BUILTIN_LINK_TYPES,
  LINK_TYPE_COLORS,

```

```

LINK_TYPE_LABELS,
FONT_FAMILIES,
setToolInk,
type StickyColor,
type WhiteboardElement,
type ConnectionElement,
type ConnectionRouting,
type ArrowheadType,
type DashStyle,
type FillStyle,
type TextAlign,
} from "@lib/whiteboard/types";
import { Button } from "@components/ui/button";
import { StepperRow, StepButton } from "../StepperControl";
import { Separator } from "@components/ui/separator";
import { Popover, PopoverContent, PopoverTrigger } from "@components/ui/popover";
import { cn } from "@lib/utils";

export function PropertiesPanel() {
  const selectedIds = useWhiteboard((s) => s.selectedIds);
  const activeBoard = useActiveBoard();
  const updateElement = useWhiteboard((s) => s.updateElement);
  const updateElements = useWhiteboard((s) => s.updateElements);
  const deleteSelected = useWhiteboard((s) => s.deleteSelected);
  const duplicateElements = useWhiteboard((s) => s.duplicateElements);
  const toggleLock = useWhiteboard((s) => s.toggleLock);
  const bringToFront = useWhiteboard((s) => s.bringToFront);
  const sendToBack = useWhiteboard((s) => s.sendToBack);
  const commitHistory = useWhiteboard((s) => s.commitHistory);
  const clearSelection = useWhiteboard((s) => s.clearSelection);
  const invertSelection = useWhiteboard((s) => s.invertSelection);
  const selectByType = useWhiteboard((s) => s.selectByType);
  const customLinkTypes = useWhiteboard((s) => s.customLinkTypes);
  const editingId = useWhiteboard((s) => s.editingId);
  const setStyle = useWhiteboard((s) => s.setStyle);
  const propsPanelCollapsed = useWhiteboard((s) => s.propsPanelCollapsed);
  const setPropsPanelCollapsed = useWhiteboard((s) => s.setPropsPanelCollapsed);

  if (selectedIds.length === 0 || !activeBoard) return null;
  // While a text box or sticky note is being typed in, the formatting bar
  // attached to it is the ONE place to style it. Showing this panel as well
  // meant two sets of controls that disagreed - and on a tablet the panel
  // covered the right half of that bar, including its Done button.
  if (editingId) return null;

  const isSingle = selectedIds.length === 1;
  // Selections in a notebook live on the active page; the canvas field is
  // empty there, so this panel rendered nothing for them.
  const boardElements = getBoardElements(activeBoard);
  const el = isSingle ? boardElements.find((item) => item.id === selectedIds[0]) : null;
  const firstSelected = boardElements.find((item) => selectedIds.includes(item.id));
  const currentOpacity = (isSingle ? el?.opacity : firstSelected?.opacity) ?? 1;

  const handleChange = (patch: Partial<WhiteboardElement>, commit = false) => {
    if (!el) return;
    if (commit) commitHistory();
    updateElement(el.id, patch, false);
  };

  /**
   * Text styling from this panel behaves exactly like the text box's own bar:
   * it styles the box AND becomes the default for the next one, so the panel,
   * the bar and the bottom tool bar can never show different values.
   */
  const applyTextStyle = (patch: { fontFamily?: string; fontSize?: number; color?: string }) => {
    if (!el) return;
    handleChange(patch as Partial<WhiteboardElement>, true);
    const st = useWhiteboard.getState().style;
    if (el.type === "text") {
      setStyle({
        ...(patch.fontFamily ? { fontFamily: patch.fontFamily } : {}),
        ...setToolInk(st, "text", { color: patch.color, size: patch.fontSize }),
      });
    } else if (el.type === "sticky" && patch.fontSize !== undefined) {
      setStyle(setToolInk(st, "sticky", { size: patch.fontSize }));
    }
  };
};

```

```

return (
  // z-50 puts this ABOVE the floating favourites pen bar (z-40). They share
  // the right-hand edge, and the bar was covering the third option of every
  // row here — Solid / Dotted / 8px were unreachable, which read as "the
  // panel does not work". The panel is transient and task-focused, so it wins.
  <div className="absolute top-16 right-5 z-50 flex max-h-[calc(100%-6rem)] w-64 max-w-[calc(100%-2.5rem)] flex-col overflow-hidden rounded-2xl border border-border/80 bg-white/95 shadow-2xl backdrop-blur-xl animate-in fade-in slide-in-from-top-2 dark:bg-zinc-900/95">
    { /* Header */ }
    <div className="flex items-center justify-between px-3.5 py-2.5 border-b border-border/70 bg-muted/30">
      <span className="text-xs font-semibold capitalize flex items-center gap-1.5 text-foreground">
        <Palette size={13} className="text-primary" />
        {isSingle ? `${el?.type} Properties` : `${selectedIds.length} Elements Selected`}
      </span>
      <div className="flex items-center gap-0.5">
        <button
          onClick={() => setPropsPanelCollapsed(!propsPanelCollapsed)}
          title={propsPanelCollapsed ? "Expand panel" : "Fold panel to its header"}
          className="p-1 rounded-md hover:bg-muted text-muted-foreground hover:text-foreground"
        >
          {propsPanelCollapsed ? <ChevronDown size={14} /> : <ChevronUp size={14} />}
        </button>
        <button
          onClick={clearSelection}
          className="p-1 rounded-md hover:bg-muted text-muted-foreground hover:text-foreground"
        >
          <X size={14} />
        </button>
      </div>
    </div>

    {!propsPanelCollapsed && (
      <div className="p-3.5 space-y-3.5 max-h-[70vh] overflow-y-auto">
        { /* === Single Element Specific Controls === */ }
        {el && (
          <>
            { /* 1. TEXT / STICKY CONTROLS */ }
            {(el.type === "text" || el.type === "sticky") && (
              <div className="space-y-2.5">
                <span className="text-[11px] font-medium text-muted-foreground uppercase tracking-wider">
                  Typography
                </span>

                { /* Font size: the ACTUAL value with -/+, presets below. Presets
                  alone could not show a size like 17, so none looked chosen. */ }
                <div className="space-y-1.5 text-xs">
                  <div className="flex items-center justify-between">
                    <span className="text-muted-foreground">Size</span>
                    <div className="flex items-center gap-1">
                      <StepButton
                        direction={-1}
                        value={(el as { fontSize: number }).fontSize}
                        min={10}
                        max={96}
                        onChange={(v) => applyTextStyle({ fontSize: v })}
                        className="h-7 w-7"
                      />
                      <span className="min-w-[44px] text-center text-[12px] font-semibold tabular-nums">
                        {(el as { fontSize: number }).fontSize}px
                      </span>
                      <StepButton
                        direction={1}
                        value={(el as { fontSize: number }).fontSize}
                        min={10}
                        max={96}
                        onChange={(v) => applyTextStyle({ fontSize: v })}
                        className="h-7 w-7"
                      />
                    </div>
                  </div>
                </div>

                <div className="grid grid-cols-5 gap-1">
                  {[14, 18, 24, 32, 48].map((sz) => (
                    <button
                      key={sz}
                      onClick={() => applyTextStyle({ fontSize: sz })}
                      className={cn(
                        "py-0.5 rounded text-[11px] font-semibold border",
                        (el as { fontSize: number }).fontSize === sz

```

```

        ? "bg-primary text-primary-foreground border-primary"
        : "border-border hover:bg-muted text-foreground"
    )}
  >
  {sz}
</button>
)}}
</div>
</div>

{/* Font family for text */}
{el.type === "text" && (
  <div className="space-y-1">
    <span className="text-xs text-muted-foreground">Font Family</span>
    <div className="grid grid-cols-2 gap-1">
      {FONT_FAMILIES.map((ff) => (
        <button
          key={ff.id}
          onClick={() => applyTextStyle({ fontFamily: ff.value })}
          className={cn(
            "px-2 py-1 rounded text-left text-[11px] border truncate",
            (el as { fontFamily: string }).fontFamily === ff.value
              ? "bg-primary text-primary-foreground border-primary"
              : "border-border hover:bg-muted text-foreground"
          )}
          style={{ fontFamily: ff.value }}
        >
          {ff.label.split(" ")[0]}
        </button>
      ))}
    </div>
  </div>
)}

{/* Text Color / Sticky Color */}
<div className="space-y-1">
  <span className="text-xs text-muted-foreground">Color</span>
  {el.type === "sticky" ? (
    <div className="flex items-center gap-1.5">
      {Object.entries(STICKY_COLORS).map(([cKey, cVal]) => (
        <button
          key={cKey}
          onClick={() => handleChange({ color: cKey as StickyColor }, true)}
          className={cn(
            "w-6 h-6 rounded-full border transition-transform",
            (el as { color: StickyColor }).color === cKey
              ? "scale-110 ring-2 ring-primary border-foreground"
              : "border-border"
          )}
          style={{ backgroundColor: cVal.bg }}
          title={cVal.label}
        >
        </button>
      ))}
    </div>
  ) : (
    <div className="grid grid-cols-5 gap-1.5">
      {PEN_COLORS.map((c) => (
        <button
          key={c}
          onClick={() => applyTextStyle({ color: c })}
          className={cn(
            "w-6 h-6 rounded-full border transition-transform",
            (el as { color: string }).color === c
              ? "scale-110 ring-2 ring-primary border-foreground"
              : "border-border"
          )}
          style={{ backgroundColor: c }}
        >
        </button>
      ))}
    </div>
  )}
</div>
)}

{/* 2. SHAPE CONTROLS (Rectangle, Ellipse, Diamond, Triangle, Star, Cloud) */}
{["rectangle", "ellipse", "diamond", "triangle", "star", "cloud"].includes(el.type) && (
  <div className="space-y-3">

```

```

<span className="text-[11px] font-medium text-muted-foreground uppercase tracking-wider">
  Shape Styling
</span>

{/* Stroke Color */}
<div className="space-y-1">
  <span className="text-xs text-muted-foreground">Border Color</span>
  <div className="grid grid-cols-5 gap-1.5">
    {PEN_COLORS.map((c) => (
      <button
        key={c}
        onClick={() => handleChange({ strokeColor: c }, true)}
        className={cn(
          "w-6 h-6 rounded-full border",
          (el as { strokeColor: string }).strokeColor === c ? "scale-110 ring-2 ring-primary"
: ""
        )}
        style={{ backgroundColor: c }}
      />
    ))}
  </div>
</div>

{/* Fill Color */}
<div className="space-y-1">
  <span className="text-xs text-muted-foreground">Fill Color</span>
  <div className="grid grid-cols-5 gap-1.5">
    {PEN_COLORS.map((c) => (
      <button
        key={c}
        onClick={() => handleChange({ fillColor: c }, true)}
        className={cn(
          "w-6 h-6 rounded-full border",
          (el as { fillColor?: string }).fillColor === c ? "scale-110 ring-2 ring-primary" : "
"
        )}
        style={{ backgroundColor: c }}
      />
    ))}
  </div>
</div>

{/* Fill Style */}
<div className="space-y-1">
  <span className="text-xs text-muted-foreground">Fill Mode</span>
  <div className="grid grid-cols-3 gap-1">
    {(["transparent", "semi", "solid"] as FillStyle[]).map((fs) => (
      <button
        key={fs}
        onClick={() => handleChange({ fillStyle: fs }, true)}
        className={cn(
          "py-1 rounded text-xs capitalize border font-medium",
          (el as { fillStyle?: FillStyle }).fillStyle === fs
            ? "bg-primary text-primary-foreground border-primary"
            : "border-border hover:bg-muted text-foreground"
        )}
      >
        {fs}
      </button>
    ))}
  </div>
</div>

{/* Stroke Style */}
<div className="space-y-1">
  <span className="text-xs text-muted-foreground">Border Line</span>
  <div className="grid grid-cols-3 gap-1">
    {(["solid", "dashed", "dotted"] as DashStyle[]).map((ds) => (
      <button
        key={ds}
        onClick={() => handleChange({ dashStyle: ds }, true)}
        className={cn(
          "py-1 rounded text-xs capitalize border font-medium",
          (el as { dashStyle?: DashStyle }).dashStyle === ds
            ? "bg-primary text-primary-foreground border-primary"
            : "border-border hover:bg-muted text-foreground"
        )}
      >

```

```

        {ds}
      </button>
    )}
  </div>
</div>

{/* Stroke Width */}
<div className="space-y-1">
  <span className="text-xs text-muted-foreground">Border Width</span>
  <div className="flex items-center justify-between gap-1">
    {[1, 2, 4, 8].map((w) => (
      <button
        key={w}
        onClick={() => handleChange({ strokeWidth: w }, true)}
        className={cn(
          "flex-1 py-1 rounded text-xs border font-medium",
          (el as { strokeWidth: number }).strokeWidth === w
            ? "bg-primary text-primary-foreground border-primary"
            : "border-border hover:bg-muted text-foreground"
        )}
      >
        {w}px
      </button>
    )]}
  </div>
</div>
</div>
)}

{/* 3. CONNECTION / ARROW CONTROLS */}
{el.type === "connection" && (
  <div className="space-y-3">
    <span className="text-[11px] font-medium text-muted-foreground uppercase tracking-wider">
      Arrow & Relationship
    </span>

    {/* Routing Mode */}
    <div className="space-y-1">
      <span className="text-xs text-muted-foreground">Arrow Routing</span>
      <div className="grid grid-cols-3 gap-1">
        {[["curved", "orthogonal", "straight"] as ConnectionRouting[]].map((r) => (
          <button
            key={r}
            onClick={() => handleChange({ routing: r }, true)}
            className={cn(
              "py-1 rounded text-xs capitalize border font-medium",
              (el as ConnectionElement).routing === r
                ? "bg-primary text-primary-foreground border-primary"
                : "border-border hover:bg-muted text-foreground"
            )}
          >
            {r}
          </button>
        )]}
      </div>
    </div>

    {/* Relationship Type */}
    <div className="space-y-1">
      <span className="text-xs text-muted-foreground">Relationship Type</span>
      <div className="space-y-1 max-h-36 overflow-y-auto pr-1">
        {BUILTIN_LINK_TYPES.map((lt) => (
          <button
            key={lt}
            onClick={() => handleChange({ linkType: lt }, true)}
            className={cn(
              "w-full text-left px-2 py-1 rounded text-xs flex items-center justify-between",
              (el as ConnectionElement).linkType === lt
                ? "bg-muted font-semibold text-primary"
                : "hover:bg-muted/50 text-foreground"
            )}
          >
            <div className="flex items-center gap-2">
              <span
                className="w-2 h-2 rounded-full"
                style={{ backgroundColor: LINK_TYPE_COLORS[lt] }}
              />
              <span>{LINK_TYPE_LABELS[lt]}</span>
            </div>
        )]}
      </div>
    </div>
  </div>
)}

```

```

        </div>
      </button>
    )}
  </div>
</div>

{/* Label edit */}
<div className="space-y-1">
  <span className="text-xs text-muted-foreground">Custom Label</span>
  <input
    type="text"
    value={(el as ConnectionElement).label || ""}
    onChange={(e) => handleChange({ label: e.target.value }, false)}
    placeholder="e.g. Depends on, Causes..."
    className="w-full px-2.5 py-1.5 rounded-lg border border-border bg-background text-xs"
  />
</div>

{/* Arrowheads */}
<div className="space-y-1">
  <span className="text-xs text-muted-foreground">Arrow Head</span>
  <div className="grid grid-cols-4 gap-1">
    {[ "arrow", "triangle", "diamond", "circle" ] as ArrowheadType[] }.map((ah) => (
      <button
        key={ah}
        onClick={() => handleChange({ endArrowhead: ah }, true)}
        className={cn(
          "py-1 rounded text-[11px] capitalize border",
          (el as ConnectionElement).endArrowhead === ah
            ? "bg-primary text-primary-foreground border-primary"
            : "border-border hover:bg-muted text-foreground"
        )}
      >
        {ah}
      </button>
    ))
  </div>
</div>
</div>
)}
</>
)}

{/* === Opacity Control (Universal for Single & Multi Selection) === */}
<div className="space-y-2 pt-1 border-t border-border/50">
  <div className="flex items-center justify-between">
    <span className="text-[11px] font-medium text-muted-foreground uppercase tracking-wider">
      Opacity
    </span>
    <span className="text-xs font-bold tabular-nums px-2 py-0.5 rounded-md bg-primary/10 text-primary
      border border-primary/20">
      {Math.round(currentOpacity * 100)}%
    </span>
  </div>

  <StepperRow
    value={Math.round(currentOpacity * 100)}
    min={10}
    max={100}
    step={5}
    onChange={(pct) => {
      const opacity = pct / 100;
      if (selectedIds.length > 1) {
        updateElements(selectedIds, { opacity }, false);
      } else if (el) {
        updateElement(el.id, { opacity }, false);
      }
    }}
  >
    <input
      type="range"
      min="10"
      max="100"
      step="5"
      value={Math.round(currentOpacity * 100)}
      aria-label="Opacity"
      onChange={(e) => {

```

```

        const opacity = Number(e.target.value) / 100;
        if (selectedIds.length > 1) {
            updateElements(selectedIds, { opacity }, false);
        } else if (el) {
            updateElement(el.id, { opacity }, false);
        }
    }}
    onMouseUp={() => commitHistory()}
    onTouchEnd={() => commitHistory()}
    className="w-full h-2 bg-muted rounded-lg appearance-none cursor-pointer accent-primary"
  />
</StepperRow>

{/* Quick jump presets */}
<div className="flex items-center justify-between gap-1 pt-0.5">
  {[25, 50, 75, 100].map((pct) => (
    <button
      key={pct}
      type="button"
      onClick={() => {
        commitHistory();
        const opacity = pct / 100;
        if (selectedIds.length > 1) {
            updateElements(selectedIds, { opacity }, false);
        } else if (el) {
            updateElement(el.id, { opacity }, false);
        }
      }}
      className={cn(
        "flex-1 py-1 rounded text-[10px] border font-medium transition-colors",
        Math.round(currentOpacity * 100) === pct
          ? "border-primary bg-primary text-primary-foreground font-semibold"
          : "border-border text-muted-foreground hover:bg-muted"
      )}
    >
      {pct}%
    </button>
  )]}
</div>
</div>

<Separator />

{/* === Universal Actions (Duplicate, Layering, Lock, Delete) === */}
<div className="space-y-2">
  <span className="text-[11px] font-medium text-muted-foreground uppercase tracking-wider">
    Arrange & Actions
  </span>
  <div className="grid grid-cols-2 gap-1.5">
    <button
      onClick={() => bringToFront(selectedIds)}
      className="flex items-center gap-1.5 px-2.5 py-1.5 rounded-lg border border-border hover:bg-muted text-xs text-foreground font-medium"
    >
      <BringToFront size={13} />
      <span>Bring to Front</span>
    </button>
    <button
      onClick={() => sendToBack(selectedIds)}
      className="flex items-center gap-1.5 px-2.5 py-1.5 rounded-lg border border-border hover:bg-muted text-xs text-foreground font-medium"
    >
      <SendToBack size={13} />
      <span>Send to Back</span>
    </button>
    <button
      onClick={() => duplicateElements(selectedIds)}
      className="flex items-center gap-1.5 px-2.5 py-1.5 rounded-lg border border-border hover:bg-muted text-xs text-foreground font-medium"
    >
      <Copy size={13} />
      <span>Duplicate</span>
    </button>
    <button
      onClick={() => toggleLock(selectedIds)}
      className="flex items-center gap-1.5 px-2.5 py-1.5 rounded-lg border border-border hover:bg-muted text-xs text-foreground font-medium"
    >

```

```

    <Lock size={13} />
    <span>Lock / Unlock</span>
  </button>
  <button
    onClick={() => invertSelection()}
    className="col-span-2 flex items-center justify-center gap-1.5 px-2.5 py-1.5 rounded-lg border b
      order-border hover:bg-muted text-xs text-foreground font-medium"
    title="Invert selection (Ctrl+Shift+V)"
  >
    <CheckSquare size={13} />
    <span>Invert Selection</span>
  </button>
  {el && (
    <button
      onClick={() => {
        const t =
          el.type === "stroke"
            ? "stroke"
            : el.type === "text"
              ? "text"
              : el.type === "sticky"
                ? "sticky"
                : el.type === "image"
                  ? "image"
                  : el.type === "connection"
                    ? "connection"
                    : "shape";
        selectByType(t);
      }}
      className="col-span-2 flex items-center justify-center gap-1.5 px-2.5 py-1.5 rounded-lg border
        border-border hover:bg-muted text-xs text-foreground font-medium"
    >
      <CheckSquare size={13} />
      <span>
        Select all{" "}
        {el.type === "stroke"
          ? "Ink Strokes"
          : el.type === "text"
            ? "Text Notes"
            : el.type === "sticky"
              ? "Sticky Notes"
              : el.type === "image"
                ? "Images"
                : el.type === "connection"
                  ? "Connectors"
                  : "Shapes"}
      </span>
    </button>
  )}
</div>

<button
  onClick={deleteSelected}
  className="w-full flex items-center justify-center gap-1.5 px-2.5 py-1.5 rounded-lg bg-destructive
    /10 text-destructive hover:bg-destructive hover:text-destructive-foreground transition-colors
    text-xs font-semibold"
>
  <Trash2 size={13} />
  <span>Delete Selected</span>
</button>
</div>
</div>
)}
</div>
);
}

```

```

=====
// === END FILE: src/components/whiteboard/PropertiesPanel.tsx ===
=====

```

```

=====
// === START FILE: src/components/whiteboard/SearchModal.tsx ===
// File 72 of 194 | Lines: 965 | Size: 38449 bytes
=====

```

```

"use client";

```

```

/**
 * Search
 * =====
 * One search with three scopes, picked at the top of the panel:
 *
 * - This note    find-in-page. Results run top to bottom, page by page, and
 *                can be stepped through ("3 of 12") from a small find bar that
 *                leaves the page visible.
 * - This folder  the open note's folder and its sub-folders.
 * - Everywhere   every note, NEAREST FIRST: this note, its folder, the
 *                folders around it, then the rest - under headers that say so.
 *
 * What is searched is spelled out in the empty panel, because "what does this
 * actually search?" was the complaint: names of notes and folders, tags, text
 * boxes, sticky notes, connection labels and PDF / document text. Handwriting,
 * shapes and pictures carry no text, so they are not searchable - and the
 * panel says that rather than implying otherwise.
 *
 * Ctrl+F opens it on "This note"; Ctrl+K and the header button on "Everywhere".
 * The ranking and scoping rules live in lib/whiteboard/search.ts.
 */

import React, { useCallback, useDeferredValue, useEffect, useMemo, useRef, useState } from "react";
import { useWhiteboard, useActiveBoard } from "@store/whiteboard";
import {
  ChevronDown,
  ChevronUp,
  FileText,
  Folder as FolderIcon,
  Globe,
  Hash,
  ListFilter,
  Search,
  Spline,
  StickyNote,
  Type,
  X,
} from "lucide-react";
import { cn } from "@lib/utls";
import type { Board } from "@lib/whiteboard/types";
import { getActivePageIndex, getBoardElements, getPages, isNotebook } from "@lib/whiteboard/notebook";
import { worldToScreen } from "@lib/whiteboard/geometry";
import {
  boardInScope,
  buildSearchIndex,
  focusBounds,
  folderPath,
  groupHits,
  highlightParts,
  isLocatable,
  makeSnippet,
  pdfHit,
  scopeInfo,
  searchEntries,
  sortHits,
  type Nearness,
  type SearchContext,
  type SearchHit,
  type SearchKind,
  type SearchScope,
} from "@lib/whiteboard/search";
import { searchNotebookPdfText } from "@lib/whiteboard/pdfTextIndex";
import { BoardKindIcon } from "../Sidebar";

interface SearchModalProps {
  initialScope: SearchScope;
}
/**
 * "bar": open straight into the compact find bar and go to the first match
 * from this page on - no panel over the page, no keyboard. For "Find in
 * note" on a text selection, whose matches you want to SEE.
 */
initialMode?: "panel" | "bar";
/** Start with this text (e.g. "Find in note" on a text selection). */
initialQuery?: string;
onClose: () => void;
}

```

```
// The last query, for this session. Reopening search shows where you were,
// with the text selected so typing simply replaces it.
let lastQuery = "";

/** How many rows each group shows before "Show more". */
const GROUP_CAP: Record<SearchScope, number> = { note: 50, folder: 8, everywhere: 6 };
const MORE_STEP = 20;

const KIND_META: Record<SearchKind, { label: string; icon: React.ElementType; tint: string }> = {
  board: { label: "Note", icon: FileText, tint: "bg-blue-500/10 text-blue-600 dark:text-blue-400" },
  folder: { label: "Folder", icon: FolderIcon, tint: "bg-amber-500/10 text-amber-600 dark:text-amber-400" },
  tag: { label: "Tag", icon: Hash, tint: "bg-emerald-500/10 text-emerald-600 dark:text-emerald-400" },
  text: { label: "Text box", icon: Type, tint: "bg-sky-500/10 text-sky-600 dark:text-sky-400" },
  sticky: { label: "Sticky note", icon: StickyNote, tint: "bg-yellow-400/20 text-yellow-700 dark:text-yellow-300" },
  connection: { label: "Connection label", icon: Spline, tint: "bg-violet-500/10 text-violet-600 dark:text-violet-400" },
  pdf: { label: "Document text", icon: FileText, tint: "bg-red-500/10 text-red-600 dark:text-red-400" },
};

/** What each scope looks through, for the empty panel. */
const SEARCHED: Record<SearchScope, SearchKind[]> = {
  note: ["text", "sticky", "connection", "pdf"],
  folder: ["board", "folder", "tag", "text", "sticky", "connection", "pdf"],
  everywhere: ["board", "folder", "tag", "text", "sticky", "connection", "pdf"],
};

const SEARCHED_LABEL: Partial<Record<SearchKind, string>> = {
  board: "Note names",
  folder: "Folder names",
  tag: "Tags",
  text: "Text boxes",
  sticky: "Sticky notes",
  connection: "Connection labels",
  pdf: "PDF & document text",
};

export function SearchModal({ initialScope, initialMode = "panel", initialQuery, onClose }: SearchModalProps) {
  const boards = useWhiteboard((s) => s.boards);
  const folders = useWhiteboard((s) => s.folders);
  const activeBoardId = useWhiteboard((s) => s.activeBoardId);
  const sidebarFolderId = useWhiteboard((s) => s.activeFolderId);
  const activeBoard = useActiveBoard();

  const [scopeChoice, setScope] = useState<SearchScope>(initialScope);
  // "This note" needs a note; with none open it quietly means everywhere.
  const scope: SearchScope = scopeChoice === "note" && !activeBoard ? "everywhere" : scopeChoice;
  const [mode, setMode] = useState<"panel" | "bar">(initialMode);
  const [query, setQueryState] = useState(() => {
    if (initialQuery) lastQuery = initialQuery;
    return lastQuery;
  });
  const setQuery = (q: string) => {
    lastQuery = q;
    setQueryState(q);
  };
  // Typing stays instant; the search itself runs at React's convenience.
  const deferredQuery = useDeferredValue(query);
  const [currentId, setCurrentId] = useState<string | null>(null);
  const [flash, setFlash] = useState<{ boardId: string; elementId: string; key: number } | null>(null);

  // Where the search started: the open note, and its folder (else the one
  // picked in the sidebar). The wide scopes rank relative to THIS, not to
  // wherever the last opened result took you - otherwise opening a result in
  // another note re-sorted the list around it, "next" led back to the start,
  // and results further away could never be reached.
  const [anchor] = useState(() => {
    const s = useWhiteboard.getState();
    const b = s.boards.find((x) => x.id === s.activeBoardId);
    return { boardId: b ? b.id : null, folderId: b ? b.folderId ?? null : s.activeFolderId ?? null };
  });
  const liveFolderId = activeBoard ? activeBoard.folderId ?? null : sidebarFolderId ?? null;
  // Primitives only: the board OBJECT changes with every stroke, and the
  // document-text search below must not re-run for each one.
  const liveBoardId = activeBoard ? activeBoardId : null;
  // "This note" always means the note on screen now.
  const ctx = useMemo<SearchContext>(() =>
```

```

    scope === "note"
      ? { activeBoardId: liveBoardId, folderId: liveFolderId, folders }
      : { activeBoardId: anchor.boardId, folderId: anchor.folderId, folders },
    [scope, liveBoardId, liveFolderId, anchor, folders]
  );
const currentFolderId = anchor.folderId;
const folderName = currentFolderId
  ? folders.find((f) => f.id === currentFolderId)?.name || "Folder"
  : "Top level";

// ----- Results -----

const index = useMemo(() => buildSearchIndex(boards, folders), [boards, folders]);
const localHits = useMemo(
  () => searchEntries(index, deferredQuery, scope, ctx),
  [index, deferredQuery, scope, ctx]
);

// Document text comes from the text layer, asynchronously and a little
// later (it may have to read a PDF), so it never holds up typing.
const trimmed = deferredQuery.trim();
const pdfKey = `${scope}|${ctx.activeBoardId}|${ctx.folderId}|${trimmed}`;
const [pdf, setPdf] = useState<{ key: string; hits: SearchHit[]; pending: boolean }>({
  key: "",
  hits: [],
  pending: false,
});
useEffect(() => {
  if (!trimmed) return;
  const abort = new AbortController();
  const timer = setTimeout(() => {
    const s = useWhiteboard.getState();
    const info = scopeInfo(ctx);
    const targets = s.boards.filter(
      (b) => isNotebook(b) && getPages(b).some((p) => p.pdf) && boardInScope(b, scope, info)
    );
    if (targets.length === 0) return;
    const collected: SearchHit[] = [];
    setPdf({ key: pdfKey, hits: [], pending: true });
    const limit = scope === "note" ? 300 : 30;
    // One notebook at a time, nearest first, and stopped for good once the
    // query changes or search closes: reading every PDF in the workspace at
    // once could exhaust the tablet's memory.
    const ordered = [...targets].sort(
      (a, b) => (a.id === ctx.activeBoardId ? -1 : 0) - (b.id === ctx.activeBoardId ? -1 : 0)
    );
    void (async () => {
      for (const b of ordered) {
        if (abort.signal.aborted) return;
        try {
          const raw = await searchNotebookPdfText(b, trimmed, limit, { signal: abort.signal });
          if (abort.signal.aborted) return;
          raw.forEach((r, i) => collected.push(pdfHit(b, r, trimmed, i, info)));
          // Show each notebook's hits as they arrive.
          setPdf({ key: pdfKey, hits: [...collected], pending: true });
        } catch {
          // A document that cannot be read just contributes no text hits.
        }
      }
      if (!abort.signal.aborted) setPdf({ key: pdfKey, hits: [...collected], pending: false });
    })();
  }, 250);
  return () => {
    abort.abort();
    clearTimeout(timer);
  };
}, [pdfKey, trimmed, scope, ctx]);

const pdfPending = pdf.key === pdfKey && pdf.pending;
const hits = useMemo(() => {
  const pdfHits = pdf.key === pdfKey ? pdf.hits : [];
  return pdfHits.length ? sortHits([...localHits, ...pdfHits], scope) : localHits;
}, [localHits, pdf, pdfKey, scope]);
const groups = useMemo(
  () =>
    scope === "note"
      ? hits.length
        ? [{ id: "note" as Nearness, label: "In this note", hits }]

```

```

    : []
    : groupHits(hits, scope, ctx).map((g) =>
      // Once a result has taken you to another note, "this note" would
      // be ambiguous: name the note the search started from.
      g.id === "note" && ctx.activeBoardId !== activeBoardId
      ? { ...g, label: `In ${boards.find((b) => b.id === ctx.activeBoardId)?.name || "the first note"}`
        : g
    ),
    [hits, scope, ctx, activeBoardId, boards]
  );
  // What the find bar steps through: places in a note, in the order shown.
  const locatable = useMemo(() => hits.filter(isLocatable), [hits]);
  const currentIndex = currentId ? locatable.findIndex((h) => h.entry.id === currentId) : -1;

  // "Show more", per group, for this query only.
  const resultsKey = `${scope}${trimmed}`;
  const [more, setMore] = useState<{ key: string; extra: Partial<Record<Nearness, number>> }>({ key: "", extra: {} });
  const extra = more.key === resultsKey ? more.extra : {};
  const visibleGroups = groups.map((g) => {
    const cap = GROUP_CAP[scope] + (extra[g.id] ?? 0);
    return { ...g, shown: g.hits.slice(0, cap), hidden: Math.max(0, g.hits.length - cap) };
  });
  const rows = visibleGroups.flatMap((g) => g.shown);

  // Keyboard cursor over the visible rows; back to the top on a new query.
  const [cursorState, setCursor] = useState({ key: "", i: 0 });
  const cursor = cursorState.key === resultsKey ? Math.min(cursorState.i, Math.max(0, rows.length - 1)) : 0;
  const rowRefs = useRef(new Map<string, HTMLButtonElement>());
  // Scrolled from the key handler, not an effect: an effect would also fire on
  // unrelated re-renders and yank the list back while you scroll it by finger.
  const moveCursor = (i: number) => {
    setCursor({ key: resultsKey, i });
    const id = rows[i]?.entry.id;
    if (id) rowRefs.current.get(id)?.scrollIntoView({ block: "nearest" });
  };

  // ----- Navigation -----

  /**
   * Opens a document in the pane you are working in. In split view a result
   * in the OTHER pane's document makes that pane the live one rather than
   * showing one document twice.
   */
  const openBoard = useCallback((boardId: string) => {
    const s = useWhiteboard.getState();
    if (s.activeBoardId === boardId) return;
    if (s.splitBoardId === boardId) {
      // Switching panes refuses to run under an open text editor (the edit
      // would be lost); finish the edit first, as tapping away would.
      if (s.editingId) s.setEditingId(null);
      s.focusSplitSide(s.splitSide === "left" ? "right" : "left");
      return;
    }
    s.setActiveBoard(boardId);
  }, []);

  /** Lands on a hit: its note, its page, the element centred, and a flash. */
  const reveal = useCallback(
    (hit: SearchHit) => {
      const e = hit.entry;
      if (!e.boardId) return;
      openBoard(e.boardId);
      const s = useWhiteboard.getState();
      if (e.kind === "pdf") {
        const board = s.boards.find((b) => b.id === e.boardId);
        const pages = getPages(board);
        const pageId = e.pageId ?? pages[e.pageIndex ?? 0]?.id;
        // Already on that page (stepping through its matches): stay, and let
        // the text layer bring the match itself into view - going to the page
        // again snapped back to its top every step.
        const onPage = pages[getActivePageIndex(board)]?.id === pageId;
        if (pageId && !onPage) s.goToPage(pageId, s.canvasViewport.height);
        // After the page has rendered, so its text layer is there to listen.
        const detail = { boardId: e.boardId, pageId, pageIndex: e.pageIndex ?? 0, query: trimmed, ordinal: e.ordinal ?? 0 };
        requestAnimationFrame(() =>

```

```

        requestAnimationFrame(() => window.dispatchEvent(new CustomEvent("wb:pdf-find", { detail })))
    );
    return;
}
if (e.elementId) {
    s.focusElement(e.boardId, e.elementId, { select: false });
    setFlash({ boardId: e.boardId, elementId: e.elementId, key: Date.now() });
}
},
[openBoard, trimmed]
);

const open = (hit: SearchHit) => {
    const e = hit.entry;
    // Put the soft keyboard away: the page is what you want to see now.
    (document.activeElement as HTMLElement | null)?.blur?();
    if (e.kind === "folder") {
        if (e.folderId) useWhiteboard.getState().revealFolder(e.folderId);
        onClose();
        return;
    }
    if (!e.boardId) return;
    if (!isLocatable(hit)) {
        openBoard(e.boardId);
        onClose();
        return;
    }
    reveal(hit);
    setCurrentId(e.id);
    setMode("bar");
};

const step = (delta: 1 | -1) => {
    const n = locatable.length;
    if (n === 0) return;
    const i = currentIndex === -1 ? (delta > 0 ? 0 : n - 1) : (currentIndex + delta + n) % n;
    const hit = locatable[i];
    reveal(hit);
    setCurrentId(hit.entry.id);
};

// "Find in note": once the matches are in, go to the first one from this
// page on - usually the very text you selected - and show it, bar only.
const autoOpenedRef = useRef(initialMode !== "bar");
const pdfDone = pdf.key === pdfKey && !pdf.pending;
useEffect(() => {
    if (autoOpenedRef.current || locatable.length === 0) return;
    const here = activeBoard ? getActivePageIndex(activeBoard) : 0;
    const inThisNote = (h: SearchHit) => h.entry.boardId === activeBoardId && (h.entry.pageIndex ?? 0) >= here;
    const hit =
        locatable.find((h) => h.entry.kind === "pdf" && inThisNote(h)) ??
        (pdfDone ? locatable.find(inThisNote) ?? locatable[0] : undefined);
    if (!hit) return;
    autoOpenedRef.current = true;
    reveal(hit);
    setCurrentId(hit.entry.id);
    // eslint-disable-next-line react-hooks/exhaustive-deps
}, [locatable, pdfDone]);

// Esc closes search wherever the focus is. Once a result has been opened,
// focus is back on the page, and Esc used to reach the canvas instead: the
// bar stayed, and your selection and tool were reset.
useEffect(() => {
    const onKey = (e: KeyboardEvent) => {
        if (e.key !== "Escape") return;
        e.preventDefault();
        e.stopImmediatePropagation();
        onClose();
    };
    window.addEventListener("keydown", onKey, true);
    return () => window.removeEventListener("keydown", onKey, true);
}, [onClose]);

// ----- Keyboard -----

const inputRef = useRef<HTMLInputElement>(null);
// Focus (and so the tablet's keyboard) only when search is first opened -

```

```

// not when coming back to the list from the find bar with the pen.
const [focusOnShow, setFocusOnShow] = useState(initialMode === "panel");
useEffect(() => {
  if (mode !== "panel" || !focusOnShow) return;
  const raf = requestAnimationFrame(() => {
    inputRef.current?.focus();
    inputRef.current?.select();
  });
  return () => cancelAnimationFrame(raf);
}, [mode, focusOnShow]);

/** Ctrl+F / Ctrl+K while search already has the keyboard switch scope. */
const scopeShortcut = (e: React.KeyboardEvent) => {
  if (!(e.ctrlKey || e.metaKey) || e.shiftKey) return false;
  if (e.code === "KeyF" && activeBoard) {
    e.preventDefault();
    setScope("note");
    if (mode === "bar") inputRef.current?.select();
    return true;
  }
  if (e.code === "KeyK") {
    e.preventDefault();
    setScope("everywhere");
    setFocusOnShow(true);
    setMode("panel");
    return true;
  }
  return false;
};

// Arrows and Enter drive the results only from the text field; on a focused
// button Enter must still press that button.
const fromInput = (e: React.KeyboardEvent) => e.target === inputRef.current;

const onPanelKeyDown = (e: React.KeyboardEvent) => {
  if (scopeShortcut(e)) return;
  if (e.key === "Escape") {
    e.preventDefault();
    onClose();
    return;
  }
  if (!fromInput(e)) return;
  if (e.key === "ArrowDown") {
    e.preventDefault();
    moveCursor(Math.min(rows.length - 1, cursor + 1));
  } else if (e.key === "ArrowUp") {
    e.preventDefault();
    moveCursor(Math.max(0, cursor - 1));
  } else if (e.key === "Enter") {
    e.preventDefault();
    if (rows[cursor]) open(rows[cursor]);
  }
};

const onBarKeyDown = (e: React.KeyboardEvent) => {
  if (scopeShortcut(e)) return;
  if (e.key === "Escape") {
    e.preventDefault();
    onClose();
    return;
  }
  if (!fromInput(e)) return;
  if (e.key === "Enter" || e.key === "ArrowDown" || e.key === "ArrowUp") {
    e.preventDefault();
    step(e.key === "ArrowUp" || (e.key === "Enter" && e.shiftKey) ? -1 : 1);
  } else if (e.key === "Escape") {
    e.preventDefault();
    onClose();
  }
};

// ----- Where a result is -----

const boardById = useMemo(() => new Map(boards.map((b) => [b.id, b])), [boards]);
const pathOf = useMemo(() => {
  const cache = new Map<string | null, string>();
  return (folderId: string | null) => {
    if (!cache.has(folderId)) cache.set(folderId, folderPath(folderId, folders).join(" > "));
  };
});

```

```

    return cache.get(folderId) as string;
  };
}, [folders]);

const whereOf = (hit: SearchHit): string => {
  const e = hit.entry;
  if (e.kind === "folder") {
    const parent = folders.find((f) => f.id === e.folderId)?.parentId ?? null;
    return parent ? `Folder in ${pathOf(parent)}` : "Folder · top level";
  }
  const board = e.boardId ? boardById.get(e.boardId) : undefined;
  const path = pathOf(e.folderId) || "Top level";
  if (e.kind === "board") {
    const pages = getPages(board).length;
    const what = board && isNotebook(board) ? `Notebook · ${pages} page${pages === 1 ? "" : "s"}` : "Whiteboard";
    return `${what} · ${path}`;
  }
  const kind = KIND_META[e.kind].label;
  // Under "In this note" the note is already named by the header.
  if (scope === "note" || hit.nearness === "note") return kind;
  return `${kind} · ${board?.name || "Untitled"} · ${path}`;
};

// ----- Render -----

const scopes: Array<{ id: SearchScope; label: string; detail: string; icon: React.ReactNode; disabled?: boolean }> = [
  {
    id: "note",
    label: "This note",
    detail: activeBoard ? activeBoard.name || "Untitled" : "No note open",
    icon: activeBoard ? <BoardKindIcon board={activeBoard} size={13} /> : <FileText size={13} />,
    disabled: !activeBoard,
  },
  {
    id: "folder",
    label: "This folder",
    detail: folderName,
    icon: <FolderIcon size={13} className="shrink-0 text-amber-500" />,
  },
  {
    id: "everywhere",
    label: "Everywhere",
    detail: `All ${boards.length} note${boards.length === 1 ? "" : "s"}`,
    icon: <Globe size={13} className="shrink-0 text-emerald-500" />,
  },
];

const counter =
  locatable.length === 0
    ? trimmed
      ? "No matches"
      : ""
    : currentIndex >= 0
      ? `${currentIndex + 1} of ${locatable.length}`
      : `${locatable.length} match${locatable.length === 1 ? "" : "es"}`;

const flashRing = flash && (
  <FlashRing key={flash.key} boardId={flash.boardId} elementId={flash.elementId} onDone={() => setFlash(null)} />
);

if (mode === "bar") {
  const current = scopes.find((s) => s.id === scope);
  return (
    <>
      {flashRing}
      <div
        data-wb-search-bar
        role="search"
        onKeyDown={onBarKeyDown}
        data-wb-searchbar
        className="absolute left-1/2 top-[4.25rem] z-40 flex w-[min(30rem,calc(100%-1.5rem))] -translate-x-1/2 items-center gap-0.5 rounded-xl border border-80 bg-white/95 p-1 shadow-lg backdrop-blur-md dark:bg-zinc-900/95"
      >
        <button

```

```

    type="button"
    onClick={() => {
      setFocusOnShow(false);
      setMode("panel");
    }}
    className="flex h-9 shrink-0 items-center gap-1.5 rounded-lg px-2 text-[11px] font-medium text-muted-foreground hover:bg-muted hover:text-foreground"
    title={`Searching ${current?.label.toLowerCase()} - show all results`}
    aria-label="Show all results"
  >
    {current?.icon}
    <span className="hidden max-w-[7rem] truncate @min-[34rem]:inline">{current?.label}</span>
  </button>
  <input
    ref={inputRef}
    value={query}
    onChange={(e) => setQuery(e.target.value)}
    placeholder="Find..."
    dir="auto"
    aria-label="Search text"
    className="h-9 min-w-0 flex-1 bg-transparent px-1.5 text-[13px] outline-none placeholder:text-muted-foreground"
  />
  <span className="shrink-0 whitespace-nowrap px-1.5 text-[12px] tabular-nums text-muted-foreground" aria-live="polite">
    {counter}
  </span>
  <BarButton label="Previous match (Shift+Enter)" onClick={() => step(-1)} disabled={!locatable.length}>
    <ChevronUp size={17} />
  </BarButton>
  <BarButton label="Next match (Enter)" onClick={() => step(1)} disabled={!locatable.length}>
    <ChevronDown size={17} />
  </BarButton>
  <BarButton
    label="All results"
    onClick={() => {
      setFocusOnShow(false);
      setMode("panel");
    }}
  >
    <ListFilter size={16} />
  </BarButton>
  <BarButton label="Close (Esc)" onClick={onClose}>
    <X size={16} />
  </BarButton>
</div>
</>
);
}

return (
  <>
    {flashRing}
    <div
      data-wb-modal
      // Above the split divider (z-60), which otherwise cuts through the
      // panel and steals taps on it in split view.
      className="fixed inset-0 z-[70] flex items-start justify-center bg-black/30 px-3 pt-3 backdrop-blur-sm sm:pt-[10vh]"
      onClick={onClose}
    >
      <div
        role="dialog"
        aria-label="Search"
        className="flex max-h-[min(80dvh,42rem)] w-full max-w-2xl flex-col overflow-hidden rounded-2xl border border-border bg-white shadow-2xl dark:bg-zinc-900"
        onClick={(e) => e.stopPropagation()}
        onKeyDown={onPanelKeyDown}
      >
        {/* Query */}
        <div className="flex items-center gap-2 border-b border-border px-3 py-2">
          <Search size={17} className="shrink-0 text-muted-foreground" />
          <input
            ref={inputRef}
            value={query}
            onChange={(e) => setQuery(e.target.value)}
            placeholder={

```

```

        scope === "note" ? "Find in this note..." : scope === "folder" ? `Search in ${folderName}...` : "Search everything..."
    }
    dir="auto"
    aria-label="Search text"
    className="h-10 min-w-0 flex-1 bg-transparent text-[15px] outline-none placeholder:text-muted-foreground"
  />
  {query && (
    <button
      type="button"
      onClick={() => {
        setQuery("");
        inputRef.current?.focus();
      }}
      className="shrink-0 rounded-md px-1.5 py-1 text-[11px] font-medium text-muted-foreground hover:bg-muted hover:text-foreground"
    >
      Clear
    </button>
  )}
  <button
    type="button"
    onClick={onClose}
    className="grid h-9 w-9 shrink-0 place-items-center rounded-lg text-muted-foreground hover:bg-muted hover:text-foreground"
    aria-label="Close search"
    title="Close (Esc)"
  >
    <X size={17} />
  </button>
</div>

{/* Scope: where to look. Each segment names the note / folder it means. */}
<div className="border-b border-border px-3 py-2">
  <div role="radiogroup" aria-label="Search scope" className="grid grid-cols-3 gap-1 rounded-xl bg-muted/60 p-1">
    {scopes.map((s) => (
      <button
        key={s.id}
        type="button"
        role="radio"
        aria-checked={scope === s.id}
        disabled={s.disabled}
        onClick={() => {
          setScope(s.id);
          inputRef.current?.focus({ preventScroll: true });
        }}
        className={cn(
          "flex min-h-11 min-w-0 flex-col items-start justify-center rounded-lg px-2.5 py-1 text-left transition",
          scope === s.id
            ? "bg-background text-foreground shadow-sm ring-1 ring-border/60"
            : "text-muted-foreground hover:text-foreground",
          s.disabled && "cursor-not-allowed opacity-45 hover:text-muted-foreground"
        )}
      >
        <span className="flex items-center gap-1.5 text-[12px] font-semibold leading-tight">
          {s.icon}
          {s.label}
        </span>
        <span dir="auto" className="w-full truncate text-[10.5px] leading-tight opacity-75">
          {s.detail}
        </span>
      </button>
    ))}
  </div>
</div>

{/* Results */}
<div className="min-h-0 flex-1 overflow-y-auto overscroll-contain px-1.5 pb-1.5">
  {!trimmed ? (
    <SearchedLegend scope={scope} folderName={folderName} noteName={activeBoard?.name || "this note"} />
  ) : hits.length === 0 ? (
    <div className="px-4 py-10 text-center text-[13px] text-muted-foreground">
      {pdfPending ? "Searching document text..." : <No results for &ldquo;{trimmed}&rdquo;</>}
      {!pdfPending && scope !== "everywhere" && (

```

```

        <div className="mt-3">
          <button
            type="button"
            onClick={() => setScope("everywhere")}
            className="rounded-lg border border-border px-3 py-1.5 text-[12px] font-medium text-foreground
            ground hover:bg-muted"
          >
            Search everywhere instead
          </button>
        </div>
      )}
    </div>
  ) : (
    visibleGroups.map((g) => (
      <section key={g.id} aria-label={g.label}>
        <div className="sticky top-0 z-10 flex items-center gap-2 bg-white/95 px-2 pb-1 pt-2.5 backdrop
        blur dark:bg-zinc-900/95">
          <NearnessGlyph level={g.id} />
          <span dir="auto" className="truncate text-[10.5px] font-semibold uppercase tracking-wider
          text-muted-foreground">
            {g.label}
          </span>
          <span className="ml-auto text-[10.5px] tabular-nums text-muted-foreground">{g.hits.length}</span>
        </div>
        {g.shown.map((hit) => {
          const i = rows.indexOf(hit);
          return (
            <ResultRow
              key={hit.entry.id}
              hit={hit}
              board={hit.entry.boardId ? boardById.get(hit.entry.boardId) : undefined}
              where={whereOf(hit)}
              active={i === cursor}
              current={hit.entry.id === currentId}
              onOpen={() => open(hit)}
              onHover={() => setCursor({ key: resultsKey, i })}
              refCallback={(node) => {
                if (node) rowRefs.current.set(hit.entry.id, node);
                else rowRefs.current.delete(hit.entry.id);
              }}
            </ResultRow>
          )
        })}
      </div>
      <div>
        <button
          type="button"
          onClick={() =>
            setMore({ key: resultsKey, extra: { ...extra, [g.id]: (extra[g.id] ?? 0) + MORE_STEP }
          )
        >
          Show {Math.min(MORE_STEP, g.hidden)} more
          <span className="text-muted-foreground"> of {g.hidden}</span>
        </button>
      </div>
    )}
  </section>
)
  )}
</div>

/* Footer */
<div className="flex items-center justify-between gap-3 border-t border-border px-3 py-1.5 text-[11px]
text-muted-foreground">
  <span className="truncate">
    {trimmed ? `${hits.length} result${hits.length === 1 ? "" : "s"}` : ""}
    {pdfPending && hits.length > 0 && " · reading documents..."}
  </span>
  <span className="hidden shrink-0 items-center gap-1.5 pointer-fine:flex">
    <kbd className="rounded border bg-muted px-1">↑↓</kbd> move
    <kbd className="rounded border bg-muted px-1">↵</kbd> open
    <kbd className="rounded border bg-muted px-1">Ctrl F</kbd> this note
    <kbd className="rounded border bg-muted px-1">Ctrl K</kbd> everywhere
  </span>
</div>
</div>

```

```

        </div>
      </>
    );
  }

// -----
// Pieces
// -----

function BarButton({
  label,
  onClick,
  disabled,
  children,
}): {
  label: string;
  onClick: () => void;
  disabled?: boolean;
  children: React.ReactNode;
} {
  return (
    <button
      type="button"
      onClick={onClick}
      disabled={disabled}
      title={label}
      aria-label={label}
      // 36 px: a comfortable target for a fingertip and the S Pen alike.
      className="grid h-9 w-9 shrink-0 place-items-center rounded-lg text-muted-foreground hover:bg-muted hove
        r:text-foreground disabled:opacity-35"
    >
      {children}
    </button>
  );
}

/**
 * Signal-strength bars for a group: full for this note, fading with distance.
 * Makes "nearest first" readable at a glance without reading the headers.
 */
function NearnessGlyph({ level }: { level: Nearness }) {
  const strength = { note: 3, folder: 2, near: 1, elsewhere: 0 }[level];
  return (
    <span aria-hidden className="flex h-2.5 shrink-0 items-end gap-[2px]">
      {[1, 2, 3].map((n) => (
        <span
          key={n}
          className={cn("w-[3px] rounded-[1px]", n <= strength ? "bg-blue-500" : "bg-muted-foreground/25")}
          style={{ height: `${n * 3 + 1}px` }}
        />
      ))}
    </span>
  );
}

function ResultRow({
  hit,
  board,
  where,
  active,
  current,
  onOpen,
  onHover,
  refCallback,
}): {
  hit: SearchHit;
  board: Board | undefined;
  where: string;
  active: boolean;
  current: boolean;
  onOpen: () => void;
  onHover: () => void;
  refCallback: (node: HTMLButtonElement | null) => void;
} {
  const e = hit.entry;
  const meta = KIND_META[e.kind];
  const Icon = meta.icon;
  const parts = highlightParts(makeSnippet(hit));

```

```

const title = e.kind === "board" || e.kind === "folder" || e.kind === "tag";
const page = e.pageIndex !== undefined ? e.pageIndex + 1 : null;
return (
  <button
    ref={refCallback}
    type="button"
    onClick={onOpen}
    onPointerMove={(ev) => {
      // Only a real mouse moves the cursor; a pen hovering over the list
      // would otherwise drag the highlight around as it approaches.
      if (ev.pointerType === "mouse") onHover();
    }}
    className={cn(
      "relative flex min-h-12 w-full items-center gap-2.5 rounded-xl px-2 py-1.5 text-left transition-colors",
      active ? "bg-muted" : "hover:bg-muted/60"
    )}
  >
    {current && <span aria-hidden className="absolute inset-y-2 left-0 w-[3px] rounded-full bg-blue-500" />}
    <span className={cn("grid h-8 w-8 shrink-0 place-items-center rounded-lg", meta.tint)}>
      {e.kind === "board" && board ? <BoardKindIcon board={board} size={15} /> : <Icon size={15} />}
    </span>
    <span className="min-w-0 flex-1">
      <span dir="auto" className={cn("block truncate text-[13px] text-foreground", title && "font-medium")}>
        {parts.map((p, i) =>
          p.hit ? (
            <mark key={i} className="rounded-[3px] bg-amber-200/90 px-[1px] text-inherit dark:bg-amber-400/35">
              {p.text}
            </mark>
          ) : (
            <React.Fragment key={i}>{p.text}</React.Fragment>
          )
        )}
      </span>
      <span dir="auto" className="block truncate text-[11px] text-muted-foreground">
        {where}
      </span>
    </span>
    {page !== null && (
      <span className="shrink-0 rounded-md bg-muted px-1.5 py-0.5 text-[10.5px] font-medium tabular-nums text-muted-foreground">
        p. {page}
      </span>
    )}
  </button>
);
}

/** The empty panel: exactly what this scope looks through, and what it cannot. */
function SearchedLegend({ scope, folderName, noteName }: { scope: SearchScope; folderName: string; noteName: string }) {
  const where =
    scope === "note" ? `in "${noteName}"` : scope === "folder" ? `in ${folderName} and its sub-folders` : "in every note and folder";
  return (
    <div className="px-3 py-5">
      <p dir="auto" className="text-[12px] text-muted-foreground">
        Searches {where}:
      </p>
      <div className="mt-2.5 flex flex-wrap gap-1.5">
        {SEARCHED[scope].map((k) => {
          const Icon = KIND_META[k].icon;
          return (
            <span
              key={k}
              className="flex items-center gap-1.5 rounded-lg border border-border/70 px-2 py-1 text-[11.5px] text-foreground"
            >
              <span className={cn("grid h-5 w-5 place-items-center rounded-md", KIND_META[k].tint)}>
                <Icon size={12} />
              </span>
              {SEARCHED_LABEL[k]}
            </span>
          );
        })}
      </div>
      <p className="mt-3 text-[11.5px] leading-snug text-muted-foreground">

```

```

    Handwriting, shapes and pictures can't be found - they hold no text. Type into a text box
    or sticky note to make something findable.
    {scope !== "note" && " Results nearest to you come first: this note, its folder, then the folders arou
      nd it."}
  </p>
</div>
);
}

/**
 * A brief ring around the element a search landed on, so the eye finds it
 * after the camera jumps. Positioned in the live pane's screen space and
 * follows the camera while it lasts.
 */
function FlashRing({ boardId, elementId, onDone }: { boardId: string; elementId: string; onDone: () => void })
{
  const board = useActiveBoard();
  const camera = useWhiteboard((s) => s.camera);
  const viewport = useWhiteboard((s) => s.canvasViewport);
  const ref = useRef<HTMLDivElement>(null);
  const doneRef = useRef(onDone);
  useEffect(() => {
    doneRef.current = onDone;
  }, [onDone]);

  useEffect(() => {
    const node = ref.current;
    if (!node || typeof node.animate !== "function") {
      const t = setTimeout(() => doneRef.current(), 1400);
      return () => clearTimeout(t);
    }
    const anim = node.animate(
      [
        { opacity: 0, transform: "scale(1.12)" },
        { opacity: 1, transform: "scale(1)", offset: 0.15 },
        { opacity: 1, transform: "scale(1)", offset: 0.35 },
        { opacity: 0.35, transform: "scale(1)", offset: 0.5 },
        { opacity: 1, transform: "scale(1)", offset: 0.65 },
        { opacity: 0, transform: "scale(1)" },
      ],
      { duration: 1500, easing: "ease-out", fill: "forwards" }
    );
    anim.onfinish = () => doneRef.current();
    return () => anim.cancel();
  }, []);

  if (!board || board.id !== boardId) return null;
  const elements = getBoardElements(board);
  const el = elements.find((x) => x.id === elementId);
  if (!el) return null;
  const b = focusBounds(el, elements);
  const p = worldToScreen(b.x, b.y, camera, viewport);
  const pad = 6;
  return (
    <div
      ref={ref}
      aria-hidden
      className="pointer-events-none absolute z-[26] rounded-xl"
      style={{
        left: p.x - pad,
        top: p.y - pad,
        width: b.width * camera.zoom + pad * 2,
        height: b.height * camera.zoom + pad * 2,
        opacity: 0,
        boxShadow: "0 0 0 3px rgb(59 130 246 / 0.95), 0 0 0 10px rgb(59 130 246 / 0.2)",
      }}
    />
  );
}

=====
// === END FILE: src/components/whiteboard/SearchModal.tsx ===
=====

=====
// === START FILE: src/components/whiteboard/SelectToolButton.tsx ===
// File 73 of 194 | Lines: 401 | Size: 16789 bytes

```

```

=====

"use client";

/**
 * One Select button, three shapes.
 *
 * Rectangle, freeform lasso, and Samsung-style strict encircle (a closed
 * hand-drawn loop that only takes fully enclosed content) do the same job –
 * pick things up – and differ only in the shape you draw, so they share one
 * button with that choice inside it, the way the Shapes button works.
 *
 * A tap selects IMMEDIATELY with whichever shape you used last; only the
 * small chevron opens the menu, and a long-press cycles the three shapes
 * (the chevron is a tiny target for a fingertip or S Pen, and double-tap
 * never reaches a button on Android). The same three live in
 * Settings > Touch & stylus. The menu also holds what the shape picks up
 * (handwriting / text / shapes / images) and Select All, which obeys the
 * same filter.
 *
 * And the Screenshot switch: while it is on, the same box or loop takes a
 * PICTURE of the area - ink, pictures, the paper, the PDF page - instead of
 * picking up objects, for copying, sharing or saving to the gallery. It is a
 * switch on this tool, not a tool of its own, because it is the same gesture
 * aimed at what you SEE rather than at objects. The button wears a camera
 * badge while it is on, so Select can never silently stop selecting.
 */

import React from "react";
import {
  Camera,
  Check,
  ChevronDown,
  CircleDashed,
  Image as ImageIcon,
  LassoSelect,
  ListChecks,
  PenLine,
  Shapes,
  SquareDashedMousePointer,
  Type,
} from "lucide-react";
import { useWhiteboard } from "@/store/whiteboard";
import { useCapture } from "@/store/capture";
import {
  effectiveSelectFilter,
  SELECT_CATEGORIES,
  type SelectCategory,
} from "@/lib/whiteboard/types";
import { useLongPress } from "@/hooks/use-long-press";
import { Popover, PopoverContent, PopoverTrigger } from "@components/ui/popover";
import { Tooltip, TooltipContent, TooltipTrigger } from "@components/ui/tooltip";
import { cn } from "@lib/utls";
import { flyoutSide } from "../DockControls";
import { LockedModeMenuItem } from "../LockedModeBar";

/**
 * The two select shapes, in long-press cycle order.
 *
 * There were briefly three, with "Encircle" as its own shape – but drawing a
 * loop that takes only what it fully contains is the SAME gesture as drawing a
 * loop that takes whatever it touches. The difference is a rule, not a shape,
 * and Samsung Notes has it exactly here: one Lasso, one Rectangle, and a
 * switch called "Include partially selected objects".
 */
export type SelectShape = "rectangle" | "lasso";

const SHAPES: Array<{ shape: SelectShape; label: string; hint: string; icon: typeof LassoSelect }> = [
  { shape: "rectangle", label: "Rectangle", hint: "Drag a box (V)", icon: SquareDashedMousePointer },
  { shape: "lasso", label: "Lasso", hint: "Draw around it (Q)", icon: LassoSelect },
];

const CATEGORY_UI: Record<SelectCategory, { label: string; icon: typeof PenLine }> = {
  ink: { label: "Handwriting", icon: PenLine },
  text: { label: "Text & notes", icon: Type },
  shapes: { label: "Shapes", icon: Shapes },
  images: { label: "Images", icon: ImageIcon },
};

```

```

const LAST_SHAPE_KEY = "pronotes:lastSelectMode";

function readLastShape(): SelectShape {
  try {
    const v = localStorage.getItem(LAST_SHAPE_KEY);
    if (v === "lasso" || v === "rectangle") return v;
    // Saves from when Encircle was its own shape, and older ones that stored
    // the tool id, both mean a loop / a box.
    if (v === "encircle") return "lasso";
    if (v === "select") return "rectangle";
  } catch {
    /* a remembered preference is a convenience, never a requirement */
  }
  return "rectangle";
}

export function SelectToolButton({ vertical }: { vertical: boolean }) {
  const tool = useWhiteboard((s) => s.tool);
  const setTool = useWhiteboard((s) => s.setTool);
  const lassoStrict = useWhiteboard((s) => s.lassoStrict);
  const setLassoStrict = useWhiteboard((s) => s.setLassoStrict);
  const selectAll = useWhiteboard((s) => s.selectAll);
  const savedFilter = useWhiteboard((s) => s.touchSettings.selectFilter);
  const setTouchSettings = useWhiteboard((s) => s.setTouchSettings);
  const toolbarPosition = useWhiteboard((s) => s.toolbarPosition);
  const clearSelection = useWhiteboard((s) => s.clearSelection);
  const flyout = flyoutSide(toolbarPosition);
  const screenshotMode = useCapture((s) => s.screenshotMode);

  const [lastShape, setLastShape] = React.useState<SelectShape>("rectangle");
  const [open, setOpen] = React.useState(false);

  // Read after mount: localStorage is not there during the static export.
  React.useEffect(() => setLastShape(readLastShape()), []);

  /** Each shape is simply its own tool; strictness is separate. */
  const applyShape = React.useCallback(
    (shape: SelectShape) => setTool(shape === "rectangle" ? "select" : "lasso"),
    [setTool]
  );

  // The keyboard (V / Q) and Settings change the shape too; follow them so the
  // button shows what you will actually get.
  const liveShape: SelectShape | null =
    tool === "select" ? "rectangle" : tool === "lasso" ? "lasso" : null;
  React.useEffect(() => {
    if (!liveShape) return;
    setLastShape(liveShape);
    try {
      localStorage.setItem(LAST_SHAPE_KEY, liveShape);
    } catch {
      /* a remembered preference is a convenience, never a requirement */
    }
  }, [liveShape]);

  const active = tool === "select" || tool === "lasso";
  const shown: SelectShape = liveShape ?? lastShape;
  const ShownIcon = SHAPES.find((m) => m.shape === shown)?.icon;
  const nextShape: SelectShape = shown === "rectangle" ? "lasso" : "rectangle";
  const nextLabel = SHAPES.find((m) => m.shape === nextShape)?.label;
  // The shape cycles on long-press: the chevron is a tiny target for a
  // fingertip or S Pen, and double-tap never reaches a button on Android –
  // the same reason the pen picker opens on long-press.
  const modeLongPress = useLongPress(() => applyShape(nextShape));
  const filter = effectiveSelectFilter(savedFilter);
  const filtered = filter.size < SELECT_CATEGORIES.length;

  const toggleCategory = (c: SelectCategory) => {
    const next = new Set(filter);
    if (next.has(c)) {
      // Never let the last one go: a filter that matches nothing makes the
      // tool look broken, which is worse than refusing the click.
      if (next.size === 1) return;
      next.delete(c);
    } else {
      next.add(c);
    }
  }

```

```

    setTouchSettings({ selectFilter: SELECT_CATEGORIES.filter((k) => next.has(k)) });
  });

const activeClass = "bg-primary text-primary-foreground shadow-sm";
const idleClass = "hover:bg-muted text-muted-foreground hover:text-foreground";

return (
  <div
    className={cn(
      "flex rounded-xl transition-all",
      vertical ? "flex-col items-center" : "items-center",
      active ? activeClass : idleClass
    )}
  >
    <Tooltip>
      <TooltipTrigger asChild>
        <button
          {...modeLongPress.handlers}
          onClick={() => {
            // A long press already switched the shape; don't also re-select.
            if (modeLongPress.firedRef.current) return;
            applyShape(shown);
          }}
          aria-label={`Select – ${SHAPES.find((m) => m.shape === shown)?.label}`}
          className={cn(
            "relative p-2.5",
            vertical ? "rounded-t-xl" : "rounded-l-xl",
            !active && "hover:bg-muted"
          )}
        >
          <ShownIcon size={18} />
          {/* A dot says "you are only picking up some things", so a filter
             left on from yesterday cannot quietly confuse you. */}
          {filtered && !screenshotMode && (
            <span className="absolute right-1.5 top-1.5 h-1.5 w-1.5 rounded-full bg-amber-500" />
          )}
          {/* Screenshot mode is on: this tool takes pictures, not objects. */}
          {screenshotMode && (
            <span
              data-screenshot-badge
              className="absolute -bottom-0.5 -right-0.5 flex h-[15px] w-[15px] items-center justify-center
              rounded-full bg-gradient-to-br from-indigo-500 to-violet-500 text-white shadow ring-2 ring-whi
              te dark:ring-zinc-900"
            >
              <Camera size={9} strokeWidth={2.5} />
            </span>
          )}
        </button>
      </TooltipTrigger>
      <TooltipContent side={flyout}>
        {screenshotMode ? "Screenshot" : "Select"} – {SHAPES.find((m) => m.shape === shown)?.label}
        <span className="block text-[10px] opacity-70">Long-press for {nextLabel}</span>
      </TooltipContent>
    </Tooltip>

    <Popover open={open} onOpenChange={setOpen}>
      <PopoverTrigger asChild>
        <button
          aria-label="Selection options"
          className={cn(
            "flex items-center justify-center",
            vertical ? "w-full rounded-b-xl pb-1.5" : "self-stretch rounded-r-xl pr-1.5 pl-0.5",
            !active && "hover:bg-muted"
          )}
        >
          <ChevronDown size={12} className="opacity-60" />
        </button>
      </PopoverTrigger>

      <PopoverContent side={flyout} align="start" className="w-64 p-2">
        <p className="px-1.5 pb-1 text-[10px] font-semibold uppercase tracking-wider text-muted-foreground">
          Shape – tip: long-press the button to switch
        </p>
        <div className="grid grid-cols-2 gap-1">
          {SHAPES.map(({ shape, label, hint, icon: Icon }) => {
            const isCurrent = shown === shape && active;
            return (
              <button

```

```

      key={shape}
      onClick={() => {
        applyShape(shape);
        setOpen(false);
      }}
      aria-pressed={isCurrent}
      className={cn(
        "flex flex-col items-center gap-1 rounded-lg px-2 py-2 text-center transition-colors",
        isCurrent ? "bg-primary text-primary-foreground" : "hover:bg-muted"
      )}
    >
    <Icon size={20} />
    <span className="text-[11px] font-medium leading-none">{label}</span>
    <span
      className={cn(
        "text-[9px] leading-none",
        isCurrent ? "text-primary-foreground/80" : "text-muted-foreground"
      )}
    >
    {hint}
    </span>
  </button>
);
}}}
</div>

/* Screenshot: the same box or loop takes a picture instead of
objects. First among the switches because it changes what the
tool DOES; the ones below only tune what it picks up. */
<button
  onClick={() => {
    const on = !screenshotMode;
    useCapture.getState().setScreenshotMode(on);
    if (on) {
      // Handles left on screen would suggest the selection still
      // works; and the mode acts through Select, so make it the tool.
      clearSelection();
      if (!active) applyShape(shown);
    }
    setOpen(false);
  }}
  role="switch"
  aria-checked={screenshotMode}
  className={cn(
    "mt-1.5 flex w-full items-start gap-2 rounded-lg border px-1.5 py-2 text-left transition-colors"
    ,
    screenshotMode
    ? "border-indigo-500/50 bg-indigo-500/10"
    : "border-transparent hover:bg-muted"
  )}
>
  <span className="flex h-6 w-6 shrink-0 items-center justify-center rounded-md bg-gradient-to-br fr
om-indigo-500 to-violet-500 text-white shadow-sm shadow-indigo-500/30">
    <Camera size={13} />
  </span>
  <span className="min-w-0 flex-1">
    <span className="block text-[11px] font-medium leading-tight">Screenshot</span>
    <span className="block text-[9px] leading-tight text-muted-foreground">
      {screenshotMode
        ? "On: the box or loop takes a picture to copy, share or save"
        : "Take a picture of an area - PDF, pictures and ink"}
    </span>
  </span>
  <span
    className={cn(
      "mt-0.5 h-4 w-7 shrink-0 rounded-full p-0.5 transition-colors",
      screenshotMode ? "bg-indigo-500" : "bg-muted-foreground/30"
    )}
  >
    <span
      className={cn(
        "block h-3 w-3 rounded-full bg-white transition-transform",
        screenshotMode ? "translate-x-3" : "translate-x-0"
      )}
    />
  </span>
</button>

```

```

    /* Samsung's own switch, under its own name: with it on the loop
       takes anything it touches, with it off only what it encloses
       completely. A rule about the loop, not a second kind of loop. */
    <button
      onClick={() => setLassoStrict(!lassoStrict)}
      role="switch"
      aria-checked={!lassoStrict}
      className="mt-1 flex w-full items-start gap-2 rounded-lg px-1.5 py-2 text-left hover:bg-muted"
    >
      <CircleDashed size={14} className="mt-0.5 shrink-0 opacity-70" />
      <span className="min-w-0 flex-1">
        <span className="block text-[11px] font-medium leading-tight">
          Include partially selected
        </span>
        <span className="block text-[9px] leading-tight text-muted-foreground">
          {lassoStrict
            ? "Only what the loop fully encloses"
            : "Anything the loop touches comes along"}
        </span>
      </span>
      <span>
        <span>
          className={cn(
            "mt-0.5 h-4 w-7 shrink-0 rounded-full p-0.5 transition-colors",
            lassoStrict ? "bg-muted-foreground/30" : "bg-primary"
          )}
        </span>
        <span>
          className={cn(
            "block h-3 w-3 rounded-full bg-white transition-transform",
            lassoStrict ? "translate-x-0" : "translate-x-3"
          )}
        </span>
      </span>
    </button>

    <p className="mt-2 px-1.5 pb-1 text-[10px] font-semibold uppercase tracking-wider text-muted-foreground">
      Pick up
    </p>
    <div className="grid grid-cols-2 gap-1">
      {SELECT_CATEGORIES.map((c) => {
        const on = filter.has(c);
        const { label, icon: Icon } = CATEGORY_UI[c];
        const lastOne = on && filter.size === 1;
        return (
          <button
            key={c}
            onClick={() => toggleCategory(c)}
            title={lastOne ? "At least one kind must stay on" : undefined}
            aria-pressed={on}
            className={cn(
              "flex items-center gap-1.5 rounded-lg border px-2 py-1.5 text-[11px] transition-colors",
              on
                ? "border-primary/40 bg-primary/10 text-foreground"
                : "border-border text-muted-foreground hover:bg-muted",
              lastOne && "cursor-not-allowed"
            )}
          >
            <Icon size={13} />
            <span className="flex-1 text-left">{label}</span>
            {on && <Check size={12} className="text-primary" />}
          </button>
        );
      })}
    </div>
    {filtered && (
      <button
        onClick={() => setTouchSettings({ selectFilter: [...SELECT_CATEGORIES] })}
        className="mt-1 w-full rounded-md px-1.5 py-1 text-left text-[10px] text-primary hover:bg-primary/10"
      >
        Pick up everything again
      </button>
    )}
  </div>
  <div className="mt-2 border-t border-border/60 pt-2">
    <button
      onClick={() => {

```

```

        if (!active) applyShape(shown);
        selectAll();
        setOpen(false);
    }}
    className="flex w-full items-center gap-2 rounded-lg px-2 py-1.5 text-[12px] hover:bg-muted"
  >
    <ListChecks size={15} />
    <span className="flex-1 text-left">
      Select all {filtered ? "matching" : ""} on this page
    </span>
    <span className="text-[10px] text-muted-foreground">Ctrl+A</span>
  </button>
  <LockedModeMenuItem onDone={() => setOpen(false)} />
</div>
</PopoverContent>
</Popover>
</div>
);
}

```

```

=====
// === END FILE: src/components/whiteboard/SelectToolButton.tsx ===
=====

```

```

=====
// === START FILE: src/components/whiteboard/SelectionActions.tsx ===
// File 74 of 194 | Lines: 217 | Size: 9270 bytes
=====

```

```

"use client";

/**
 * Floating Selection Actions Bar (v3 – Obsidian Canvas Grade)
 * =====
 * Contextual floating pill positioned directly above selected text or sticky cards.
 *
 * Every OTHER selection - ink, shapes, pictures, several things at once - gets
 * a single Capture chip instead, beside the selection's top-right corner:
 * clear of the rotate handle (top centre) and of the picture bar (below).
 * Capture takes a picture of the selection's area, background and all; the
 * capture panel can narrow it to the selected things only.
 */

import React, { useState } from "react";
import { useWhiteboard, useActiveBoard } from "@store/whiteboard";
import { getActivePageIndex, getBoardElements, isNotebook } from "@lib/whiteboard/notebook";
import { Link2, Hash, Edit3, Plus, Scan } from "lucide-react";
import { Popover, PopoverContent, PopoverTrigger } from "@components/ui/popover";
import { getElementBounds, getUnionBounds, type Viewport } from "@lib/whiteboard/geometry";
import { clampCaptureRegion, selectionCaptureRegion } from "@lib/whiteboard/capture";
import type { Board } from "@lib/whiteboard/types";
import { useCapture } from "@store/capture";

/** Opens the capture panel on the selection's area. */
function captureSelection(board: Board, ids: string[], zoom: number) {
  const region = selectionCaptureRegion(getBoardElements(board), ids, zoom);
  if (!region) return;
  const anchorIndex = isNotebook(board) ? Math.max(0, getActivePageIndex(board)) : 0;
  // A selection hanging off the sheet is captured as far as the paper goes,
  // the same trim a drawn area gets.
  const clamped = clampCaptureRegion(board, anchorIndex, region) ?? region;
  useCapture.getState().open({
    source: { board, region: clamped, anchorIndex },
    origin: "selection",
    shape: "rectangle",
    selectionIds: ids,
    zoom,
    dpr: typeof window !== "undefined" ? window.devicePixelRatio || 1 : 1,
  });
}

export function SelectionActions({ viewport }: { viewport: Viewport }) {
  const selectedIds = useWhiteboard((s) => s.selectedIds);
  const editingId = useWhiteboard((s) => s.editingId);
  const setEditingId = useWhiteboard((s) => s.setEditingId);
  const setTool = useWhiteboard((s) => s.setTool);
  const camera = useWhiteboard((s) => s.camera);

```

```
const activeBoard = useActiveBoard();
const boards = useWhiteboard((s) => s.boards);
const updateElement = useWhiteboard((s) => s.updateElement);

const studioOpen = useWhiteboard((s) => s.imageStudioId !== null);
const capturing = useCapture((s) => s.request !== null);

const [tagInput, setTagInput] = useState("");
const [popoverOpen, setPopoverOpen] = useState(false);

if (selectedIds.length === 0 || editingId !== null || !activeBoard || studioOpen || capturing) {
  return null;
}

const selectedElement =
  selectedIds.length === 1 ? getBoardElements(activeBoard).find((el) => el.id === selectedIds[0]) : undefined;
d;
if (!selectedElement || (selectedElement.type !== "text" && selectedElement.type !== "sticky")) {
  return <CaptureChip board={activeBoard} ids={selectedIds} viewport={viewport} />;
}

const bounds = getElementBounds(selectedElement);
const screenX = (bounds.x - camera.x) * camera.zoom + viewport.width / 2;
const screenY = (bounds.y - camera.y) * camera.zoom + viewport.height / 2;
const screenW = bounds.width * camera.zoom;

const barX = screenX + screenW / 2;
const barY = Math.max(screenY - 44, 20);

const currentText = (selectedElement as { text: string }).text || "";

const insertLink = (boardName: string) => {
  const linkSyntax = `[[${boardName}]]`;
  const newText =
    currentText +
    (currentText.endsWith(" ") || currentText === "" ? "" : " ") +
    linkSyntax +
    " ";
  updateElement(selectedElement.id, { text: newText }, true);
  setPopoverOpen(false);
};

const insertTag = (e: React.FormEvent) => {
  e.preventDefault();
  const tag = tagInput.trim().replace(/^#/ , "");
  if (!tag) return;
  const tagSyntax = `#${tag}`;
  const newText =
    currentText +
    (currentText.endsWith(" ") || currentText === "" ? "" : " ") +
    tagSyntax +
    " ";
  updateElement(selectedElement.id, { text: newText }, true);
  setTagInput("");
};

return (

```

```

        <button className="flex items-center gap-1 px-2 py-1 rounded-lg hover:bg-muted font-medium text-foreground"
            ground">
            <Link2 size={13} />
            <span>Link to Board</span>
        </button>
    </PopoverTrigger>
    <PopoverContent className="w-60 p-1.5 text-xs" align="center">
        <div className="px-2 py-1 text-[10px] font-semibold text-muted-foreground uppercase tracking-wider">
            Select Board to Link
        </div>
        <div className="max-h-48 overflow-y-auto space-y-0.5">
            {boards
                .filter((b) => b.id !== activeBoard.id)
                .map((b) => (
                    <button
                        key={b.id}
                        onClick={() => insertLink(b.name)}
                        className="w-full text-left px-2.5 py-1.5 rounded-md hover:bg-muted flex items-center justify-between text-xs transition-colors"
                    >
                        <span className="truncate">{b.name}</span>
                        <span className="text-[10px] text-muted-foreground">[...]</span>
                    </button>
                ))}
        </div>
    </PopoverContent>
</Popover>

<span className="w-px h-3.5 bg-border mx-0.5" />

{/* Add Tag Form */}
<form onSubmit={insertTag} className="flex items-center gap-1">
    <div className="flex items-center gap-0.5 px-1.5 py-0.5 rounded-lg bg-muted/60 border border-border/50"
        >
        <Hash size={11} className="text-emerald-500" />
        <input
            type="text"
            value={tagInput}
            onChange={(e) => setTagInput(e.target.value)}
            placeholder="tag"
            className="w-14 bg-transparent outline-none text-[11px]"
        />
    </div>
    {tagInput.trim() && (
        <button type="submit" className="p-1 rounded hover:bg-muted text-emerald-600">
            <Plus size={12} />
        </button>
    )}
</form>

<span className="w-px h-3.5 bg-border mx-0.5" />

<button
    onClick={() => captureSelection(activeBoard, selectedIds, camera.zoom)}
    title="Capture a picture of this to copy or share"
    className="flex items-center gap-1 px-2 py-1 rounded-lg hover:bg-muted font-medium text-indigo-600 dark:text-indigo-300"
>
    <Scan size={13} />
    <span>Capture</span>
</button>
</div>
);
}

/** The one-button bar for every selection that is not a text box or note. */
function CaptureChip({ board, ids, viewport }: { board: Board; ids: string[]; viewport: Viewport }) {
    const camera = useWhiteboard((s) => s.camera);
    const set = new Set(ids);
    const picked = getBoardElements(board).filter((el) => set.has(el.id) && el.type !== "connection");
    const b = getUnionBounds(picked);
    if (!b) return null;

    const left = (b.x - camera.x) * camera.zoom + viewport.width / 2;
    const right = left + b.width * camera.zoom;
    const top = (b.y - camera.y) * camera.zoom + viewport.height / 2;
    const chipW = 104;
    // Beside the top-right corner handle; flipped to the left edge when the

```

```
// selection runs off the right of the screen, and kept on screen always.
const x = right + 14 + chipW <= viewport.width - 8 ? right + 14 : Math.max(8, left - 14 - chipW);
const y = Math.min(Math.max(top - 6, 8), viewport.height - 48);

return (
  <button
    onClick={() => captureSelection(board, ids, camera.zoom)}
    onPointerDown={(e) => e.stopPropagation()}
    onMouseDown={(e) => e.stopPropagation()}
    title="Capture a picture of the selection to copy, share or add to a notebook"
    className="absolute z-30 flex h-9 items-center gap-1.5 rounded-full border-white/10 bg-zinc-900/90
      5 pl-2 pr-3.5 text-[12px] font-semibold text-white shadow-xl shadow-black/20 backdrop-blur-md transi
      tion hover:bg-zinc-800 active:scale-95 animate-in fade-in zoom-in-95"
    style={{ left: x, top: y, width: chipW }}
  >
    <span className="flex h-6 w-6 items-center justify-center rounded-full bg-gradient-to-br from-indigo-500
      to-violet-500">
      <Scan size={13} />
    </span>
    Capture
  </button>
);
}
```

```
=====
// === END FILE: src/components/whiteboard/SelectionActions.tsx ===
=====
```

```
=====
// === START FILE: src/components/whiteboard/SelectionOverlay.tsx ===
// File 75 of 194 | Lines: 189 | Size: 6007 bytes
=====
```

```
"use client";

/**
 * Selection Overlay (v3)
 * =====
 * Renders the selection bounding box and 8 interactive resize/rotation handles.
 */

import React from "react";
import { useWhiteboard } from "@store/whiteboard";
import type { Camera, WhiteboardElement } from "@lib/whiteboard/types";
import {
  getElementBounds,
  getResizeHandles,
  getUnionBounds,
  screenSpaceOverlayTransform,
  type Viewport,
} from "@lib/whiteboard/geometry";

interface SelectionOverlayProps {
  selectedIds: string[];
  elements: WhiteboardElement[];
  camera: Camera;
  viewport: Viewport;
}

export function SelectionOverlay({
  selectedIds,
  elements,
  camera,
  viewport,
}: SelectionOverlayProps) {
  const editingId = useWhiteboard((s) => s.editingId);

  const selected = elements.filter((el) => selectedIds.includes(el.id));
  if (selected.length === 0) return null;

  // Don't render selection overlay while the element is actively being edited inline
  if (editingId && selectedIds.includes(editingId)) return null;

  const unionBounds = getUnionBounds(selected);
  if (!unionBounds) return null;

  const rotation = selected.length === 1 ? selected[0].rotation : 0;
```

```

// Locked things can be selected (to unlock them) but never resized or
// turned, so they get the box without handles that would do nothing.
const allLocked = selected.every((el) => el.locked);
const handles = allLocked ? [] : getResizeHandles(unionBounds, rotation, camera, viewport);

const cx = unionBounds.x + unionBounds.width / 2;
const cy = unionBounds.y + unionBounds.height / 2;
const corners = [
  { x: unionBounds.x, y: unionBounds.y },
  { x: unionBounds.x + unionBounds.width, y: unionBounds.y },
  { x: unionBounds.x + unionBounds.width, y: unionBounds.y + unionBounds.height },
  { x: unionBounds.x, y: unionBounds.y + unionBounds.height },
].map((c) => {
  const cos = Math.cos(rotation);
  const sin = Math.sin(rotation);
  const dx = c.x - cx;
  const dy = c.y - cy;
  const rx = cx + dx * cos - dy * sin;
  const ry = cy + dx * sin + dy * cos;
  return {
    x: (rx - camera.x) * camera.zoom + viewport.width / 2,
    y: (ry - camera.y) * camera.zoom + viewport.height / 2,
  };
});

// Drawn size, and a separate, invisible TOUCH target around each handle.
// The handles used to be 9 px and were also the only thing you could hit:
// about 2 mm on a Galaxy Tab S9, against Android's 48 dp touch-target
// guideline, so resize and rotate were effectively unreachable by finger or
// pen. The hit circle below is 44 px across; the visible handle stays small
// enough not to cover the picture.
const handleSize = 14;
const handleStrokeWidth = 2;
const HIT_R = 22;

// Handles/corners above are in SCREEN space; undo the world group so they
// land where the element actually is at every zoom level.
const inverseTransform = screenSpaceOverlayTransform(camera, viewport);

return (
  <g transform={inverseTransform} style={{ pointerEvents: "none" }}>
    { /* Selection bounding box */ }
    <polygon
      points={corners.map((c) => `${c.x},${c.y}`).join(" ")}
      fill="none"
      stroke="#6366f1"
      strokeWidth={handleStrokeWidth}
      strokeDasharray="5,4"
    />

    { /* Resize & Rotate handles */ }
    {handles.map((h) => {
      if (h.handle === "rotate") {
        const topCenter = {
          x: (corners[0].x + corners[1].x) / 2,
          y: (corners[0].y + corners[1].y) / 2,
        };
        return (
          <g key={h.handle}>
            <line
              x1={topCenter.x}
              y1={topCenter.y}
              x2={h.x}
              y2={h.y}
              stroke="#6366f1"
              strokeWidth={handleStrokeWidth}
            />
            <circle
              cx={h.x}
              cy={h.y}
              r={handleSize / 2}
              fill="white"
              stroke="#6366f1"
              strokeWidth={handleStrokeWidth}
            />
            <circle
              cx={h.x}
              cy={h.y}

```

```

        r={HIT_R}
        fill="transparent"
        style={{ pointerEvents: "all", cursor: "grab" }}
        data-handle="rotate"
        data-element-id={selected[0].id}
      />
    </g>
  );
}

const cursor = getHandleCursor(h.handle, rotation);
return (
  <g key={h.handle}>
    <rect
      x={h.x - handleSize / 2}
      y={h.y - handleSize / 2}
      width={handleSize}
      height={handleSize}
      fill="white"
      stroke="#6366f1"
      strokeWidth={handleStrokeWidth}
      rx={3}
    />
    <circle
      cx={h.x}
      cy={h.y}
      r={HIT_R}
      fill="transparent"
      style={{ pointerEvents: "all", cursor }}
      data-handle={h.handle}
      data-element-id={selected[0].id}
    />
  </g>
);
  })}
</g>
);
}
}

function getHandleCursor(handle: string, rotation: number): string {
  const deg = ((rotation * 180) / Math.PI) % 360;
  const normalized = ((deg % 360) + 360) % 360;
  const isDiagonal =
    (normalized > 22.5 && normalized < 67.5) ||
    (normalized > 112.5 && normalized < 157.5) ||
    (normalized > 202.5 && normalized < 247.5) ||
    (normalized > 292.5 && normalized < 337.5);
  const isFlipped =
    (normalized > 67.5 && normalized < 112.5) || (normalized > 247.5 && normalized < 292.5);

  switch (handle) {
    case "n":
    case "s":
      return isFlipped ? "ew-resize" : "ns-resize";
    case "e":
    case "w":
      return isFlipped ? "ns-resize" : "ew-resize";
    case "ne":
    case "sw":
      return isDiagonal ? "nwse-resize" : "nesw-resize";
    case "nw":
    case "se":
      return isDiagonal ? "nesw-resize" : "nwse-resize";
    default:
      return "default";
  }
}

```

```

=====
// === END FILE: src/components/whiteboard/SelectionOverlay.tsx ===
=====

```

```

=====
// === START FILE: src/components/whiteboard/SettingsPanel.tsx ===
// File 76 of 194 | Lines: 797 | Size: 36425 bytes
=====

```

```

"use client";

/**
 * Settings
 * =====
 * One home for everything that used to be scattered across the toolbar as
 * always-visible toggles. The bottom dock is for TOOLS you reach constantly;
 * anything you set once and forget lives here.
 *
 * This is also the single owner of the theme control – it used to exist twice
 * (header and toolbar), which is why two sun/moon icons were on screen.
 */

import React from "react";
import { useTheme } from "next-themes";
import { useWhiteboard, useActiveBoard } from "@store/whiteboard";
import {
  BOARD_BACKGROUNDs,
  BOARD_PATTERNS,
  BOARD_PATTERN_DEFAULT_SIZE,
  BOARD_PATTERN_MAX_SIZE,
  BOARD_PATTERN_MIN_SIZE,
  BOARD_PATTERN_SIZE_PRESETS,
  PAGE_SIZES,
} from "@lib/whiteboard/types";
import type { PageSizeId, TouchInputMode, PressureCurve } from "@lib/whiteboard/types";
import { isNotebook, pageSizeOf } from "@lib/whiteboard/notebook";
import {
  Grid,
  HelpCircle,
  Keyboard,
  Layers,
  Map as MapIcon,
  Monitor,
  Moon,
  Pointer,
  Sparkles,
  Star,
  Sun,
  Tag,
  Wand2,
  X,
  ZoomIn,
} from "lucide-react";
import { Button } from "@components/ui/button";
import { StepperRow } from "../StepperControl";
import { cn } from "@lib/utils";

interface Props {
  open: boolean;
  onClose: () => void;
  legendVisible: boolean;
  onToggleLegend: () => void;
}

/** A labelled on/off row – the repeated shape of most settings here. */
function ToggleRow({
  icon,
  label,
  hint,
  checked,
  onChange,
}: {
  icon: React.ReactNode;
  label: string;
  hint?: string;
  checked: boolean;
  onChange: () => void;
}) {
  return (
    <button
      onClick={onChange}
      role="switch"
      aria-checked={checked}
      className="flex w-full items-center gap-3 rounded-lg px-2 py-2 text-left transition-colors hover:bg-mute
        d"
    >
      <span className="text-muted-foreground">{icon}</span>

```

```

<span className="min-w-0 flex-1">
  <span className="block text-xs font-medium">{label}</span>
  {hint && <span className="block text-[11px] leading-snug text-muted-foreground">{hint}</span>}
</span>
<span
  className={cn(
    "relative h-5 w-9 shrink-0 rounded-full transition-colors",
    checked ? "bg-primary" : "bg-muted-foreground/30"
  )}
>
  <span
    className={cn(
      "absolute top-0.5 h-4 w-4 rounded-full bg-white shadow transition-all",
      checked ? "left-[1.125rem]" : "left-0.5"
    )}
  />
</span>
</button>
);
}

function SectionTitle({ children }: { children: React.ReactNode }) {
  return (
    <h3 className="px-2 pb-1 pt-1 text-[11px] font-semibold uppercase tracking-wider text-muted-foreground">
      {children}
    </h3>
  );
}

export function SettingsPanel({ open, onClose, legendVisible, onToggleLegend }: Props) {
  const { theme, setTheme, resolvedTheme } = useTheme();
  const activeBoard = useActiveBoard();
  const setBoardBackground = useWhiteboard((s) => s.setBoardBackground);
  const setPageSize = useWhiteboard((s) => s.setPageSize);
  const pageDims = pageSizeOf(activeBoard);
  const paperSize = activeBoard?.background?.size ?? BOARD_PATTERN_DEFAULT_SIZE;

  const minimapOpen = useWhiteboard((s) => s.minimapOpen);
  const toggleMinimap = useWhiteboard((s) => s.toggleMinimap);
  const defaultPaper = useWhiteboard((s) => s.defaultPaper);
  const setDefaultPaper = useWhiteboard((s) => s.setDefaultPaper);
  // Scope for the paper controls below. Off = the notebook default only (new
  // pages start with it); on = every existing page is rewritten too. Always
  // starts off – rewriting history is opt-in per session.
  const [applyAllPages, setApplyAllPages] = React.useState(false);
  const pageScope = { allPages: applyAllPages };
  const snapToGrid = useWhiteboard((s) => s.snapToGrid);
  const toggleSnapToGrid = useWhiteboard((s) => s.toggleSnapToGrid);
  const zoomDockOpen = useWhiteboard((s) => s.zoomDockOpen);
  const toggleZoomDock = useWhiteboard((s) => s.toggleZoomDock); const pinnedBarOpen = useWhiteboard((s) => s
    .pinnedBarOpen);
  const togglePinnedBar = useWhiteboard((s) => s.togglePinnedBar);
  const touchSettings = useWhiteboard((s) => s.touchSettings);
  const setTouchSettings = useWhiteboard((s) => s.setTouchSettings);
  const tool = useWhiteboard((s) => s.tool);
  const setTool = useWhiteboard((s) => s.setTool);
  const lassoStrict = useWhiteboard((s) => s.lassoStrict);
  const setLassoStrict = useWhiteboard((s) => s.setLassoStrict);

  if (!open) return null;

  const themes: Array<{ id: string; label: string; icon: React.ReactNode }> = [
    { id: "light", label: "Light", icon: <Sun size={13} /> },
    { id: "dark", label: "Dark", icon: <Moon size={13} /> },
    { id: "system", label: "System", icon: <Monitor size={13} /> },
  ];

  const touchModes: Array<{ id: TouchInputMode; label: string; hint: string }> = [
    { id: "touch-draw", label: "Touch & pen draw", hint: "Fingers and stylus both draw" },
    { id: "stylus-only", label: "Stylus only", hint: "Fingers pan and zoom – palm rejection" },
    { id: "touch-pan", label: "Touch navigation", hint: "Fingers only pan; draw with the pen" },
  ];

  return (
    <div
      data-wb-modal
      className="fixed inset-0 z-[90] flex items-center justify-center bg-black/50 backdrop-blur-sm"
      onClick={onClose}
    >

```

```

>
<div
  className="flex max-h-[80vh] w-full max-w-sm flex-col overflow-hidden rounded-2xl border border-border
    bg-white shadow-2xl dark:bg-zinc-900"
  onClick={(e) => e.stopPropagation()}
>
  <div className="flex items-center justify-between border-b border-border px-5 py-3">
    <h2 className="text-base font-semibold">Settings</h2>
    <Button variant="ghost" size="icon" className="h-7 w-7" onClick={onClose}>
      <X size={16} />
    </Button>
  </div>

  {/* A plain scrolling div, deliberately not Radix's ScrollArea.
    Radix's viewport is sized with height:100%, which needs a parent
    with a definite height; inside a flex column it resolved to the
    content height instead, so the panel grew past the card and the
    bottom of Settings was unreachable. `min-h-0 flex-1 overflow-y-auto`
    needs no percentage resolution and cannot be overridden by the
    inline styles Radix sets on its own root. */}
  <div className="min-h-0 flex-1 overflow-y-auto overscroll-contain">
    <div className="space-y-3 p-3">
      {/* ---- Appearance ---- */}
      <section>
        <SectionTitle>Appearance</SectionTitle>
        <div className="flex gap-1 rounded-lg bg-muted p-0.5">
          {themes.map((t) => (
            <button
              key={t.id}
              onClick={() => setTheme(t.id)}
              className={cn(
                "flex flex-1 items-center justify-center gap-1.5 rounded-md px-2 py-1.5 text-xs font-med
                ium transition-colors",
                theme === t.id
                  ? "bg-background shadow-sm"
                  : "text-muted-foreground hover:text-foreground"
              )}
            >
              {t.icon}
              {t.label}
            </button>
          ))}
        </div>
      </section>

      {/* ---- This board's paper ---- */}
      <section>
        <SectionTitle>Board background</SectionTitle>
        <div className="px-2 pb-1">
          <div className="mb-1.5 flex flex-wrap items-center gap-1.5">
            {BOARD_BACKGROUNDS.map((bg) => {
              const swatch =
                bg.id === "default"
                  ? "linear-gradient(135deg,#ffffff 50%,#18181b 50%)"
                  : resolvedTheme === "dark"
                    ? bg.dark
                    : bg.light;
              const active = (activeBoard?.background?.color ?? "default") === bg.id;
              return (
                <button
                  key={bg.id}
                  title={bg.label}
                  onClick={() =>
                    activeBoard &&
                    setBoardBackground(
                      activeBoard.id,
                      {
                        color: bg.id === "default" ? undefined : bg.id,
                      },
                      pageScope
                    )
                >
                  <div className={cn(
                    "h-7 w-7 rounded-full border transition-transform",
                    active
                      ? "scale-110 border-foreground ring-2 ring-primary"
                      : "border-black/15 hover:scale-105 dark:border-white/20"
                  )}
                >

```

```

        style={{ background: swatch }}
      />
    );
  }}
}
}
/* Any colour at all, stored literally rather than by preset id. */
<label
  title="Custom background colour..."
  className="relative h-7 w-7 cursor-pointer rounded-full border border-dashed border-border
"
  style={{
    background:
      "conic-gradient(#ef4444,#f97316,#eab308,#22c55e,#06b6d4,#3b82f6,#8b5cf6,#ec4899,#ef444
4)",
  }}
>
  <input
    type="color"
    className="absolute inset-0 h-full w-full cursor-pointer opacity-0"
    onChange={(e) =>
      activeBoard &&
      setBoardBackground(activeBoard.id, { color: e.target.value }, pageScope)
    }
  />
  <span className="pointer-events-none absolute inset-[7px] rounded-full bg-background" />
</label>
</div>

<div className="mb-1 text-[11px] text-muted-foreground">Paper</div>
<div className="flex gap-1">
  {BOARD_PATTERNS.map((pt) => (
    <button
      key={pt.id}
      onClick={() =>
        activeBoard && setBoardBackground(activeBoard.id, { pattern: pt.id }, pageScope)
      }
      className={cn(
        "flex-1 rounded-lg border px-2 py-1.5 text-[11px] font-medium transition-colors",
        (activeBoard?.background?.pattern ?? "dots") === pt.id
          ? "border-primary bg-primary/10 text-primary"
          : "border-border hover:bg-muted"
      )}
    >
      {pt.label}
    </button>
  ))}
</div>

/* Paper size. Hidden for "Plain" – there is no spacing to set
when nothing is drawn. */
{(activeBoard?.background?.pattern ?? "dots") !== "plain" && (
  <div className="mt-2.5">
    <div className="mb-1 flex items-center justify-between">
      <span className="text-[11px] text-muted-foreground">Paper size</span>
      <span className="rounded-md bg-muted px-1.5 py-0.5 text-[11px] font-semibold tabular-num
s">
        {paperSize}px
      </span>
    </div>
    <StepperRow
      value={paperSize}
      min={BOARD_PATTERN_MIN_SIZE}
      max={BOARD_PATTERN_MAX_SIZE}
      onChange={(size) =>
        activeBoard && setBoardBackground(activeBoard.id, { size }, pageScope)
      }
    >
      <input
        type="range"
        min={BOARD_PATTERN_MIN_SIZE}
        max={BOARD_PATTERN_MAX_SIZE}
        value={paperSize}
        aria-label="Paper size"
        onChange={(e) =>
          activeBoard &&
          setBoardBackground(activeBoard.id, { size: Number(e.target.value) }, pageScope)
        }
        className="h-1.5 w-full cursor-pointer appearance-none rounded-lg bg-muted accent-prim
ary"

```

```

    />
  </StepperRow>
  <div className="mt-1.5 flex gap-1">
    {BOARD_PATTERN_SIZE_PRESETS.map((sz) => (
      <button
        key={sz}
        onClick={() =>
          activeBoard && setBoardBackground(activeBoard.id, { size: sz }, pageScope)
        }
        className={cn(
          "flex-1 rounded-md border px-1.5 py-1 text-[10px] font-medium transition-colors",
          paperSize === sz
            ? "border-primary bg-primary/10 text-primary"
            : "border-border text-muted-foreground hover:bg-muted"
        )}
      >
        {sz}
      </button>
    ))}
  </div>
)}

<p className="mt-1.5 text-[11px] leading-snug text-muted-foreground">
  The default for <strong>{activeBoard?.name ?? "this board"}</strong> – new
  pages start with it, and pages without their own paper follow it.
  Pages you styled individually keep theirs. Snap-to-grid follows
  this spacing.
</p>
{ /* Scope: future pages only, or every existing page too. The
  panel always edits the board you last touched (in split view
  that is the last-used pane) – never the whole app. */ }
{isNotebook(activeBoard) && (
  <div className="mt-1">
    <ToggleRow
      icon={<Layers size={15} />}
      label="Apply to all pages"
      hint="Writes the current paper onto every existing page now, and keeps later changes in
step"
      checked={applyAllPages}
      onChange={() => {
        const next = !applyAllPages;
        setApplyAllPages(next);
        // Flipping the toggle must DO something, not just arm
        // future taps: turning it on stamps the board's current
        // paper onto every page immediately.
        if (next && activeBoard?.background) {
          setBoardBackground(
            activeBoard.id,
            { ...activeBoard.background },
            { allPages: true }
          );
        }
      }}
    </ToggleRow>
  </div>
)}

<div className="mt-1.5 flex items-center justify-between gap-2">
  <span className="text-[11px] text-muted-foreground">
    App default:{" "}
    <strong>
      {BOARD_PATTERNS.find((p) => p.id === defaultPaper.pattern)?.label ??
        "Plain"}
      {defaultPaper.pattern !== "plain" &&
        ` · ${defaultPaper.size ?? BOARD_PATTERN_DEFAULT_SIZE}px`}
    </strong>
  </span>
  <button
    onClick={() =>
      activeBoard?.background && setDefaultPaper({ ...activeBoard.background })
    }
    title="Use this board's paper for all future boards and pages"
    className="shrink-0 rounded-lg border border-border px-2 py-1 text-[11px] font-medium tran
sition-colors hover:bg-muted"
  >
    Set as app default
  </button>
</div>

```

```

    </div>
  </section>

  {/* ---- Notebook ---- */}
  {isNotebook(activeBoard) && (
    <section>
      <SectionTitle>Notebook</SectionTitle>
      <div className="px-2 pb-1">
        <div className="mb-1 flex items-center justify-between">
          <span className="text-[11px] text-muted-foreground">Page size</span>
          <span className="rounded-md bg-muted px-1.5 py-0.5 text-[11px] font-semibold tabular-nums"
            >
            {pageDims.width} × {pageDims.height}
          </span>
        </div>
      </div>
      {/* A PDF notebook uses its document's page size – the buttons
        below change `pageSize`, which pageSizeOf ignores while a
        valid custom size exists, so they would silently do
        nothing here. */}
      {activeBoard?.customPageSize &&
        activeBoard.customPageSize.width > 0 &&
        activeBoard.customPageSize.height > 0 ? (
        <p className="mt-1.5 text-[11px] leading-snug text-muted-foreground">
          This notebook uses its PDF's page size.
        </p>
        ) : (
        <div className="grid grid-cols-2 gap-1">
          {(Object.keys(PAGE_SIZES) as PageSizeId[]).map((id) => (
            <button
              key={id}
              onClick={() => setPageSize(id)}
              className={cn(
                "rounded-lg border px-2 py-1.5 text-left text-[11px] transition-colors",
                (activeBoard?.pageSize ?? "a4") === id
                  ? "border-primary bg-primary/10 text-primary"
                  : "border-border hover:bg-muted"
              )}
            >
              {PAGE_SIZES[id].label}
            </button>
          ))}
        </div>
        )}
      <p className="mt-1.5 text-[11px] leading-snug text-muted-foreground">
        Applies to every page of{" "}
        <strong>{activeBoard?.name ?? "this notebook"}</strong>. Existing ink is not
        moved or scaled, so shrinking the page can push content past its edge.
      </p>
    </div>
  </section>
)}

  {/* ---- Canvas ---- */}
  <section>
    <SectionTitle>Canvas</SectionTitle>
    <ToggleRow
      icon={<Grid size={15} />}
      label="Snap to grid"
      hint="Aligns to the paper spacing (freehand strokes stay free)"
      checked={snapToGrid}
      onChange={toggleSnapToGrid}
    />
    <ToggleRow
      icon={<ZoomIn size={15} />}
      label="Zoom controls"
      hint="Bottom-left readout that appears while you zoom, then fades"
      checked={zoomDockOpen}
      onChange={toggleZoomDock}
    />
    <ToggleRow
      icon={<MapIcon size={15} />}
      label="Minimap"
      hint="Corner navigator showing the whole board"
      checked={minimapOpen}
      onChange={toggleMinimap}
    />
    <ToggleRow
      icon={<Tag size={15} />}

```

```

      label="Link legend"
      hint="Colour key for connection types"
      checked={legendVisible}
      onChange={onToggleLegend}
    />
  </section>

  {/ * ---- Pens ---- */}
  <section>
    <SectionTitle>Pens</SectionTitle>
    <ToggleRow
      icon={<Star size={15} />}
      label="Favorite pens bar"
      hint="Pin pens by type, colour and size; drag to move it"
      checked={pinnedBarOpen}
      onChange={togglePinnedBar}
    />
  </section>

  {/ * ---- Touch & stylus ---- */}
  <section>
    <SectionTitle>Touch & stylus</SectionTitle>
    <div className="space-y-1 px-2 pb-1">
      {touchModes.map((m) => (
        <button
          key={m.id}
          onClick={() => setTouchSettings({ mode: m.id })}
          className={cn(
            "flex w-full flex-col items-start rounded-lg border px-2.5 py-1.5 text-left transition-c
olors",
            touchSettings.mode === m.id
              ? "border-primary bg-primary/10"
              : "border-border hover:bg-muted"
          )}
        >
          <span className="text-xs font-medium">{m.label}</span>
          <span className="text-[11px] leading-snug text-muted-foreground">{m.hint}</span>
        </button>
      )})}
    </div>
    <div className="px-2 pb-1 pt-1">
      <div className="mb-1 text-[11px] text-muted-foreground">Select shape</div>
      <div className="flex gap-1">
        {
          [
            { id: "rectangle", label: "Box", hint: "Drag a box" },
            { id: "lasso", label: "Lasso", hint: "Draw around it with the pen" },
          ] as const
        ).map((m) => {
          const current =
            tool === "select" ? "rectangle" : tool === "lasso" ? "lasso" : null;
          return (
            <button
              key={m.id}
              onClick={() => setTool(m.id === "rectangle" ? "select" : "lasso")}
              title={m.hint}
              aria-pressed={current === m.id}
              className={cn(
                "flex-1 rounded-lg border px-2 py-1.5 text-[11px] font-medium transition-colors",
                current === m.id
                  ? "border-primary bg-primary/10 text-primary"
                  : "border-border hover:bg-muted"
              )}
            >
              {m.label}
            </button>
          );
        })}
      </div>
      <button
        onClick={() => setLassoStrict(!lassoStrict)}
        role="switch"
        aria-checked={!lassoStrict}
        className="mt-2 flex w-full items-start gap-2 rounded-lg px-1 py-1.5 text-left hover:bg-mute
d"
      >
        <span className="min-w-0 flex-1">
          <span className="block text-[11px] font-medium leading-tight">

```

```

        Include partially selected
      </span>
      <span className="block text-[11px] leading-snug text-muted-foreground">
        {lassoStrict
          ? "The lasso takes only what it fully encloses."
          : "The lasso takes anything it touches."}
      </span>
    </span>
    <span
      className={cn(
        "mt-0.5 h-4 w-7 shrink-0 rounded-full p-0.5 transition-colors",
        lassoStrict ? "bg-muted-foreground/30" : "bg-primary"
      )}
    >
      <span
        className={cn(
          "block h-3 w-3 rounded-full bg-white transition-transform",
          lassoStrict ? "translate-x-0" : "translate-x-3"
        )}
      />
    </span>
  </button>
</div>
<div className="px-2 pb-1 pt-1">
  <div className="mb-1 text-[11px] text-muted-foreground">Mouse & trackpad</div>
  <div className="flex gap-1">
    {[
      { id: "auto", label: "Auto", hint: "Detect per event" },
      { id: "trackpad", label: "Trackpad", hint: "Scroll pans, pinch zooms" },
      { id: "mouse", label: "Mouse", hint: "Wheel zooms" },
    ] as const}.map((w) => (
      <button
        key={w.id}
        onClick={() => setTouchSettings({ wheelMode: w.id })}
        title={w.hint}
        className={cn(
          "flex-1 rounded-lg border px-2 py-1.5 text-[11px] font-medium transition-colors",
          (touchSettings.wheelMode ?? "auto") === w.id
            ? "border-primary bg-primary/10 text-primary"
            : "border-border hover:bg-muted"
        )}
      >
        {w.label}
      </button>
    ))}
  </div>
  <p className="mt-1.5 text-[11px] leading-snug text-muted-foreground">
    Ctrl/⌘ + wheel always zooms. Shift + wheel scrolls sideways.
  </p>
</div>

<ToggleRow
  icon={<Pointer size={15} />}
  label="Two-finger tap to undo"
  hint="Three-finger tap redoes"
  checked={touchSettings.twoFingerUndo}
  onChange={() => setTouchSettings({ twoFingerUndo: !touchSettings.twoFingerUndo })}
/>
<ToggleRow
  icon={<Pointer size={15} />}
  label="Stylus button erases"
  hint="Hold the pen's side button (or flip to its eraser end) to erase, release for your pen ba
ck"
  checked={touchSettings.stylusButtonErases !== false}
  onChange={() =>
    setTouchSettings({ stylusButtonErases: !(touchSettings.stylusButtonErases !== false) })
  }
/>
<ToggleRow
  icon={<Pointer size={15} />}
  label="Show pen diagnostics"
  hint="Prints what your stylus actually sends (buttons, pressure, tilt). Turn on if the side bu
tton does not erase, then screenshot the panel."
  checked={touchSettings.penDiagnostics === true}
  onChange={() => setTouchSettings({ penDiagnostics: !(touchSettings.penDiagnostics === true) })
  }
/>
<ToggleRow

```

```

        icon={<Pointer size={15} />}
        label="Fingers never draw"
        hint="With a pen tool selected, fingers only navigate. Contact while the stylus is in use is a
        lways ignored, and holding a finger on an object selects it."
        checked={touchSettings.palmRejection}
        onChange={() => setTouchSettings({ palmRejection: !touchSettings.palmRejection })}
    />

    /* S Pen Inking Engine & Stabilization */
    <div className="mt-3 px-2">
        <div className="mb-1 flex items-center justify-between">
            <span className="text-xs font-medium">Stroke stabilization & smoothing</span>
            <span className="rounded-md bg-muted px-1.5 py-0.5 text-[11px] font-semibold tabular-nums">
                {touchSettings.penSmoothing ?? 50}%
            </span>
        </div>
        <div className="mb-2 flex gap-1">
            {[
                { label: "Off", val: 0 },
                { label: "Light", val: 25 },
                { label: "Balanced", val: 50 },
                { label: "High", val: 75 },
                { label: "Ultra", val: 100 },
            ].map((preset) => (
                <button
                    key={preset.val}
                    type="button"
                    onClick={() => setTouchSettings({ penSmoothing: preset.val })}
                    className={cn(
                        "flex-1 rounded-md border py-1 text-[10px] font-medium transition-colors",
                        (touchSettings.penSmoothing ?? 50) === preset.val
                            ? "border-primary bg-primary/10 text-primary"
                            : "border-border text-muted-foreground hover:bg-muted"
                    )}
                >
                    {preset.label}
                </button>
            ))}
        </div>
        <StepperRow
            value={touchSettings.penSmoothing ?? 50}
            min={0}
            max={100}
            onChange={(v) => setTouchSettings({ penSmoothing: v })}
        >
            <input
                type="range"
                min={0}
                max={100}
                value={touchSettings.penSmoothing ?? 50}
                aria-label="Stroke stabilization"
                onChange={(e) => setTouchSettings({ penSmoothing: Number(e.target.value) })}
                className="h-1.5 w-full cursor-pointer appearance-none rounded-lg bg-muted accent-primary"
            />
        </StepperRow>
        <p className="mt-1 text-[11px] leading-snug text-muted-foreground">
            Filters electromagnetic digitizer jitter. 50% gives natural responsive handwriting; High/Ultra
            stabilizes tremors.
        </p>
    </div>

    /* S Pen Pressure Calibration */
    <div className="mt-3 px-2">
        <div className="mb-1 flex items-center justify-between">
            <span className="text-xs font-medium">S Pen pressure response curve</span>
            <span className="rounded-md bg-muted px-1.5 py-0.5 text-[10px] font-semibold capitalize text-
            -muted-foreground">
                {touchSettings.penPressureCurve ?? "standard"}
            </span>
        </div>
        <div className="mb-2 flex gap-1">
            {[
                { id: "off", label: "Off", hint: "Uniform" },
                { id: "soft", label: "Soft", hint: "Light touch" },
                { id: "standard", label: "Standard", hint: "Natural" },
                { id: "firm", label: "Firm", hint: "Deliberate" },
            ].map((c) => (
                <button

```

```

        key={c.id}
        type="button"
        onClick={() => setTouchSettings({ penPressureCurve: c.id as PressureCurve })}
        className={cn(
          "flex-1 rounded-md border py-1 text-center transition-colors",
          (touchSettings.penPressureCurve ?? "standard") === c.id
            ? "border-primary bg-primary/10 text-primary"
            : "border-border text-muted-foreground hover:bg-muted"
        )}
      >
        <span className="block text-[11px] font-medium leading-tight">{c.label}</span>
        <span className="block text-[9px] text-muted-foreground leading-tight">{c.hint}</span>
      </button>
    )}
  </div>

  {(touchSettings.penPressureCurve ?? "standard") !== "off" && (
    <div>
      <div className="mb-1 flex items-center justify-between">
        <span className="text-[11px] text-muted-foreground">Pressure dynamic range</span>
        <span className="rounded-md bg-muted px-1.5 py-0.5 text-[10px] font-semibold tabular-num
s">
          {touchSettings.penPressureSensitivity ?? 50}%
        </span>
      </div>
      <StepperRow
        value={touchSettings.penPressureSensitivity ?? 50}
        min={0}
        max={100}
        onChange={(v) => setTouchSettings({ penPressureSensitivity: v })}
      >
        <input
          type="range"
          min={0}
          max={100}
          value={touchSettings.penPressureSensitivity ?? 50}
          aria-label="Pressure sensitivity"
          onChange={(e) => setTouchSettings({ penPressureSensitivity: Number(e.target.value) })}
          className="h-1.5 w-full cursor-pointer appearance-none rounded-lg bg-muted accent-prim
ary"
        />
      </StepperRow>
    </div>
  )}
</div>

{/* Hardware Toggles */}
<div className="mt-2">
  <ToggleRow
    icon={<Sparkles size={15} />}
    label="Lift-off anti-hook filter"
    hint="Filters out the trailing hook flick caused when lifting the S Pen nib off the glass"
    checked={touchSettings.penAntiHook !== false}
    onChange={() => setTouchSettings({ penAntiHook: !(touchSettings.penAntiHook !== false) })}
  />
  <ToggleRow
    icon={<Wand2 size={15} />}
    label="Draw-and-hold auto-straighten"
    hint="Pausing for 550ms at the end of a stroke snaps a freehand line perfectly straight"
    checked={touchSettings.penAutoStraighten !== false}
    onChange={() =>
      setTouchSettings({ penAutoStraighten: !(touchSettings.penAutoStraighten !== false) })
    }
  />
  <ToggleRow
    icon={<Pointer size={15} />}
    label="S Pen hover indicator"
    hint="Shows live cursor dot and nib size preview when hovering S Pen above the screen"
    checked={touchSettings.penHoverIndicator !== false}
    onChange={() =>
      setTouchSettings({ penHoverIndicator: !(touchSettings.penHoverIndicator !== false) })
    }
  />
</div>
</section>

{/* ---- Help ---- */}
<section>

```

```

<SectionTitle>Help</SectionTitle>
<button
  onClick={() => {
    onClose();
    window.dispatchEvent(new CustomEvent("wb:open-shortcuts"));
  }}
  className="flex w-full items-center gap-3 rounded-lg px-2 py-2 text-left text-xs font-medium transition-colors hover:bg-muted"
>
  <Keyboard size={15} className="text-muted-foreground" />
  Keyboard shortcuts & gestures
</button>
<button
  onClick={() => {
    onClose();
    window.dispatchEvent(new CustomEvent("wb:open-tutorial"));
  }}
  className="flex w-full items-center gap-3 rounded-lg px-2 py-2 text-left text-xs font-medium transition-colors hover:bg-muted"
>
  <HelpCircle size={15} className="text-muted-foreground" />
  Show the tutorial again
</button>
</section>

{ /* ---- Which build is this ----
  Installing an APK and seeing no change is indistinguishable
  from a fix that did not work, unless the app can say which
  build it is. The build number is Android's versionCode and
  rises with every APK; a browser or desktop build has none. */
<p className="pt-1 text-center text-[11px] text-muted-foreground">
  ProNotes {process.env.NEXT_PUBLIC_APP_VERSION || "?"}
  {process.env.NEXT_PUBLIC_BUILD_CODE ? ` · build ${process.env.NEXT_PUBLIC_BUILD_CODE}` : ""}
</p>
</div>
</div>
</div>
</div>
);
}

```

```

=====
// === END FILE: src/components/whiteboard/SettingsPanel.tsx ===
=====

```

```

=====
// === START FILE: src/components/whiteboard/ShapeAssist.tsx ===
// File 77 of 194 | Lines: 90 | Size: 2787 bytes
=====

```

```
"use client";
```

```

/**
 * Shape recognition: its tool-bar icon and the note that appears after a
 * stroke was turned into a shape. The recognising itself is pure and lives in
 * lib/whiteboard/shapeRecognition.ts.
 */

```

```

import React, { useEffect } from "react";
import { Check, Undo2 } from "lucide-react";

```

```

/**
 * A pen beside a triangle, a circle and a square. Drawn on the 24px grid the
 * other tool icons use, same 1.8 stroke, nothing overlapping: the pen runs
 * corner to corner on the left, the three shapes sit clear of it on the right.
 */

```

```

export function ShapeAssistIcon({ size = 18, className }: { size?: number; className?: string }) {
  return (
    <svg
      width={size}
      height={size}
      viewBox="0 0 24 24"
      fill="none"
      stroke="currentColor"
      strokeWidth={1.8}
      strokeLinecap="round"
      strokeLinejoin="round"
    >

```

```

        className={className}
        aria-hidden
      >
        { /* pen: nib at bottom left, rounded end at top right */ }
        <path d="M3.5 17.5 4.17 13.99 11.93 3.83A1.6 1.6 0 0 1 14.47 5.77L6.71 15.93Z" />
        <path d="M4.17 13.99 6.71 15.93" />
        { /* triangle */ }
        <path d="M17.6 9.4 20.3 13.8H14.9Z" />
        { /* circle */ }
        <circle cx="11.3" cy="18.7" r="2.3" />
        { /* square */ }
        <rect x="15.3" y="16.4" width="4.6" height="4.6" rx="0.7" />
      </svg>
    );
  }

export interface ShapeNote {
  id: string;
  label: string;
  /** Screen position (px, within the canvas) to show it at. */
  x: number;
  y: number;
  key: number;
}

/**
 * "Circle ✓ Keep as drawn" - shows what the stroke became, with a one-tap
 * way back to the stroke as you drew it. Goes by itself after a few seconds.
 */
export function ShapeNoteChip({
  note,
  onKeepDrawn,
  onDone,
}): {
  note: ShapeNote;
  onKeepDrawn: () => void;
  onDone: () => void;
}) {
  useEffect(() => {
    const t = setTimeout(onDone, 3200);
    return () => clearTimeout(t);
  }, [note.key, onDone]);

  return (
    <div
      className="pointer-events-auto absolute z-30 flex -translate-x-1/2 items-center gap-0.5 rounded-full border border-white/10 bg-zinc-900/92 py-0.5 pl-2.5 pr-0.5 text-[12px] font-medium text-white shadow-xl backdrop-blur-md animate-in fade-in zoom-in-95 duration-150"
      style={{ left: note.x, top: note.y }}
      onPointerDown={(e) => e.stopPropagation()}
    >
      <Check size={13} className="text-emerald-400" />
      <span className="px-1">{note.label}</span>
      <button
        type="button"
        onClick={onKeepDrawn}
        className="flex h-7 items-center gap-1 rounded-full px-2.5 text-white/80 transition-colors hover:bg-wh
          ite/10 hover:text-white"
        title="Put back the stroke exactly as you drew it"
      >
        <Undo2 size={13} />
        Keep as drawn
      </button>
    </div>
  );
}

```

```

=====
// === END FILE: src/components/whiteboard/ShapeAssist.tsx ===
=====

```

```

=====
// === START FILE: src/components/whiteboard/Sidebar.tsx ===
// File 78 of 194 | Lines: 2049 | Size: 81914 bytes
=====

```

```

"use client";

```

```

/**
 * Sidebar (Folder & Board Tree)
 * =====
 * Microsoft Whiteboard-style left sidebar showing:
 * - A tree of folders and boards
 * - Create / rename / delete / duplicate boards
 * - Create / rename / delete folders
 * - Drag-and-drop to move boards and folders: hold a row, then drop it on a
 *   folder (or on empty tree space for the top level)
 * - Active board highlighting
 *
 * Tree rendering:
 * - Folders can be nested (parentId chains)
 * - Each folder can be expanded/collapsed
 * - Boards are leaf nodes inside folders (or at root)
 *
 * Touch rules (pointer events, never HTML5 drag-and-drop):
 * - A tap still opens/toggles; only a 300ms hold picks a row up, and moving
 *   first cancels the pickup so the tree keeps scrolling from the row.
 * - Dropping a folder into itself or its own subtree is rejected (the store
 *   guards it too – a parentId cycle would hang every tree walk).
 * - Holding over a collapsed folder springs it open so its children accept drops.
 *
 * Future debugging notes:
 * - All mutations go through the Zustand store; sidebar is purely a view
 * - Empty folders are allowed and shown with a muted hint
 */

```

```

import React, { useRef, useState } from "react";
import {
  Folder as FolderIcon,
  FolderInput,
  FolderOpen,
  FileText,
  Plus,
  BookOpen,
  NotebookText,
  Presentation,
  Files,
  FileUp,
  ImagePlus,
  Columns2,
  FolderPlus,
  MoreHorizontal,
  Trash2,
  Pencil,
  Copy,
  ChevronRight,
  ChevronDown,
  PanelLeftClose,
  Search,
  Star,
  Link2,
  Download,
  ArrowUpDown,
  CheckSquare,
  Square,
  X,
} from "lucide-react";
import { useWhiteboard } from "@/store/whiteboard";
import { getPages, isNotebook } from "@/lib/whiteboard/notebook";
import { FOLDER_COLORS, type Board, type Folder } from "@/lib/whiteboard/types";
import { sortItems } from "@/lib/whiteboard/sort";
import {
  buildExportBlob,
  zipExportFiles,
  type ExportFormat,
} from "@/lib/whiteboard/export";
import { saveBlob } from "@/lib/whiteboard/share";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { ScrollArea } from "@/components/ui/scroll-area";
import {
  Dialog,
  DialogContent,
  DialogFooter,
  DialogHeader,

```

```

    DialogTitle,
  } from "@components/ui/dialog";
import {
  DropdownMenu,
  DropdownMenuContent,
  DropdownMenuItem,
  DropdownMenuSeparator,
  DropdownMenuTrigger,
} from "@components/ui/dropdown-menu";
import {
  ContextMenu,
  ContextMenuContent,
  ContextMenuItem,
  ContextMenuSeparator,
  ContextMenuTrigger,
} from "@components/ui/context-menu";
import { cn } from "@lib/utils";
import { MoveToDialog, BatchMoveDialog } from "../MoveToDialog";
import { DuplicateNotebookModal } from "../DuplicateNotebookModal";
import { importPicturesAsNotebook } from "../FileDropZone";
import { DOCUMENT_ACCEPT } from "@lib/whiteboard/officeImport";
import { trashWithUndo } from "../trashToast";

// -----
// Tree drag & drop: hold a row, drop it on a folder (or empty space = root)
// -----

/** What a drag carries: a board/notebook, or a whole folder. */
type SidebarDragPayload = { kind: "board" | "folder"; id: string };

/** Drop-target ids: a folder id, or "root" for the top level. */
const ROOT_DROP_ID = "root";

/** Hold still this long to pick a row up (a tap still opens/toggles). */
const TREE_HOLD_MS = 300;
/** Two taps within this open the full row menu (pen and finger alike). */
const DOUBLE_TAP_MS = 350;
/** Moving further first means a scroll, which cancels the pickup. */
const TREE_HOLD_PX = 24;
/** Dragging within this far of the tree's top/bottom edge scrolls it. */
const TREE_SCROLL_EDGE_PX = 64;
const TREE_SCROLL_STEP_PX = 24;
/** Hovering a collapsed folder this long springs it open. */
const SPRING_OPEN_MS = 600;

interface SidebarDrag {
  drag: SidebarDragPayload | null;
  dropTarget: string | null;
  pickup: (p: SidebarDragPayload) => void;
}

const SidebarDragContext = React.createContext<SidebarDrag>({
  drag: null,
  dropTarget: null,
  pickup: () => undefined,
});

/** Selection ids for folders live next to board ids with a prefix. */
export const FOLDER_SEL_PREFIX = "folder:";

export function splitSelection(ids: string[]): { boardIds: string[]; folderIds: string[] } {
  const boardIds: string[] = [];
  const folderIds: string[] = [];
  for (const id of ids) {
    if (id.startsWith(FOLDER_SEL_PREFIX)) folderIds.push(id.slice(FOLDER_SEL_PREFIX.length));
    else boardIds.push(id);
  }
  return { boardIds, folderIds };
}

/** True for notebooks imported from a PDF (drawn over document pages). */
export function isPdfNotebook(board: Board): boolean {
  if (!isNotebook(board)) return false;
  if ((board.pdfDocIds?.length ?? 0) > 0) return true;
  return getPages(board).some((p) => !!p.pdf);
}

/**

```

```

* One icon per document kind, everywhere: canvas boards, notebooks, PDFs and
* folders each keep their own glyph in rows, menus and dialogs. (Rows used to
* show FileText for everything and "New Board" a bare "+".)
*/
export function BoardKindIcon({ board, size = 14 }: { board: Board; size?: number }) {
  if (isPdfNotebook(board)) {
    return <FileText size={size} className="shrink-0 text-red-400" />;
  }
  if (isNotebook(board)) {
    return <NotebookText size={size} className="shrink-0" style={{ color: board.color || "#3b82f6" }} />;
  }
  // A canvas board is a whiteboard, so it wears one – not a pencil, which
  // read as "edit" (and the old "New Board" row showed a bare "+").
  return <Presentation size={size} className="shrink-0" style={{ color: board.color || "#3b82f6" }} />;
}

/** True when `id` sits inside `ancestorId`'s subtree (invalid drop styling). */
export function isDescendantFolder(folders: Folder[], ancestorId: string, id: string): boolean {
  let cursor = folders.find((f) => f.id === id)?.parentId ?? null;
  const seen = new Set<string>();
  while (cursor) {
    if (cursor === ancestorId) return true;
    if (seen.has(cursor)) break;
    seen.add(cursor);
    cursor = folders.find((f) => f.id === cursor)?.parentId ?? null;
  }
  return false;
}

export { sortItems } from "@lib/whiteboard/sort";

/**
* Press-and-hold a tree row: holding still selects (entering multi-select
* with the row chosen when coming from outside); moving after the hold
* picks the row up for drag-and-drop instead. Moving FIRST cancels (the
* tree still scrolls from the row). Returns props for the row root plus a
* click suppressor for the tap that follows either gesture.
*/
function useHoldRow({ onHold, onDragStart }: { onHold: () => void; onDragStart: () => void }) {
  const timer = React.useRef<ReturnType<typeof setTimeout> | null>(null);
  const start = React.useRef<{ x: number; y: number } | null>(null);
  const suppressClick = React.useRef(false);
  const held = React.useRef(false);
  const dragged = React.useRef(false);
  const cancel = React.useCallback(() => {
    if (timer.current) clearTimeout(timer.current);
    timer.current = null;
    start.current = null;
  }, []);
  React.useEffect(() => cancel, [cancel]);
  return {
    suppressClick,
    holdHandlers: {
      onPointerDown: (e: React.PointerEvent) => {
        if (!e.isPrimary) return;
        if ((e.target as Element).closest?("[data-hold-ignore]") return;
        suppressClick.current = false;
        held.current = false;
        dragged.current = false;
        start.current = { x: e.clientX, y: e.clientY };
        timer.current = setTimeout(() => {
          timer.current = null;
          held.current = true;
          suppressClick.current = true;
          try {
            navigator.vibrate?.(15);
          } catch {
            /* haptic is a courtesy, never required */
          }
        }, onHold());
      },
      onHold: () => {
        onHold();
      },
      onDragStart: () => {
        onDragStart();
      },
      onPointerMove: (e: React.PointerEvent) => {
        const s = start.current;
        if (!s) return;
        if (Math.hypot(e.clientX - s.x, e.clientY - s.y) <= TREE_HOLD_PX) return;
        if (timer.current) {
          // Moved first: a scroll, never a pickup.
        }
      },
    },
  };
}

```

```

        cancel();
        return;
    }
    if (held.current && !dragged.current) {
        dragged.current = true;
        onDragStart();
    }
},
onPointerUp: cancel,
onPointerCancel: cancel,
},
});
}

/**
 * What the sidebar shows about each board, as one string. Strokes do not
 * change it, so writing never re-renders the sidebar; renaming, moving,
 * starring or adding a board does.
 *
 * For "Recently updated" sorting, we add the sorted order of notebook IDs to
 * this key, NOT their raw updatedAt times. Otherwise the sidebar redraws after
 * every pen stroke and the ~240ms freeze after each pen lift comes back.
 */
export function boardListKey(boards: Board[]): string {
    const sortedIds = [...boards]
        .sort((a, b) => (b.updatedAt || b.createdAt || 0) - (a.updatedAt || a.createdAt || 0))
        .map((b) => b.id)
        .join(",");

    const entries = boards
        .map((b) =>
            [b.id, b.name, b.folderId ?? "", b.starred ? 1 : 0, b.color ?? "", b.kind ?? "", (b.tags ?? []).join(",")].join(
                "\u0001"
            )
        )
        .join("\u0002");

    return `${sortedIds}\u0003${entries}`;
}

/**
 * A notebook's page count, with a pages glyph so it reads as pages. It used
 * to be the number of strokes and shapes inside ("1929"), which told the
 * reader nothing. Its own subscription, and a page count only changes when a
 * page is added or removed, so writing never re-renders it.
 */
function PageCount({ id }: { id: string }) {
    const n = useWhiteboard((s) => {
        const b = s.boards.find((x) => x.id === id);
        return b && isNotebook(b) ? getPages(b).length : 0;
    });
    if (!n) return null;
    return (
        <span
            className="flex shrink-0 items-center gap-0.5 text-[10px] tabular-nums text-muted-foreground"
            title={` ${n} page${n === 1 ? "" : "s"} `}
        >
            <Files size={10} className="opacity-70" />
            {n}
        </span>
    );
}

/** One row of the New menu: a coloured icon, a name and what it is for. */
function NewMenuItem({
    icon,
    title,
    hint,
    onClick,
    disabled,
}: {
    icon: React.ReactNode;
    title: string;
    hint: string;
    onClick: () => void;
    disabled?: boolean;
}) {
```

```

    return (
      <DropdownMenuItem onClick={onClick} disabled={disabled} className="gap-3 rounded-lg px-2 py-2">
        <span className="flex h-8 w-8 shrink-0 items-center justify-center rounded-lg bg-muted">{icon}</span>
        <span className="flex min-w-0 flex-col">
          <span className="text-[13px] font-medium leading-tight">{title}</span>
          <span className="truncate text-[11px] leading-tight text-muted-foreground">{hint}</span>
        </span>
      </DropdownMenuItem>
    );
  }

  /** A small section heading in the tree: Starred / Folders / Notes. */
  function SectionLabel({ icon, children }: { icon?: React.ReactNode; children: React.ReactNode }) {
    return (
      <div className="flex items-center gap-1.5 px-2 pb-1 pt-3 text-[10px] font-semibold uppercase tracking-wide
        r text-muted-foreground first:pt-1">
        {icon}
        {children}
      </div>
    );
  }
}

/**
 * Name-before-create dialog for the New menu. Mounts only while open, autofocuses,
 * Enter confirms and Escape cancels. `contextLabel` says where the item lands
 * ("Top level" or a folder name).
 */
export function NameDialog({
  open,
  title,
  defaultName,
  confirmLabel,
  contextLabel,
  onConfirm,
  onClose,
}: {
  open: boolean;
  title: string;
  defaultName: string;
  confirmLabel: string;
  contextLabel?: string;
  onConfirm: (name: string) => void;
  onClose: () => void;
}) {
  const [value, setValue] = React.useState(defaultName);
  React.useEffect(() => {
    if (open) setValue(defaultName);
  }, [open, defaultName]);
  if (!open) return null;
  const confirm = () => {
    const trimmed = value.trim();
    if (!trimmed) return;
    onConfirm(trimmed);
    onClose();
  };
  return (
    <Dialog open={open} onOpenChange={{o} => !o && onClose()}>
      <DialogContent className="max-w-xs p-0 gap-0 overflow-hidden">
        <DialogHeader className="px-4 pb-2 pt-4 text-left">
          <DialogTitle className="text-base">{title}</DialogTitle>
          {contextLabel && <p className="text-xs text-muted-foreground">in {contextLabel}</p>}
        </DialogHeader>
        <div className="px-4 pb-2">
          <Input
            value={value}
            onChange={(e) => setValue(e.target.value)}
            onKeyDown={(e) => {
              if (e.key === "Enter") confirm();
              if (e.key === "Escape") onClose();
            }}
            autoFocus
            onFocus={(e) => e.target.select()}
            aria-label={title}
          />
        </div>
        <DialogFooter className="flex-row items-center justify-end gap-2 border-t border-border px-4 py-3">
          <Button variant="ghost" size="sm" onClick={onClose}>
            Cancel
          </Button>
        </DialogFooter>
      </DialogContent>
    </Dialog>
  );
}

```

```

        </Button>
        <Button size="sm" onClick={confirm} disabled={!value.trim()}>
          {confirmLabel}
        </Button>
      </DialogFooter>
    </DialogContent>
  </Dialog>
);
}

/**
 * The selection actions, docked at the bottom of the sidebar while selecting.
 * Four equal buttons - icon over label, so all four fit the narrow sidebar
 * with finger-sized targets - and everything less common behind "More". The
 * count lives ONCE, in the header above; this bar used to repeat it.
 * Disabled (not hidden) while nothing is picked, so it never jumps about.
 */
function SelectionBar({
  boardIds,
  folderIds,
  boards,
  folders,
  busy,
  onDelete,
  onMove,
  onExport,
  onDuplicate,
  onStar,
  onRename,
  onCopyLink,
}): {
  boardIds: string[];
  folderIds: string[];
  boards: Board[];
  folders: Folder[];
  busy: string | null;
  onDelete: () => void;
  onMove: () => void;
  onExport: (format: ExportFormat) => void;
  onDuplicate: () => void;
  onStar: () => void;
  onRename: (kind: "board" | "folder", id: string) => void;
  onCopyLink: (id: string) => void;
}) {
  const total = boardIds.length + folderIds.length;
  const none = total === 0;
  const singleBoardId = boardIds.length === 1 && folderIds.length === 0 ? boardIds[0] : null;
  const singleFolderId = folderIds.length === 1 && boardIds.length === 0 ? folderIds[0] : null;
  const singleName =
    singleBoardId != null
      ? boards.find((b) => b.id === singleBoardId)?.name
      : singleFolderId != null
        ? folders.find((f) => f.id === singleFolderId)?.name
        : null;
  const allStarred = boardIds.length > 0 && boardIds.every((id) => boards.find((b) => b.id === id)?.starred);
  const cell =
    "flex h-12 min-w-0 flex-1 flex-col items-center justify-center gap-0.5 rounded-lg text-[10px] font-medium
    transition-colors disabled:pointer-events-none disabled:opacity-35";
  return (
    <div className="border-t border-sidebar-border bg-sidebar px-1.5 py-1.5 animate-in slide-in-from-bottom-2">
      <div className="flex items-stretch gap-1">
        <button
          type="button"
          className={cn(cell, "text-destructive hover:bg-destructive/10")}
          onClick={onDelete}
          disabled={none}
          title="Delete – you can Undo right after"
        >
          <Trash2 size={17} />
          Delete
        </button>
        <button
          type="button"
          className={cn(cell, "text-sidebar-foreground hover:bg-sidebar-accent")}
          onClick={onMove}
          disabled={none}
          title="Move to a folder"

```

```

>
  <FolderInput size={17} />
  Move
</button>
<DropdownMenu>
  <DropdownMenuTrigger asChild>
    <button
      type="button"
      className={cn(cell, "text-sidebar-foreground hover:bg-sidebar-accent")}
      disabled={boardIds.length === 0 || busy !== null}
      title={boardIds.length === 0 ? "Pick a notebook or board to export" : "Export as..."}
    >
      <Download size={17} />
      <span className="max-w-full truncate">{busy ?? "Export"}</span>
    </button>
  </DropdownMenuTrigger>
  <DropdownMenuContent side="top" align="center" className="w-44">
    <div className="px-2 py-1 text-[10px] font-semibold uppercase tracking-wider text-muted-foreground">
      >
        {boardIds.length > 1 ? `${boardIds.length} files, as one .zip` : "Export as"}
      </div>
      {[["pdf", "png", "svg"] as ExportFormat[]].map((f) => (
        <DropdownMenuItem key={f} onClick={() => onExport(f)}>
          {f.toUpperCase()}
        </DropdownMenuItem>
      ))}
    </DropdownMenuContent>
  </DropdownMenu>
<DropdownMenu>
  <DropdownMenuTrigger asChild>
    <button
      type="button"
      className={cn(cell, "text-sidebar-foreground hover:bg-sidebar-accent")}
      disabled={none}
      aria-label="More actions"
    >
      <MoreHorizontal size={17} />
      More
    </button>
  </DropdownMenuTrigger>
  <DropdownMenuContent side="top" align="end" className="w-48">
    <DropdownMenuItem onClick={onDuplicate} disabled={boardIds.length === 0}>
      <Copy size={13} className="mr-2" /> Duplicate
    </DropdownMenuItem>
    <DropdownMenuItem onClick={onStar} disabled={boardIds.length === 0}>
      <Star size={13} className="mr-2" /> {allStarred ? "Unstar" : "Star"}
    </DropdownMenuItem>
    <DropdownMenuItem
      onClick={() => {
        if (singleBoardId) onRename("board", singleBoardId);
        else if (singleFolderId) onRename("folder", singleFolderId);
      }}
      disabled={singleName == null}
    >
      <Pencil size={13} className="mr-2" /> Rename
      {singleName == null && <span className="ml-auto text-[10px] text-muted-foreground">one item</span>}
    </DropdownMenuItem>
    <DropdownMenuItem onClick={() => singleBoardId && onCopyLink(singleBoardId)} disabled={singleBoardId == null}>
      <Link2 size={13} className="mr-2" /> Copy [{link}]
      {singleBoardId == null && <span className="ml-auto text-[10px] text-muted-foreground">one item</span>}
    </DropdownMenuItem>
  </DropdownMenuContent>
</DropdownMenu>
</div>
</div>
);
}

export function Sidebar() {
  const folders = useWhiteboard((s) => s.folders);
  // Every pen stroke replaces `boards`; the sidebar only needs the list. It
  // re-rendered - every row and folder, with their menus - after each stroke
  // (~240 ms on a real 20-notebook workspace, the freeze after each pen lift).
  const boardsKey = useWhiteboard((s) => boardListKey(s.boards));
  // eslint-disable-next-line react-hooks/exhaustive-deps

```

```

const boards = React.useMemo(() => useWhiteboard.getState().boards, [boardsKey]);
const activeBoardId = useWhiteboard((s) => s.activeBoardId);
const setActiveBoard = useWhiteboard((s) => s.setActiveBoard);
const pdfInputRef = useRef<HTMLInputElement>(null);
const [pdfBusy, setPdfBusy] = useState<string | null>(null);
const [pdfError, setPdfError] = useState<string | null>(null);
const toggleSidebar = useWhiteboard((s) => s.toggleSidebar);
const [searchQuery, setSearchQuery] = React.useState("");

const sidebarSortBy = useWhiteboard((s) => s.sidebarSortBy);
const sidebarSortOrder = useWhiteboard((s) => s.sidebarSortOrder);
const setSidebarSort = useWhiteboard((s) => s.setSidebarSort);
const [multiSelectMode, setMultiSelectMode] = React.useState(false);
// Board ids plus `folder:<id>` entries — one list, split at action time.
const [selectedBatchIds, setSelectedBatchIds] = React.useState<string[]>([]);
const [batchMoveOpen, setBatchMoveOpen] = React.useState(false);
/** Folder the selection is scoped to (from its "... menu), or null for all. */
const [scopeFolderId, setScopeFolderId] = React.useState<string | null>(null);
/** Single row awaiting inline rename ({ kind: "board" | "folder", id }). */
const [renameRequest, setRenameRequest] = React.useState<{ kind: "board" | "folder"; id: string } | null>(null);
const [batchBusy, setBatchBusy] = useState<string | null>(null);
const [batchError, setBatchError] = useState<string | null>(null);
const [nameDialog, setNameDialog] = React.useState<
  null | { kind: "board" | "notebook" | "folder"; folderId: string | null }
>(null);
/** Shared file inputs: folder menus set the target, then open the picker. */
const imageInputRef = useRef<HTMLInputElement>(null);
const importTargetFolder = React.useRef<string | null>(null);
const [importBusy, setImportBusy] = useState<string | null>(null);

// Fresh reads for the hold gesture, which fires 300 ms after the press.
const multiSelectModeRef = React.useRef(multiSelectMode);
multiSelectModeRef.current = multiSelectMode;

const toggleBatchSelect = React.useCallback((boardId: string) => {
  setSelectedBatchIds((prev) =>
    prev.includes(boardId) ? prev.filter((id) => id !== boardId) : [...prev, boardId]
  );
}, []);

const toggleFolderSelect = React.useCallback((folderId: string) => {
  const key = FOLDER_SEL_PREFIX + folderId;
  setSelectedBatchIds((prev) =>
    prev.includes(key) ? prev.filter((id) => id !== key) : [...prev, key]
  );
}, []);

/**
 * Long-press entry: from outside, enters selection mode scoped nowhere with
 * just this row; inside, it ensures the row is selected (drag arms on move).
 */
const holdRowSelect = React.useCallback((kind: "board" | "folder", id: string) => {
  const key = kind === "folder" ? FOLDER_SEL_PREFIX + id : id;
  if (!multiSelectModeRef.current) {
    multiSelectModeRef.current = true;
    setScopeFolderId(null);
    setMultiSelectMode(true);
    setSelectedBatchIds([key]);
  } else {
    setSelectedBatchIds((prev) => (prev.includes(key) ? prev : [...prev, key]));
  }
}, []);

const exitSelectMode = React.useCallback(() => {
  setMultiSelectMode(false);
  multiSelectModeRef.current = false;
  setSelectedBatchIds([]);
  setScopeFolderId(null);
}, []);

/** Batch export: one file straight out, several packed into one .zip. */
const runBatchExport = React.useCallback(
  async (format: ExportFormat) => {
    const st = useWhiteboard.getState();
    const { boardIds } = splitSelection(selectedBatchIds);
    const targets = boardIds
      .map((id) => st.boards.find((b) => b.id === id))

```

```

    .filter((b) => b !== undefined);
    if (targets.length === 0) return;
    setBatchBusy("Exporting...");
    setBatchError(null);
    try {
      if (targets.length === 1) {
        const { blob, filename } = await buildExportBlob(targets[0], format, {}, st.customLinkTypes);
        await saveBlob(blob, filename);
      } else {
        const files: Array<{ filename: string; data: Uint8Array }> = [];
        for (const b of targets) {
          const { blob, filename } = await buildExportBlob(b, format, {}, st.customLinkTypes);
          files.push({ filename, data: new Uint8Array(await blob.arrayBuffer()) });
        }
        // Same-named boards would collide inside the zip; suffix extras.
        const seen = new Map<string, number>();
        for (const f of files) {
          const n = seen.get(f.filename) ?? 0;
          seen.set(f.filename, n + 1);
          if (n > 0) {
            const dot = f.filename.lastIndexOf(".");
            f.filename = `${f.filename.slice(0, dot)} (${n + 1})${f.filename.slice(dot)}`;
          }
        }
        // Copy into ArrayBuffer-backed bytes: fflate's view type is wider
        // than what Blob accepts.
        const zipped = new Uint8Array(zipExportFiles(files));
        const d = new Date();
        const stamp = `${d.getFullYear()}${String(d.getMonth() + 1).padStart(2, "0")}${String(d.getDate()).padStart(2, "0")}`;
        await saveBlob(new Blob([zipped], { type: "application/zip" }), `pronotes-export-${stamp}.zip`);
      }
      setSelectedBatchIds([]);
    } catch (err) {
      setBatchError((err as Error).message || "Export failed.");
    } finally {
      setBatchBusy(null);
    }
  },
  [selectedBatchIds]
);

// --- tree drag & drop -----
// State lives here; rows only pick up (via the context) while the window
// listeners below track the pointer anywhere, even off the tree.
const [drag, setDrag] = React.useState<SidebarDragPayload | null>(null);
const [dropTarget, setDropTarget] = React.useState<string | null>(null);
const dragRef = React.useRef(drag);
dragRef.current = drag;
const dropRef = React.useRef(dropTarget);
dropRef.current = dropTarget;
// Synchronous drag flag for the contextmenu killer below. State (`drag`)
// arrives a render late; Radix's own long-press menu must die the moment a
// pickup happens, not after React catches up.
const dragActiveRef = React.useRef(false);
const springTimer = React.useRef<ReturnType<typeof setTimeout> | null>(null);
const scrollerRef = React.useRef<HTMLElement | null>(null);

const clearSpring = React.useCallback(() => {
  if (springTimer.current) clearTimeout(springTimer.current);
  springTimer.current = null;
}, []);
React.useEffect(() => clearSpring, [clearSpring]);

const pickUp = React.useCallback((p: SidebarDragPayload) => {
  dragActiveRef.current = true;
  scrollerRef.current = null; // re-measured on first move of each drag
  setDrag(p);
  setDropTarget(null);
}, []);

const moveDrag = React.useCallback((clientX: number, clientY: number) => {
  if (!dragRef.current) return;
  // Auto-scroll near the edges so a row can be dragged past the viewport.
  if (!scrollerRef.current) {
    scrollerRef.current = document.querySelector(
      "[data-radix-scroll-area-viewport]"
    ) as HTMLElement | null;
  }

```

```

    }
    const scroller = scrollerRef.current;
    if (scroller) {
      const r = scroller.getBoundingClientRect();
      if (clientY < r.top + TREE_SCROLL_EDGE_PX) scroller.scrollTop -= TREE_SCROLL_STEP_PX;
      else if (clientY > r.bottom - TREE_SCROLL_EDGE_PX)
        scroller.scrollTop += TREE_SCROLL_STEP_PX;
    }
    const el = document.elementFromPoint(clientX, clientY);
    const zone = el?.closest?("[data-drop-id]") as HTMLElement | null;
    const id = zone?.dataset.dropId ?? null;
    if (id === dropRef.current) return;
    setDropTarget(id);
    clearSpring();
    // Spring-open a collapsed folder held over, so its children take drops.
    if (id && id !== ROOT_DROP_ID) {
      const st = useWhiteboard.getState();
      const f = st.folders.find((x) => x.id === id);
      if (f && !f.expanded) {
        springTimer.current = setTimeout(() => st.toggleFolderExpanded(id), SPRING_OPEN_MS);
      }
    }
  }, [clearSpring]);

const endDrag = React.useCallback(() => {
  clearSpring();
  const d = dragRef.current;
  const t = dropRef.current;
  scrollerRef.current = null;
  dragActiveRef.current = false;
  setDrag(null);
  setDropTarget(null);
  if (!d || !t) return;
  const st = useWhiteboard.getState();
  const folder = t === ROOT_DROP_ID ? null : t;
  if (d.kind === "board") {
    const b = st.boards.find((x) => x.id === d.id);
    if (b && (b.folderId ?? null) !== folder) st.moveBoard(d.id, folder);
  } else {
    const f = st.folders.find((x) => x.id === d.id);
    // The store rejects cycles, self-drops and ghosts; the equality check
    // here just skips a pointless save for an already-correct parent.
    if (f && (f.parentId ?? null) !== folder) st.moveFolder(d.id, folder);
  }
}, [clearSpring]);

const cancelDrag = React.useCallback(() => {
  clearSpring();
  scrollerRef.current = null;
  dragActiveRef.current = false;
  setDrag(null);
  setDropTarget(null);
}, [clearSpring]);

React.useEffect(() => {
  if (!drag) return;
  const move = (e: PointerEvent) => moveDrag(e.clientX, e.clientY);
  const up = () => endDrag();
  const cancel = () => cancelDrag();
  window.addEventListener("pointermove", move);
  window.addEventListener("pointerup", up);
  window.addEventListener("pointercancel", cancel);
  return () => {
    window.removeEventListener("pointermove", move);
    window.removeEventListener("pointerup", up);
    window.removeEventListener("pointercancel", cancel);
  };
}, [drag, moveDrag, endDrag, cancelDrag]);

// A drag that outlives the app's attention (home button, power key, alert)
// would otherwise stick forever: no pointerup ever arrives, the contextmenu
// killer stays armed, and every menu in the tree stops opening. Losing focus
// cancels the drag – re-holding is cheap, a dead tree is not.
React.useEffect(() => {
  const onBlur = () => cancelDrag();
  window.addEventListener("blur", onBlur);
  return () => window.removeEventListener("blur", onBlur);
}, [cancelDrag]);

```

```

// Radix opens its context menu on a ~700ms long-press – AFTER our 300ms
// pickup. Without this, holding a row opens star/rename/duplicate/export on
// top of the drag and swallows it. A window-level CAPTURE listener fires
// before Radix ever sees the event; the ref (not state) is read so there is
// no render delay to race. Row-level `disabled` props stay as backup.
React.useEffect(() => {
  const kill = (e: Event) => {
    if (dragActiveRef.current) {
      e.preventDefault();
      e.stopPropagation();
    }
  };
  window.addEventListener("contextmenu", kill, true);
  return () => window.removeEventListener("contextmenu", kill, true);
}, []);

const dragCtx = React.useMemo(
  () => ({ drag, dropTarget, pickUp }),
  [drag, dropTarget, pickUp]
);

const rootFolders = React.useMemo(() => {
  const list = folders.filter((f) => !f.parentId || f.parentId === null);
  return sortItems(list, sidebarSortBy, sidebarSortOrder);
}, [folders, sidebarSortBy, sidebarSortOrder]);

// Filtered boards based on search, sorted by user preference
const filteredBoards = React.useMemo(() => {
  let list = boards;
  if (searchQuery.trim()) {
    const q = searchQuery.toLowerCase();
    list = boards.filter(
      (b) =>
        b.name.toLowerCase().includes(q) ||
        (Array.isArray(b.tags) && b.tags.some((t) => t.toLowerCase().includes(q)))
    );
  }
  return sortItems(list, sidebarSortBy, sidebarSortOrder);
}, [boards, searchQuery, sidebarSortBy, sidebarSortOrder]);

// Selection scope: a folder's direct contents, or the whole sidebar.
const scopeFolder = scopeFolderId ? folders.find((f) => f.id === scopeFolderId) ?? null : null;
const scopeAllIds = React.useMemo(() => {
  if (!scopeFolder) return filteredBoards.map((b) => b.id);
  const ids = boards.filter((b) => b.folderId === scopeFolder.id).map((b) => b.id);
  for (const f of folders) {
    if (f.parentId === scopeFolder.id) ids.push(FOLDER_SEL_PREFIX + f.id);
  }
  return ids;
}, [scopeFolder, boards, folders, filteredBoards]);
const allInScopeSelected =
  scopeAllIds.length > 0 && scopeAllIds.every((id) => selectedBatchIds.includes(id));
const toggleSelectAllInScope = React.useCallback(() => {
  setSelectedBatchIds((prev) => {
    const all = scopeAllIdsRef.current;
    const hasAll = all.length > 0 && all.every((id) => prev.includes(id));
    if (hasAll) return prev.filter((id) => !all.includes(id));
    return [...prev, ...all.filter((id) => !prev.includes(id))];
  });
}, []);
const scopeAllIdsRef = React.useRef(scopeAllIds);
scopeAllIdsRef.current = scopeAllIds;

/**
 * Select inside one folder: the folder opens and selection mode starts,
 * scoped to it, with NOTHING picked - you choose, or tap "Select all" (which
 * then means all of this folder). It used to pre-select everything.
 */
const selectFolderContents = React.useCallback((folderId: string) => {
  const st = useWhiteboard.getState();
  const f = st.folders.find((x) => x.id === folderId);
  if (f && !f.expanded) st.toggleFolderExpanded(folderId);
  multiSelectModeRef.current = true;
  setMultiSelectMode(true);
  setScopeFolderId(folderId);
  setSelectedBatchIds([]);
}, []);

```

```

/** Shared pickers: folder menus set the target, then open the same inputs. */
const pickPdfFile = React.useCallback((folderId: string | null) => {
  importTargetFolder.current = folderId;
  pdfInputRef.current?.click();
}, []);
const pickImageFiles = React.useCallback((folderId: string | null) => {
  importTargetFolder.current = folderId;
  imageInputRef.current?.click();
}, []);
const handlePdfFile = React.useCallback(async (e: React.ChangeEvent<HTMLInputElement>) => {
  const file = e.target.files?.[0];
  e.target.value = "";
  if (!file) return;
  setPdfBusy("Opening...");
  setPdfError(null);
  try {
    // importPdfAsNotebook opens the new notebook itself, wherever it lands.
    // PDF, Word or PowerPoint: one store action converts what needs it.
    await useWhiteboard.getState().importDocumentAsNotebook(file, importTargetFolder.current, (p) =>
      setPdfBusy(p.total > 1 ? `${p.label}...` : p.label)
    );
  } catch (err) {
    // Import can fail for real reasons – an encrypted PDF, a corrupt file,
    // no storage left – and silence would look exactly like a slow import.
    setPdfError((err as Error).message || "Could not open that PDF.");
  } finally {
    setPdfBusy(null);
    importTargetFolder.current = null;
  }
}, []);
const [importError, setImportError] = useState<string | null>(null);
const handleImageFiles = React.useCallback(async (e: React.ChangeEvent<HTMLInputElement>) => {
  const files = Array.from(e.target.files ?? []);
  e.target.value = "";
  if (files.length === 0) return;
  const target = importTargetFolder.current;
  importTargetFolder.current = null;
  setImportBusy("Adding...");
  setImportError(null);
  try {
    await importPicturesAsNotebook(files, target);
  } catch (err) {
    setImportError((err as Error).message || "Could not import those pictures.");
  } finally {
    setImportBusy(null);
  }
}, []);

return (
  <div className="h-full w-64 flex flex-col bg-sidebar border-r border-sidebar-border">
    { /* Header */ }
    <div className="flex items-center justify-between px-3 py-3 border-b border-sidebar-border">
      <div className="flex items-center gap-2">
        
        <h2 className="text-sm font-semibold tracking-tight text-sidebar-foreground">ProNotes</h2>
      </div>
      <Button variant="ghost" size="icon" className="h-7 w-7" onClick={toggleSidebar} title="Hide sidebar">
        <PanelLeftClose size={16} />
      </Button>
    </div>

    { /* Search + Sort + Multi-select + Action buttons */ }
    <div className="p-2 border-b border-sidebar-border space-y-2">
      <div className="flex items-center gap-1">
        <div className="relative flex-1">
          <Search size={12} className="absolute left-2.5 top-1/2 -translate-y-1/2 text-muted-foreground" />
          <Input
            value={searchQuery}
            onChange={(e) => setSearchQuery(e.target.value)}
            placeholder="Search..."
            className="h-7 pl-7 text-xs"
          />
        </div>
      </div>
    </div>
  </div>

```

```

{/* Sort dropdown */}
<DropdownMenu>
  <DropdownMenuTrigger asChild>
    <Button
      variant="ghost"
      size="icon"
      className="h-7 w-7 text-muted-foreground hover:text-foreground shrink-0"
      title={`Sort: ${sidebarSortBy} (${sidebarSortOrder})`}
    >
      <ArrowUpDown size={13} />
    </Button>
  </DropdownMenuTrigger>
  <DropdownMenuContent align="end" className="w-48 text-xs">
    <div className="px-2 py-1 text-[10px] font-semibold uppercase tracking-wider text-muted-foreground">
      Sort By
    </div>
    <DropdownMenuItem
      onClick={() =>
        setSidebarSort(
          "updated",
          sidebarSortBy === "updated"
            ? sidebarSortOrder === "asc"
              ? "desc"
              : "asc"
            : "desc"
        )
      }
    >
      <span className="flex-1">Last Updated</span>
      {sidebarSortBy === "updated" && (
        <span className="text-[10px] font-medium text-primary ml-1">
          {sidebarSortOrder === "desc" ? "↓ Recent" : "↑ Oldest"}
        </span>
      )}
    </DropdownMenuItem>
    <DropdownMenuItem
      onClick={() =>
        setSidebarSort(
          "created",
          sidebarSortBy === "created"
            ? sidebarSortOrder === "asc"
              ? "desc"
              : "asc"
            : "desc"
        )
      }
    >
      <span className="flex-1">Date Created</span>
      {sidebarSortBy === "created" && (
        <span className="text-[10px] font-medium text-primary ml-1">
          {sidebarSortOrder === "desc" ? "↓ Newest" : "↑ Oldest"}
        </span>
      )}
    </DropdownMenuItem>
    <DropdownMenuItem
      onClick={() =>
        setSidebarSort(
          "name",
          sidebarSortBy === "name"
            ? sidebarSortOrder === "asc"
              ? "desc"
              : "asc"
            : "asc"
        )
      }
    >
      <span className="flex-1">Name</span>
      {sidebarSortBy === "name" && (
        <span className="text-[10px] font-medium text-primary ml-1">
          {sidebarSortOrder === "asc" ? "A → Z" : "Z → A"}
        </span>
      )}
    </DropdownMenuItem>
  </DropdownMenuContent>
</DropdownMenu>

```

```

    /* Multi-select toggle. Entering never pre-selects: you choose. */
    <Button
      variant={multiSelectMode ? "secondary" : "ghost"}
      size="icon"
      className={cn(
        "h-7 w-7 shrink-0",
        multiSelectMode ? "text-primary bg-primary/15" : "text-muted-foreground hover:text-foreground"
      )}
      onClick={() => {
        if (multiSelectMode) exitSelectMode();
        else {
          setMultiSelectMode(true);
          multiSelectModeRef.current = true;
          setSelectedBatchIds([]);
          setScopeFolderId(null);
        }
      }}
      title={multiSelectMode ? "Exit multi-select mode" : "Select boards and folders"}
    >
      <CheckSquare size={13} />
    </Button>
  </div>

  {multiSelectMode ? (
    /* Selecting: ONE bar says how many, where, and offers Select all.
       New and Link step aside so nothing competes with the selection. */
    <div className="flex h-9 items-center gap-1 rounded-lg bg-primary/10 pl-1 pr-1 ring-1 ring-inset ring-
      g-primary/20">
      <button
        type="button"
        onClick={exitSelectMode}
        className="flex h-7 w-7 shrink-0 items-center justify-center rounded-md text-primary hover:bg-pr
          imary/15"
        aria-label="Stop selecting"
        title="Done"
      >
        <X size={15} />
      </button>
      <span className="min-w-0 flex-1 truncate text-[12px] font-semibold text-primary">
        {selectedBatchIds.length === 0 ? "Tap to select" : `${selectedBatchIds.length} selected`}
        {scopeFolder && <span className="font-normal opacity-80"> · {scopeFolder.name}</span>}
      </span>
      <button
        type="button"
        onClick={toggleSelectAllInScope}
        disabled={scopeAllIds.length === 0}
        className="h-7 shrink-0 rounded-md px-2 text-[11px] font-semibold text-primary hover:bg-primary/
          15 disabled:opacity-40"
      >
        {allInScopeSelected ? "Select none" : "Select all"}
      </button>
    </div>
  ) : (
    <div className="flex items-center gap-1.5">
      /* ONE New button: every creatable/importable lives in its menu. */
      <DropdownMenu>
        <DropdownMenuTrigger asChild>
          <Button size="sm" className="h-9 flex-1 gap-1.5 text-xs" title="Create or import">
            <Plus size={15} className="shrink-0" />
            <span className="font-semibold">New</span>
            <ChevronDown size={13} className="opacity-70" />
          </Button>
        </DropdownMenuTrigger>
        <DropdownMenuContent align="start" className="w-60 p-1.5">
          <NewMenuItem
            icon={<NotebookText size={16} className="text-blue-500" />}
            title="Notebook"
            hint="Pages to write on"
            onClick={() => setNameDialog({ kind: "notebook", folderId: null })}
          />
          <NewMenuItem
            icon={<Presentation size={16} className="text-violet-500" />}
            title="Board"
            hint="An endless whiteboard"
            onClick={() => setNameDialog({ kind: "board", folderId: null })}
          />
          <NewMenuItem
            icon={<FolderPlus size={16} className="text-amber-500" />}

```

```

        title="Folder"
        hint="To group your notes"
        onClick={() => setNameDialog({ kind: "folder", folderId: null })}
      />
      <DropdownMenuSeparator />
      <NewMenuItem
        icon={<FileText size={16} className="text-red-500" />}
        title={pdfBusy ?? "Open a document"}
        hint="PDF, Word or PowerPoint - write on it"
        onClick={() => pickPdfFile(null)}
        disabled={pdfBusy !== null}
      />
      <NewMenuItem
        icon={<ImagePlus size={16} className="text-emerald-500" />}
        title={importBusy ?? "Import pictures"}
        hint="Each picture becomes a page"
        onClick={() => pickImageFiles(null)}
        disabled={importBusy !== null}
      />
    </DropdownMenuContent>
  </DropdownMenu>
  {/* Linking files is a tool, not the main action: an icon button,
    where it used to be a full-width black bar louder than New. */}
  <Button
    variant="outline"
    size="icon"
    className="h-9 w-9 shrink-0"
    onClick={() => window.dispatchEvent(new CustomEvent("wb:open-link-manager"))}
    title="Link files"
    aria-label="Link files"
  >
    <Link2 size={15} />
  </Button>
</div>
)}
<input
  ref={pdfInputRef}
  type="file"
  accept={DOCUMENT_ACCEPT}
  className="hidden"
  onChange={handlePdfFile}
/>
<input
  ref={imageInputRef}
  type="file"
  accept="image/*"
  multiple
  className="hidden"
  onChange={handleImageFiles}
/>

{
  (pdfError || importError) && (
    <p className="rounded-md border border-destructive/40 bg-destructive/10 px-2 py-1 text-[11px] leading-4" >
      g-snug text-destructive">
        {pdfError ?? importError}
      </p>
    )
  }
</div>

{/* Tree */}
<ScrollArea className="flex-1">
  <SidebarDragContext.Provider value={dragCtx}>
    <div
      data-drop-id={ROOT_DROP_ID}
      className={cn(
        "p-2 space-y-0.5 no-pan-x",
        dropTarget === ROOT_DROP_ID && "rounded-lg ring-2 ring-inset ring-emerald-500/60"
      )}
    >
      {/* Starred section */}
      {!searchQuery && boards.some((b) => b.starred) && (
        <
          <SectionLabel icon={<Star size={10} className="fill-amber-400 text-amber-400" />}>Starred</SectionLabel>
          <div>
            {sortItems(
              boards.filter((b) => b.starred),
              sidebarSortBy,
              sidebarSortOrder
            )}
          </div>
        </>
      )}
    </div>
  </SidebarDragContext.Provider>
</ScrollArea>

```

```

    ).map((board) => (
      <BoardItem
        key={board.id}
        board={board}
        isActive={board.id === activeBoardId}
        onClick={() => setActiveBoard(board.id)}
        depth={0}
        multiSelectMode={multiSelectMode}
        isSelected={selectedBatchIds.includes(board.id)}
        onToggleSelect={toggleBatchSelect}
        onHoldSelect={() => holdRowSelect("board", board.id)}
        renameRequested={renameRequest?.kind === "board" && renameRequest.id === board.id}
        onRenameHandled={() => setRenameRequest(null)}
      />
    ))}
  </>
)}

{/* Folders first, then the notes that live outside any folder. Each
  gets a heading: without one, loose notes sat straight under
  "Starred" and read as starred. */}
{rootFolders.length > 0 && <SectionLabel>Folders</SectionLabel>}
{rootFolders.map((folder) => (
  <FolderItem
    key={folder.id}
    folder={folder}
    folders={folders}
    boards={searchQuery ? filteredBoards : boards}
    activeBoardId={activeBoardId}
    depth={0}
    onSelectBoard={setActiveBoard}
    multiSelectMode={multiSelectMode}
    selectedBatchIds={selectedBatchIds}
    onToggleSelectBoard={toggleBatchSelect}
    onToggleSelectFolder={toggleFolderSelect}
    onHoldSelect={holdRowSelect}
    onSelectFolderContents={selectFolderContents}
    onImportHere={(kind, folderId) =>
      kind === "pdf" ? pickPdfFile(folderId) : pickImageFiles(folderId)
    }
    onNewHere={(kind, folderId) => setNameDialog({ kind, folderId })}
    renameRequest={renameRequest}
    onRenameHandled={() => setRenameRequest(null)}
  />
)}}

{/* Notes outside any folder. While searching, starred ones must
  appear here too - the Starred section is hidden during search. */}
{(() => {
  const loose = filteredBoards.filter(
    (b) => b.folderId === null && (!b.starred || !!searchQuery.trim())
  );
  if (loose.length === 0) return null;
  return (
    <>
      <SectionLabel>{searchQuery.trim() ? "Results" : "Notes"}</SectionLabel>
      {loose.map((board) => (
        <BoardItem
          key={board.id}
          board={board}
          isActive={board.id === activeBoardId}
          onClick={() => setActiveBoard(board.id)}
          depth={0}
          multiSelectMode={multiSelectMode}
          isSelected={selectedBatchIds.includes(board.id)}
          onToggleSelect={toggleBatchSelect}
          onHoldSelect={() => holdRowSelect("board", board.id)}
          renameRequested={renameRequest?.kind === "board" && renameRequest.id === board.id}
          onRenameHandled={() => setRenameRequest(null)}
        />
      ))}
    </>
  );
})()}

{folders.length === 0 && boards.length === 0 && (
  <div className="text-xs text-muted-foreground text-center py-8">
    No whiteboards yet.
  </div>
)}

```

```

        <br />
        Create one above.
    </div>
  )}
  {searchQuery.trim() !== "" &&
    filteredBoards.length === 0 &&
    !folders.some((f) => f.name.toLowerCase().includes(searchQuery.trim().toLowerCase())) && (
      <div className="text-xs text-muted-foreground text-center py-8">
        No matches for "{searchQuery.trim()}".
      </div>
    )}
</div>
</SidebarDragContext.Provider>
</ScrollArea>

{/* Selection action bar: count, the three common actions, the rest in ■. */}
{multiSelectMode && (
  <SelectionBar
    boardIds={splitSelection(selectedBatchIds).boardIds}
    folderIds={splitSelection(selectedBatchIds).folderIds}
    boards={boards}
    folders={folders}
    busy={batchBusy}
    onDelete={() => {
      const { boardIds, folderIds } = splitSelection(selectedBatchIds);
      trashWithUndo(boardIds, folderIds);
      setSelectedBatchIds([]);
    }}
    onMove={() => setBatchMoveOpen(true)}
    onExport={runBatchExport}
    onDuplicate={() => {
      const { boardIds } = splitSelection(selectedBatchIds);
      const st = useWhiteboard.getState();
      const made = st.batchDuplicateBoards(boardIds);
      if (made.length > 0) setSelectedBatchIds([]);
    }}
    onStar={() => {
      const { boardIds } = splitSelection(selectedBatchIds);
      useWhiteboard.getState().batchStarBoards(boardIds);
    }}
    onRename={(kind, id) => setRenameRequest({ kind, id })}
    onCopyLink={(id) => {
      const b = boards.find((x) => x.id === id);
      if (b) navigator.clipboard?.writeText(`${b.name}`).catch(() => {});
    }}
  >
  </div>
)}
{batchError && (
  <p className="mx-2 mb-1 rounded-md border border-destructive/40 bg-destructive/10 px-2 py-1 text-[11px]
    ] leading-snug text-destructive">
    {batchError}
  </p>
)}

{/* Footer */}
<div className="border-t border-sidebar-border p-2 text-xs text-muted-foreground">
  Auto-saved locally
</div>

{/* Batch Move Dialog */}
<BatchMoveDialog
  open={batchMoveOpen}
  onClose={() => setBatchMoveOpen(false)}
  boardIds={splitSelection(selectedBatchIds).boardIds}
  folderIds={splitSelection(selectedBatchIds).folderIds}
  onMoved={exitSelectMode}
/>

{/* Name-before-create for the New menu */}
<NameDialog
  open={nameDialog !== null}
  title={
    nameDialog?.kind === "notebook"
      ? "New notebook"
      : nameDialog?.kind === "board"
      ? "New board"
      : "New folder"
  }
/>

```

```

    defaultName={
      nameDialog?.kind === "notebook"
        ? "Untitled Notebook"
        : nameDialog?.kind === "board"
          ? "Untitled Board"
          : "New Folder"
    }
    confirmLabel="Create"
    contextLabel={
      nameDialog?.folderId ? folders.find((f) => f.id === nameDialog.folderId)?.name ?? "Top level" : "Top
        level"
    }
  }
  onConfirm={(name) => {
    if (!nameDialog) return;
    const st = useWhiteboard.getState();
    if (nameDialog.kind === "board") setActiveBoard(st.createBoard(name, nameDialog.folderId));
    else if (nameDialog.kind === "notebook")
      setActiveBoard(st.createNotebook(name, nameDialog.folderId));
    else st.createFolder(name, nameDialog.folderId);
  }}
  onClose={() => setNameDialog(null)}
/>
</div>
);
}

// =====
// Folder item (recursive)
// =====

function FolderItem({
  folder,
  folders,
  boards,
  activeBoardId,
  depth,
  onSelectBoard,
  multiSelectMode,
  selectedBatchIds,
  onToggleSelectBoard,
  onToggleSelectFolder,
  onHoldSelect,
  onSelectFolderContents,
  onImportHere,
  onNewHere,
  renameRequest,
  onRenameHandled,
}): {
  folder: Folder;
  folders: Folder[];
  boards: Board[];
  activeBoardId: string | null;
  depth: number;
  onSelectBoard: (id: string) => void;
  multiSelectMode?: boolean;
  selectedBatchIds?: string[];
  onToggleSelectBoard?: (id: string) => void;
  onToggleSelectFolder?: (id: string) => void;
  /** Long-press entry + threaded pass-through (same callback at every level). */
  onHoldSelect?: (kind: "board" | "folder", id: string) => void;
  onSelectFolderContents?: (id: string) => void;
  onImportHere?: (kind: "pdf" | "pictures", folderId: string) => void;
  /** New notebook/board/folder in this folder - through the name dialog. */
  onNewHere?: (kind: "board" | "notebook" | "folder", folderId: string) => void;
  renameRequest?: { kind: "board" | "folder"; id: string } | null;
  onRenameHandled?: () => void;
} {
  const toggleFolderExpanded = useWhiteboard((s) => s.toggleFolderExpanded);
  const renameFolder = useWhiteboard((s) => s.renameFolder);
  const setFolderColor = useWhiteboard((s) => s.setFolderColor);
  const sidebarSortBy = useWhiteboard((s) => s.sidebarSortBy);
  const sidebarSortOrder = useWhiteboard((s) => s.sidebarSortOrder);
  const wantRename = renameRequest?.kind === "folder" && renameRequest.id === folder.id;

  const [isRenaming, setIsRenaming] = React.useState(false);
  const [renameValue, setRenameValue] = React.useState(folder.name);
  // Double-tap (pen or finger, timed) opens the full row menu, same as BoardItem.
  const [menuOpen, setMenuOpen] = React.useState(false);

```

```

const [moveOpen, setMoveOpen] = React.useState(false);
const lastTap = React.useRef(0);

// The batch bar's Rename lands here from anywhere in the tree.
React.useEffect(() => {
  if (wantRename && !isRenaming) {
    setRenameValue(folder.name);
    setIsRenaming(true);
    onRenameHandled?.();
  }
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [wantRename]);

const isFolderSelected = selectedBatchIds?.includes(FOLDER_SEL_PREFIX + folder.id) ?? false;

const { drag, dropTarget, pickUp } = React.useContext(SidebarDragContext);
const { suppressClick, holdHandlers } = useHoldRow({
  onHold: () => onHoldSelect?.("folder", folder.id),
  onDragStart: () => pickUp({ kind: "folder", id: folder.id }),
});
const isDragged = drag?.id === folder.id;
// Dropping onto itself or its own subtree is rejected by the store; do not
// promise it with a highlight either.
const invalidTarget =
  !!drag &&
  drag.kind === "folder" &&
  (drag.id === folder.id || isDescendantFolder(folders, drag.id, folder.id));
const isDropTarget = dropTarget === folder.id && !invalidTarget;

const childFolders = React.useMemo(() => {
  const list = folders.filter((f) => f.parentId === folder.id);
  return sortItems(list, sidebarSortBy, sidebarSortOrder);
}, [folders, folder.id, sidebarSortBy, sidebarSortOrder]);

const childBoards = React.useMemo(() => {
  const list = boards.filter((b) => b.folderId === folder.id);
  return sortItems(list, sidebarSortBy, sidebarSortOrder);
}, [boards, folder.id, sidebarSortBy, sidebarSortOrder]);

const commitRename = () => {
  const trimmed = renameValue.trim();
  if (trimmed && trimmed !== folder.name) {
    renameFolder(folder.id, trimmed);
  } else {
    setRenameValue(folder.name);
  }
  setIsRenaming(false);
};

return (
  <>
    <ContextMenu>
      <ContextMenuTrigger asChild disabled={!isDragged}>
        <div data-drop-id={folder.id}>
          {/* Folder row (select-none: a hold must pick the row up, not select its name) */}
          <div
            {...holdHandlers}
            className={cn(
              "flex items-center gap-1 px-2 py-1.5 rounded-md hover:bg-sidebar-accent cursor-pointer select-no",
              "ne text-sm group",
              isDragged && "opacity-40",
              isDropTarget && "ring-2 ring-inset ring-emerald-500/70 bg-emerald-500/10",
              isFolderSelected &&
              multiSelectMode &&
              "bg-primary/10 text-primary font-medium ring-1 ring-inset ring-primary/30"
            )}
          >
            style={{ paddingLeft: `${depth * 12 + 8}px` }}
            onClick={() => {
              if (suppressClick.current) {
                suppressClick.current = false;
                return;
              }
              if (multiSelectMode) {
                onToggleSelectFolder?.(folder.id);
                return;
              }
              const now = Date.now();
              if (now - lastTap.current < DOUBLE_TAP_MS) {

```

```

        lastTap.current = 0;
        setMenuOpen(true);
        return;
    }
    lastTap.current = now;
    toggleFolderExpanded(folder.id);
  }}
  onContextMenu={(e) => {
    if (drag) e.preventDefault();
  }}
}
>
{multiSelectMode && (
  <button
    type="button"
    onClick={(e) => {
      e.stopPropagation();
      onToggleSelectFolder?.(folder.id);
    }}
    data-hold-ignore
    className="shrink-0 p-0.5 text-primary hover:text-primary/80 focus:outline-none"
    aria-label={isFolderSelected ? `Deselect folder ${folder.name}` : `Select folder ${folder.name}`}
  >
    {isFolderSelected ? (
      <CheckSquare size={14} className="text-primary fill-primary/15" />
    ) : (
      <Square size={14} className="text-muted-foreground/60 hover:text-foreground" />
    )}
  </button>
)}
{folder.expanded ? (
  <ChevronDown size={14} className="text-muted-foreground shrink-0" />
) : (
  <ChevronRight size={14} className="text-muted-foreground shrink-0" />
)}
{folder.expanded ? (
  <FolderOpen size={14} className="shrink-0" style={{ color: folder.color || "#f59e0b" }} />
) : (
  <FolderIcon size={14} className="shrink-0" style={{ color: folder.color || "#f59e0b" }} />
)}
{isRenaming ? (
  <Input
    value={renameValue}
    onChange={(e) => setRenameValue(e.target.value)}
    onBlur={commitRename}
    onKeyDown={(e) => {
      if (e.key === "Enter") commitRename();
      if (e.key === "Escape") {
        setRenameValue(folder.name);
        setIsRenaming(false);
      }
    }}
    onClick={(e) => e.stopPropagation()}
    data-hold-ignore
    className="h-5 text-xs px-1 py-0"
    autoFocus
  />
) : (
  <span className="min-w-0 flex-1 truncate text-sidebar-foreground">{folder.name}</span>
)}
<DropdownMenu open={menuOpen} onOpenChange={setMenuOpen}>
  <DropdownMenuTrigger asChild>
    <Button
      variant="ghost"
      size="icon"
      data-hold-ignore
      className={cn("h-5 w-5 opacity-0 group-hover:opacity-100 show-on-touch shrink-0", multiSelectMode && "hidden")}
      onClick={(e) => e.stopPropagation()}
    >
      <MoreHorizontal size={12} />
    </Button>
  </DropdownMenuTrigger>
  <DropdownMenuContent align="end" className="w-56" onClick={(e) => e.stopPropagation()}>
    <DropdownMenuItem onClick={() => onSelectFolderContents?.(folder.id)}>
      <CheckSquare size={12} className="mr-2" /> Select in this folder
    </DropdownMenuItem>
    <DropdownMenuItem onClick={() => onImportHere?.("pdf", folder.id)}>

```

```

      <FileUp size={12} className="mr-2" /> Import document here
    </DropdownMenuItem>
    <DropdownMenuItem onClick={() => onImportHere?.("pictures", folder.id)}>
      <ImagePlus size={12} className="mr-2" /> Import pictures here
    </DropdownMenuItem>
    <DropdownMenuSeparator />
    <DropdownMenuItem onClick={() => onNewHere?.("notebook", folder.id)}>
      <NotebookText size={12} className="mr-2" /> New notebook here
    </DropdownMenuItem>
    <DropdownMenuItem onClick={() => onNewHere?.("board", folder.id)}>
      <Presentation size={12} className="mr-2" /> New board here
    </DropdownMenuItem>
    <DropdownMenuItem onClick={() => onNewHere?.("folder", folder.id)}>
      <FolderPlus size={12} className="mr-2" /> New subfolder
    </DropdownMenuItem>
    <DropdownMenuItem onClick={() => setMoveOpen(true)}>
      <FolderInput size={12} className="mr-2" /> Move to...
    </DropdownMenuItem>
    <DropdownMenuItem onClick={() => setIsRenaming(true)}>
      <Pencil size={12} className="mr-2" /> Rename
    </DropdownMenuItem>
    <DropdownMenuSeparator />
    <div className="px-2 py-1.5">
      <div className="text-xs text-muted-foreground mb-1.5">Folder color</div>
      <div className="flex gap-1.5">
        {FOLDER_COLORS.map((c) => (
          <button
            key={c}
            className={cn(
              "w-5 h-5 rounded-full border-2 transition-transform hover:scale-110",
              (folder.color || "#f59e0b") === c ? "border-ring ring-1 ring-ring/30" : "border-whit
e"
            )}
            style={{ backgroundColor: c }}
            onClick={() => setFolderColor(folder.id, c)}
          />
        ))}
      </div>
    </div>
    <DropdownMenuSeparator />
    <DropdownMenuItem
      className="text-destructive focus:text-destructive"
      onClick={() => trashWithUndo([], [folder.id])}
      title="Deletes the folder; its boards move to the top level. Undo restores everything."
    >
      <Trash2 size={12} className="mr-2" /> Delete
    </DropdownMenuItem>
  </DropdownMenuContent>
</DropdownMenu>
</div>

{/* Children (if expanded) */}
{folder.expanded && (
  <div>
    {childBoards.map((board) => (
      <BoardItem
        key={board.id}
        board={board}
        isActive={board.id === activeBoardId}
        onClick={() => onSelectBoard(board.id)}
        depth={depth + 1}
        multiSelectMode={multiSelectMode}
        isSelected={selectedBatchIds?.includes(board.id)}
        onToggleSelect={onToggleSelectBoard}
        onHoldSelect={onHoldSelect ? () => onHoldSelect("board", board.id) : undefined}
        renameRequested={renameRequest?.kind === "board" && renameRequest.id === board.id}
        onRenameHandled={onRenameHandled}
      />
    ))}
    {childFolders.map((childFolder) => (
      <FolderItem
        key={childFolder.id}
        folder={childFolder}
        folders={folders}
        boards={boards}
        activeBoardId={activeBoardId}
        depth={depth + 1}
        onSelectBoard={onSelectBoard}

```

```

        multiSelectMode={multiSelectMode}
        selectedBatchIds={selectedBatchIds}
        onToggleSelectBoard={onToggleSelectBoard}
        onToggleSelectFolder={onToggleSelectFolder}
        onHoldSelect={onHoldSelect}
        onSelectFolderContents={onSelectFolderContents}
        onImportHere={onImportHere}
        onNewHere={onNewHere}
        renameRequest={renameRequest}
        onRenameHandled={onRenameHandled}
      />
    )))
    {childBoards.length === 0 && childFolders.length === 0 && (
      <div
        className="text-xs text-muted-foreground italic py-1"
        style={{ paddingLeft: `${(depth + 1) * 12 + 24}px` }}
      >
        Empty folder
      </div>
    )}
  </div>
)
</div>
</ContextMenuTrigger>
<ContextMenuContent onClick={(e) => e.stopPropagation()}>
  <ContextMenuItems onClick={() => onSelectFolderContents?.(folder.id)}>
    <CheckSquare size={12} className="mr-2" /> Select in this folder
  </ContextMenuItems>
  <ContextMenuItems onClick={() => onImportHere?.("pdf", folder.id)}>
    <FileUp size={12} className="mr-2" /> Import document here
  </ContextMenuItems>
  <ContextMenuItems onClick={() => onImportHere?.("pictures", folder.id)}>
    <ImagePlus size={12} className="mr-2" /> Import pictures here
  </ContextMenuItems>
  <ContextMenuSeparator />
  <ContextMenuItems onClick={() => onNewHere?.("notebook", folder.id)}>
    <NotebookText size={12} className="mr-2" /> New notebook here
  </ContextMenuItems>
  <ContextMenuItems onClick={() => onNewHere?.("board", folder.id)}>
    <Presentation size={12} className="mr-2" /> New board here
  </ContextMenuItems>
  <ContextMenuItems onClick={() => onNewHere?.("folder", folder.id)}>
    <FolderPlus size={12} className="mr-2" /> New subfolder
  </ContextMenuItems>
  <ContextMenuItems onClick={() => setMoveOpen(true)}>
    <FolderInput size={12} className="mr-2" /> Move to...
  </ContextMenuItems>
  <ContextMenuItems onClick={() => setIsRenaming(true)}>
    <Pencil size={12} className="mr-2" /> Rename
  </ContextMenuItems>
  <ContextMenuSeparator />
  <ContextMenuItems
    className="text-destructive focus:text-destructive"
    onClick={() => trashWithUndo([], [folder.id])}
  >
    <Trash2 size={12} className="mr-2" /> Delete
  </ContextMenuItems>
</ContextMenuContent>
</ContextMenu>
/* Dialogs exist only while open. Each listens to the whole workspace,
   and one per row meant ~40 closed dialogs re-rendering after every
   pen stroke. */
{moveOpen && (
  <MoveToDialog
    open={moveOpen}
    onClose={() => setMoveOpen(false)}
    item={{ kind: "folder", id: folder.id, name: folder.name }}
    currentParentId={folder.parentId ?? null}
  />
)}
</>
);
}

// =====
// Board item
// =====

```

```

export type BoardItemProps = {
  board: Board;
  isActive: boolean;
  onClick: () => void;
  depth: number;
  multiSelectMode?: boolean;
  isSelected?: boolean;
  onToggleSelect?: (id: string) => void;
  onHoldSelect?: () => void;
  renameRequested?: boolean;
  onRenameHandled?: () => void;
};

/**
 * A row re-draws only when what it SHOWS changes. The sidebar listens to the
 * whole workspace, so every pen stroke used to re-draw every notebook row -
 * each with its own menus - which measured ~240 ms after EVERY stroke on a
 * real 20-notebook workspace: the freeze after each pen lift that made fast
 * writing lag and drop the start of the next letter.
 * `onClick` is left out: it always opens this same board. So are the
 * callbacks (`onToggleSelect`, `onHoldSelect`, `onRenameHandled`): they are
 * stable per row purpose, never compared.
 */
export function boardItemPropsEqual(a: BoardItemProps, b: BoardItemProps): boolean {
  if (a.isActive !== b.isActive || a.depth !== b.depth) return false;
  if (a.multiSelectMode !== b.multiSelectMode || a.isSelected !== b.isSelected) return false;
  if ((a.renameRequested ?? false) !== (b.renameRequested ?? false)) return false;
  const x = a.board;
  const y = b.board;
  if (x === y) return true;
  return (
    x.id === y.id &&
    x.name === y.name &&
    x.folderId === y.folderId &&
    x.starred === y.starred &&
    x.color === y.color &&
    x.kind === y.kind &&
    (x.tags ?? []).join("\u0000") === (y.tags ?? []).join("\u0000")
  );
}

const BoardItem = React.memo(BoardItemImpl, boardItemPropsEqual);

function BoardItemImpl({
  board,
  isActive,
  onClick,
  depth,
  multiSelectMode,
  isSelected,
  onToggleSelect,
  onHoldSelect,
  renameRequested,
  onRenameHandled,
}: BoardItemProps) {
  const renameBoard = useWhiteboard((s) => s.renameBoard);
  const duplicateBoard = useWhiteboard((s) => s.duplicateBoard);
  const convertToNotebook = useWhiteboard((s) => s.convertToNotebook);
  const setSplitBoard = useWhiteboard((s) => s.setSplitBoard);
  const splitBoardId = useWhiteboard((s) => s.splitBoardId);
  const activeBoardId = useWhiteboard((s) => s.activeBoardId);
  const toggleStarred = useWhiteboard((s) => s.toggleStarred);
  const setBoardColor = useWhiteboard((s) => s.setBoardColor);
  const setActiveBoard = useWhiteboard((s) => s.setActiveBoard);

  const [isRenaming, setIsRenaming] = React.useState(false);
  const [renameValue, setRenameValue] = React.useState(board.name);
  // Double-tap (pen or finger, timed - mobile WebViews never fire dblclick)
  // opens the full row menu: move, delete, rename, duplicate and the rest.
  const [menuOpen, setMenuOpen] = React.useState(false);
  const [moveOpen, setMoveOpen] = React.useState(false);
  const [duplicateModalOpen, setDuplicateModalOpen] = React.useState(false);
  const lastTap = React.useRef(0);

  // The batch bar's Rename lands here from anywhere in the tree.
  React.useEffect(() => {
    if (renameRequested && !isRenaming) {
      setRenameValue(board.name);
    }
  }, [renameRequested, isRenaming, board.name]);

```

```

    setIsRenaming(true);
    onRenameHandled?();
  }
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [renameRequested]);

const { drag, pickUp } = React.useContext(SidebarDragContext);
const { suppressClick, holdHandlers } = useHoldRow({
  onHold: () => onHoldSelect?(),
  onDragStart: () => pickUp({ kind: "board", id: board.id }),
});
const isDragged = drag?.id === board.id;

// Copy a [[wikilink]] to this board to the clipboard, so it can be pasted
// into any text/sticky element to create a link.
const copyBoardLink = (b: Board) => {
  const linkText = `[${b.name}]`;
  navigator.clipboard?.writeText(linkText).catch(() => {
    // Fallback: do nothing if clipboard is unavailable
  });
};

const commitRename = () => {
  const trimmed = renameValue.trim();
  if (trimmed && trimmed !== board.name) {
    renameBoard(board.id, trimmed);
  } else {
    setRenameValue(board.name);
  }
  setIsRenaming(false);
};

return (
  <>
    <ContextMenu>
      <ContextMenuTrigger asChild disabled={!drag}>
        <div>
          {...holdHandlers}
          // Pass-through target: dropping onto a board files it into that
          // board's folder (boards cannot contain anything).
          data-drop-id={board.folderId ?? ROOT_DROP_ID}
          className={cn(
            "flex items-center gap-1 px-2 py-1.5 rounded-md cursor-pointer select-none text-sm group",
            isActive && !multiSelectMode
              ? "bg-sidebar-accent text-sidebar-accent-foreground font-medium"
              : isSelected && multiSelectMode
                ? "bg-primary/10 text-primary font-medium ring-1 ring-inset ring-primary/30"
                : "hover:bg-sidebar-accent/50 text-sidebar-foreground",
            isDragged && "opacity-40"
          )}
          // While selecting, every checkbox sits in one column: a board row
          // drops the indent that lines its icon up with a folder's name.
          style={{ paddingLeft: `${depth * 12 + (multiSelectMode ? 8 : 24)}px` }}
          onClick={() => {
            if (multiSelectMode) {
              onToggleSelect?.(board.id);
              return;
            }
            if (suppressClick.current) {
              suppressClick.current = false;
              return;
            }
            const now = Date.now();
            if (now - lastTap.current < DOUBLE_TAP_MS) {
              lastTap.current = 0;
              setMenuOpen(true);
              return;
            }
            lastTap.current = now;
            onClick();
          }}
          onContextMenu={(e) => {
            if (drag) e.preventDefault();
          }}
        >
        {multiSelectMode && (
          <button
            type="button"

```

```

      onClick={(e) => {
        e.stopPropagation();
        onToggleSelect?.(board.id);
      }}
      data-hold-ignore
      className="shrink-0 p-0.5 text-primary hover:text-primary/80 focus:outline-none"
      aria-label={isSelected ? `Deselect ${board.name}` : `Select ${board.name}`}
    >
      {isSelected ? (
        <CheckSquare size={14} className="text-primary fill-primary/15" />
      ) : (
        <Square size={14} className="text-muted-foreground/60 hover:text-foreground" />
      )}
    </button>
  )}
<BoardKindIcon board={board} />
{isRenaming ? (
  <Input
    value={renameValue}
    onChange={(e) => setRenameValue(e.target.value)}
    onBlur={commitRename}
    onKeyDown={(e) => {
      if (e.key === "Enter") commitRename();
      if (e.key === "Escape") {
        setRenameValue(board.name);
        setIsRenaming(false);
      }
    }}
    onClick={(e) => e.stopPropagation()}
    data-hold-ignore
    className="h-5 text-xs px-1 py-0"
    autoFocus
  />
) : (
  <span className="min-w-0 flex-1 truncate">{board.name}</span>
)}
{board.starred && (
  <Star size={10} className="fill-amber-400 text-amber-400 shrink-0" />
)}
{Array.isArray(board.tags) && board.tags.length > 0 && (
  <span className="text-[9px] text-emerald-600 shrink-0">#{board.tags.length}</span>
)}
<PageCount id={board.id} />
<DropdownMenu open={menuOpen} onOpenChange={setMenuOpen}>
  <DropdownMenuTrigger asChild>
    <Button
      variant="ghost"
      size="icon"
      data-hold-ignore
      className={cn("h-5 w-5 opacity-0 group-hover:opacity-100 show-on-touch shrink-0", multiSelectMode && "hidden")}
      onClick={(e) => e.stopPropagation()}
    >
      <MoreHorizontal size={12} />
    </Button>
  </DropdownMenuTrigger>
  <DropdownMenuContent align="end" className="w-48" onClick={(e) => e.stopPropagation()}>
    <DropdownMenuItem onClick={() => toggleStarred(board.id)}>
      <Star size={12} className="mr-2" /> {board.starred ? "Unstar" : "Star"}
    </DropdownMenuItem>
    <DropdownMenuItem onClick={() => setIsRenaming(true)}>
      <Pencil size={12} className="mr-2" /> Rename
    </DropdownMenuItem>
    <DropdownMenuItem
      onClick={() => {
        if (isNotebook(board)) {
          setDuplicateModalOpen(true);
        } else {
          duplicateBoard(board.id);
        }
      }}
    >
      <Copy size={12} className="mr-2" /> Duplicate
    </DropdownMenuItem>
    <DropdownMenuItem onClick={() => setMoveOpen(true)}>
      <FolderInput size={12} className="mr-2" /> Move to...
    </DropdownMenuItem>
  </DropdownMenuContent>
  {board.id !== activeBoardId && (

```

```

      <DropdownMenuItem
        onClick={() => setSplitBoard(splitBoardId === board.id ? null : board.id)}
      >
        <Columns2 size={12} className="mr-2" />
        {splitBoardId === board.id ? "Close split view" : "Open beside this one"}
      </DropdownMenuItem>
    )}
    {board.kind !== "notebook" && (
      <DropdownMenuItem onClick={() => convertToNotebook(board.id)}>
        <BookOpen size={12} className="mr-2" /> Convert to Notebook
      </DropdownMenuItem>
    )}
    <DropdownMenuItem onClick={() => copyBoardLink(board)}>
      <Link2 size={12} className="mr-2" /> Copy link
    </DropdownMenuItem>
    <DropdownMenuItem onClick={() => {
      setActiveBoard(board.id);
      window.dispatchEvent(new CustomEvent("wb:open-export"));
    }}>
      <Download size={12} className="mr-2" /> Export Board
    </DropdownMenuItem>
    <DropdownMenuSeparator />
    <div className="px-2 py-1.5">
      <div className="text-xs text-muted-foreground mb-1.5">Board color</div>
      <div className="flex gap-1.5">
        {FOLDER_COLORS.map((c) => (
          <button
            key={c}
            className={cn(
              "w-5 h-5 rounded-full border-2 transition-transform hover:scale-110",
              (board.color || "#3b82f6") === c ? "border-ring ring-1 ring-ring/30" : "border-white"
            )}
            style={{ backgroundColor: c }}
            onClick={() => setBoardColor(board.id, c)}
          />
        ))}
      </div>
    </div>
    <DropdownMenuSeparator />
    <DropdownMenuItem
      className="text-destructive focus:text-destructive"
      onClick={() => trashWithUndo([board.id], [])}
    >
      <Trash2 size={12} className="mr-2" /> Delete
    </DropdownMenuItem>
  </DropdownMenuContent>
</DropdownMenu>
</div>
</ContextMenuTrigger>
<ContextMenuContent onClick={(e) => e.stopPropagation()}>
  <ContextMenuItems onOpen={() => toggleStarred(board.id)}>
    <Star size={12} className="mr-2" /> {board.starred ? "Unstar" : "Star"}
  </ContextMenuItems>
  <ContextMenuItems onOpen={() => setIsRenaming(true)}>
    <Pencil size={12} className="mr-2" /> Rename
  </ContextMenuItems>
  <ContextMenuItems onOpen={() => {
    if (isNotebook(board)) {
      setDuplicateModalOpen(true);
    } else {
      duplicateBoard(board.id);
    }
  }}>
    <Copy size={12} className="mr-2" /> Duplicate
  </ContextMenuItems>
  <ContextMenuItems onOpen={() => setMoveOpen(true)}>
    <FolderInput size={12} className="mr-2" /> Move to...
  </ContextMenuItems>
  {board.id !== activeBoardId && (
    <ContextMenuItems onOpen={() => setSplitBoard(splitBoardId === board.id ? null : board.id)}>
      <Columns2 size={12} className="mr-2" />
      {splitBoardId === board.id ? "Close split view" : "Open beside this one"}
    </ContextMenuItems>
  )}
</ContextMenuContent>

```

```

    {board.kind !== "notebook" && (
      <ContextMenuItem onClick={() => convertToNotebook(board.id)}>
        <BookOpen size={12} className="mr-2" /> Convert to Notebook
      </ContextMenuItem>
    )}
    <ContextMenuItem onClick={() => {
      setActiveBoard(board.id);
      window.dispatchEvent(new CustomEvent("wb:open-export"));
    }}>
      <Download size={12} className="mr-2" /> Export Board
    </ContextMenuItem>
    <ContextMenuSeparator />
    <ContextMenuItem
      className="text-destructive focus:text-destructive"
      onClick={() => trashWithUndo([board.id], [])}
    >
      <Trash2 size={12} className="mr-2" /> Delete
    </ContextMenuItem>
  </ContextMenuContent>
</ContextMenu>
{ /* Dialogs exist only while open. Each listens to the whole workspace,
   and one per row meant ~40 closed dialogs re-rendering after every
   pen stroke. */}
moveOpen && (
  <MoveToDialog
    open={moveOpen}
    onClose={() => setMoveOpen(false)}
    item={{ kind: "board", id: board.id, name: board.name }}
    currentParentId={board.folderId ?? null}
  />
)
{isNotebook(board) && duplicateModalOpen && (
  <DuplicateNotebookModal
    open={duplicateModalOpen}
    onClose={() => setDuplicateModalOpen(false)}
    // The row's `board` is a list snapshot; the dialog previews pages,
    // so it gets the notebook as it is now.
    sourceBoard={useWhiteboard.getState().boards.find((b) => b.id === board.id) ?? board}
  />
)
}
</>
);
}

```

```

=====
// === END FILE: src/components/whiteboard/Sidebar.tsx ===
=====

```

```

=====
// === START FILE: src/components/whiteboard/SplitPane.tsx ===
// File 79 of 194 | Lines: 626 | Size: 24977 bytes
=====

```

```

"use client";

/**
 * Split view
 * =====
 * Two documents side by side: the lecture PDF on one side, your notes on the
 * other. Either side can be written on.
 *
 * The store keeps ONE live document (`activeBoardId`) with its camera,
 * selection and editing target. The other half is drawn here. Touching it,
 * scrolling it or turning its page makes it the live one - the note you last
 * acted on is the note every tool, colour, paper and setting applies to.
 *
 * The invariant this file exists to keep
 * -----
 * Switching which side is live must not move either document. Not a page, not
 * a pixel. Three things used to break that, all fixed here:
 *
 * 1. Different pictures from the same camera. The live canvas draws a notebook
 * as a continuous strip anchored on its active page; this pane drew a
 * single sheet at y = 0. Both now use `visibleSheets` / `reanchorDirection`
 * from notebook.ts, so one camera means one picture.
 *
 * 2. Different rules for a legal camera. The live canvas clamps to the paper
 * (`clampNotebookCamera`) and floors the zoom (`minNotebookZoom`); this

```